F2K_CUDA

From Diffusion Theory to a
Blackwell GPU Implementation

by Chris Hebert and Claude

A long-form course on how FLUX.2-klein works and how it is implemented, from scratch, in C++/CUDA/cuDNN for an NVIDIA GB10.

Dedicated to Mr Bojangles

a dog like no other

Foreword

by Rocket

woof woof grrrrr woof

Contents

F2K_CUDA — From Diffusion Theory to a Blackwell GPU Implementation

A long-form course on how FLUX.2-klein works and how it is implemented, from scratch, in C++/CUDA/cuDNN for an NVIDIA GB10 (Blackwell) GPU.

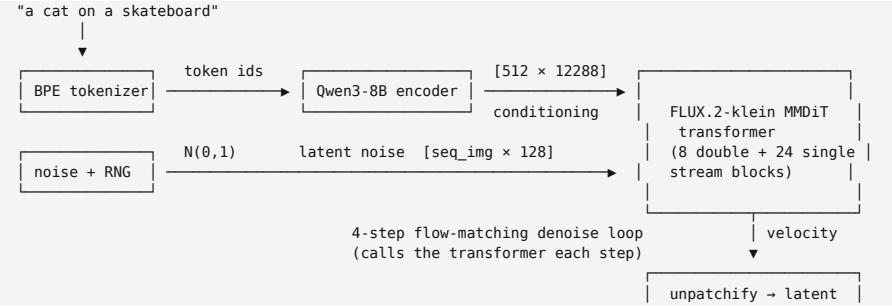
Who this is for

You are comfortable with C++ and CUDA at a systems level — kernels, shared memory, warps, occupancy — but you want to **understand the machine learning**: what a diffusion transformer actually is, why FLUX.2-klein is shaped the way it is, and how every mathematical object in the model becomes a tensor-core instruction on a real GPU.

The course assumes no prior machine-learning background. It does assume you can read CUDA C++ without hand-holding. Where we discuss GPU mechanics we go deep; where we discuss the math we build it up from first principles.

What this teaches

By the end you will be able to explain, and modify, **every stage** of the path from a text prompt to a 1024×1024 PNG:



Concretely, you will understand: rectified-flow diffusion and why it samples in 4 steps; the MMDiT architecture (joint text/image attention, AdaLN modulation, 4-axis RoPE); the Qwen3 text encoder; the VAE; and on the implementation side: NVFP4/MXFP8 block-scaled quantization, CUTLASS tensor-core GEMM, a flash-attention kernel taken all the way to peak HBM bandwidth with `mma.sync`/`ldmatrix`, the cuDNN-9 graph API for convolutions, a from-scratch byte-level BPE tokenizer, and the orchestration that ties it together. Part VIII then goes *beyond* text-to-image: the **VAE encoder** closes the autoencoder loop, which unlocks **img2img, inpainting, and upscaling** (all one idea — denoise from a noised real latent), and a **persistent worker + streaming web UI** serve it interactively, watching the image form live from a phone.

The target machine

Everything here targets one specific computer, and the choices only make sense in that light:

```
 |
GPU | NVIDIA GB10 (Blackwell), compute capability sm_121
Compute | 5th-gen tensor cores; ~1 PFLOP/s sparse FP4
Memory | 128 GB unified LPDDR5X @ ~273 GB/s, shared CPU+GPU (NVLink-C2C,
cache-coherent)
SMs | 48, ~99 KiB opt-in shared memory per block
CPU | 10× Cortex-X925 + 10× Cortex-A725 (ARM, aarch64)
Toolchain | CUDA 13.0, cuDNN 9, g++ 13, CMake
```

The single most important fact: **1 PFLOP/s of FP4 divided by 273 GB/s means this machine is overwhelmingly memory-bandwidth-bound for inference.** That ratio drive s nearly every implementation decision — quantize weights to shrink bytes moved, keep data on-chip, and treat a kernel as "done" when it saturates HBM, not when it saturates the

tensor cores. Chapter 11 makes this quantitative.

How the course is organized

It is split into parts. Each *theory* chapter is followed by the *implementation* that realizes it in this repo, with direct references to source files and the tests that prove them correct. You can read straight through, or use the math chapters as reference and live in the implementation chapters.

Part 0 — Orientation

- **00 — Overview & the big picture** — the dataflow end to end, the repo map, the hardware, and how to build and run.

Part I — The mathematics of generation

- **01 — Diffusion from first principles** — forward/reverse processes, score, why iteratively denoising noise produces samples.
- **02 — Flow matching & rectified flow** — the objective FLUX actually trains on, velocity prediction, the sampler/scheduler, and why a *distilled* model needs only 4 steps.
- **03 — Latent diffusion & the VAE** — why we diffuse in latent space, the autoencoder, and the FLUX VAE's channel/patch layout.

Part II — The FLUX.2-klein architecture

- **04 — Diffusion transformers (MMDiT)** — patches as tokens, the two-stream idea.
- **05 — Text conditioning with Qwen3** — why a 8B LLM is the text encoder, and the three-layer hidden-state concatenation.
- **06 — Rotary position embeddings** — RoPE, and the

4-axis variant for 2D images.

- **07 — Modulation / AdaLN** — turning a timestep into per-block scale/shift/gate.
- **08 — Double-stream blocks** — joint attention between text and image streams.
- **09 — Single-stream blocks** — the fused QKV+MLP layers.
- **10 — Assembling the transformer** — embedders, the block stack, final projection.

Part III — GPU implementation foundations

- **11 — The Blackwell target & the roofline** — sm_121, unified memory, and making "bandwidth-bound" quantitative.
- **12 — Numerical formats & quantization** — BF16, FP8 (E4M3/MXFP8), NVFP4 (E2M1 + block scales): what they are, why, and the measured quality tradeoffs.
- **13 — Tensor-core GEMM with CUTLASS** — block-scaled MMA, `fp4_gemm`/`fp8_gemm`, and the `Linear` class that wraps them.
- **14 — Weights on disk: the F2K format, loader & router** — a custom mmap-and-go container, multi-shard loading, and name→role tensor routing.
- **15 — The core kernel library** — RMSNorm, LayerNorm, GroupNorm, SiLU/SwiGLU, modulation, patchify, RoPE: small kernels, written once, reused everywhere.

Part IV — Attention, in depth

- **16 — Attention and the memory wall** — the math, and why naïve attention is a bandwidth disaster.
- **17 — The kernel journey** — row-at-a-time → flash (online softmax) → WMMA tensor cores → register-resident O (occupancy) → `mma.sync`+`ldmatrix` (peak bandwidth). The full optimization story with real measurements (56.5 → 19.7 ms).

- **18 — `mma.sync` & `ldmatrix` up close** — fragment thread→register layouts, the inline PTX, and how we validated it bit-exactly.

Part V — The VAE decoder on the GPU

- **19 — Convolutions with the cuDNN 9 graph API** — why the legacy API misses Blackwell tensor cores, NHWC, and engine selection (18.7 s → 1.35 s).
- **20 — Assembling the decoder** — resnet/groupnorm/upsample/attention, and the `post_quant_conv` and batch-norm de-normalization subtleties.

Part VI — Text encoder & tokenizer

- **21 — The Qwen3 encoder** — grouped-query attention, per-head q/k norm, and the capture-layer extraction.
- **22 — A native byte-level BPE tokenizer** — bytes-to-unicode, the GPT-2/Qwen pretokenizer regex, BPE merges, and the chat template — matching HF bit-for-bit.

Part VII — Integration & engineering practice

- **23 — The end-to-end pipeline** — `generate.cu` start to finish.
- **24 — Performance methodology** — profiling, occupancy analysis, the roofline, and how each bottleneck was actually found.
- **25 — Validation methodology** — golden tests against diffusers, cosine similarity, bit-exactness, and the debugging sagas that hardened the pipeline.

Part VIII — Image conditioning & serving

- **26 — The VAE encoder** — the mirror of Chapter 20 that closes the autoencoder loop (image → latent), the asymmetric downsample pad, verified at cos 0.9997.
- **27 — Image conditioning** — img2img, inpainting, and

upscaling as one idea: start the flow-matching denoise from a *noised real latent*. Strength schedules, RePaint masking, the hi-res pass.

- **28 — Serving & live preview** — the persistent worker (resident models, a line protocol), the web UI over Tailscale, and streaming the image as it forms.

Appendices

- **A — Glossary** of every term and acronym.
- **B — The bug museum** — the real bugs (modulation scale/shift swap, the VAE "mush", the FP4 layout trap, the `gridDim.y` 2K wall, the streaming `SIGPIPE`) and what each one teaches.
- **C — Build & reproduce** — toolchain, CMake, running the tests, regenerating every figure and number in the course.

A note on sourcing

Every implementation claim is anchored to a file in this repository and, where possible, to a passing test under `tests/`. Performance numbers were measured on the GB10 described above with `tools/bench_attention.cu` and `tools/generate.cu`. When the course says "we measured X," there is a way to reproduce X in Appendix C.

> Status: **complete** — all 29 chapters (00–28) and three appendices (A–C) are written, each anchored to the source and verified against it. Parts 0–VII cover text-to-image; Part VIII adds the VAE encoder, image conditioning (img2img / inpainting / upscaling), and the serving layer with live preview.

Chapter 00 — Overview & the Big Picture

> *Goal of this chapter:* build an accurate mental model of the whole system — what data flows where, which file does what, and what the hardware constraints are — so that every later chapter has a place to attach. Nothing here is deep; everythin g here is a map.

0.1 What the program does, in one sentence

Given a text prompt and a random seed, `generate` produces an image by **starting from pure Gaussian noise in a compressed "latent" space and repeatedly nudging it t oward the prompt**, then decoding the cleaned-up latent into pixels.

That sentence hides three different neural networks and a sampling loop. The rest of the course unpacks it. This chapter just names the pieces.

0.2 The three models

FLUX.2-klein is not one model; it is a *pipeline* of three, each doing a distinct job. This is the single most important structural fact to internalize early.

1. **The text encoder (Qwen3-8B).** A large language model, used here not to generate text but to *read* the prompt and turn it into a numerical summary — a `[512 × 12288]` tensor of "conditioning" that the image model can attend to. Surprisingly, FLUX.2-klein's text encoder is a full 8-billion-parameter LLM. (Why an LLM and not a smaller text model is the subject of Chapter 05.)

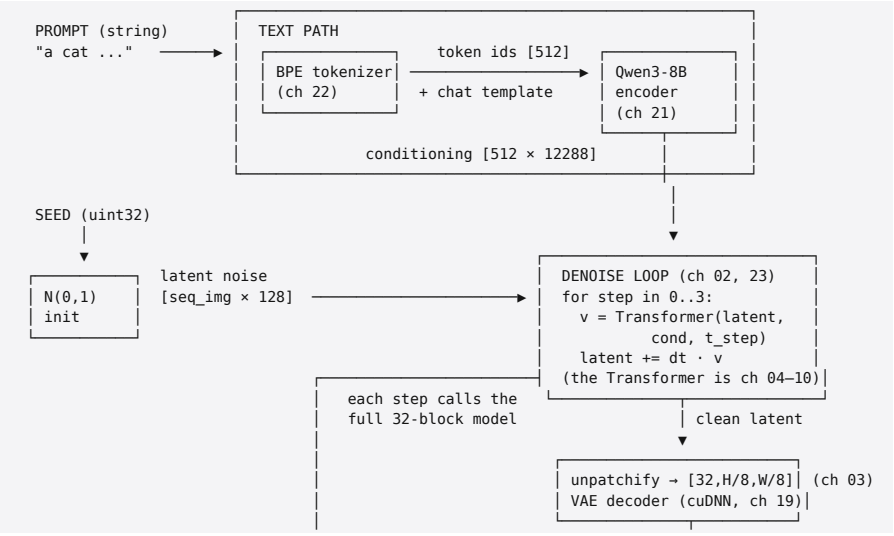
2. **The diffusion transformer (the MMDiT).** This is the heart of FLUX. It takes a noisy image latent plus the text conditioning plus a "how noisy is this right now" timestep, and predicts a **velocity** — the direction to move the latent to make it less noisy and more like the prompt. It is called repeatedly. It is ~9 billion parameters arranged as 8 "double-stream" blocks followed by 24 "single-stream" blocks (Chapters 08–09).

3. **The VAE decoder.** The transformer works in a compressed latent space (roughly 1/8 the spatial resolution, with 16/32 channels instead of 3). Once the latent is clean, the VAE decoder — a stack of convolutions — expands it back into a full-resolution RGB image (Chapters 19–20).

Three models, three jobs: **read the prompt, denoise the latent, decode to pixels.**

0.3 The dataflow, stage by stage

Here is the full path. Keep this diagram nearby; later chapters zoom into each box.



```
                    ▼                                   ▼
          (transformer internals)          RGB [3,H,W] → PNG (ch 23)
```

Two things to notice now:

- **The transformer is called once per denoise step.** With the distilled 4-step schedule that is 4 forward passes of a 9B model. Everything about performance is dominated by making that forward pass cheap. (This is why Chapters 11–18 exist.)
- **The text path runs once**; its `[512 × 12288]` output is reused across all 4 steps. The latent path runs every step.

0.4 A first contact with the numbers

For a 1024×1024 image, FP8 precision, 4 steps, on the GB10 (measured this session):

Stage | Time | Notes
Build transformer (load + repack weights) | ~5.3 s | one-time per process
Build VAE | ~0.5 s | one-time
Load + build Qwen3 encoder | ~10.5 s | one-time; only with --prompt/--tokens
Text encode (tokenize + encoder forward) | ~0.28 s | native; replaced a 98 s Python path (ch 22)
Denoise loop (4 steps) | ~6.5 s | the hot path; ~1.6 s/step
VAE decode | ~1.3 s | cuDNN graph API (ch 19)

The denoise loop is the thing worth optimizing, and within it, the attention and the GEMMs. Chapter 17 tells the story of taking *just the attention* from 56.5 ms to 19.7 ms per call — a 2.87× win that is felt directly in that 6.5 s.

0.5 The repository map

The code is organized so that the curriculum can follow it almost one-to-one.

```
F2KTest/
├── src/
│   ├── common/                 # host-side, no CUDA — formats, loading, tokenizer
│   │   ├── f2k_format.{h,cpp}      # the on-disk weight container (ch 14)
│   │   ├── f2k_model_loader.*      # multi-shard mmap loader (ch 14)
│   │   ├── tensor_router.*         # tensor name → structural role (ch 14)
│   │   ├── safetensors.*           # reads HF .safetensors during conversion
│   │   └── bpe_tokenizer.{h,cpp}   # native byte-level BPE tokenizer (ch 22)
│   │
```

```
│   └── backend/cuda/           # the GPU implementation
│       ├── fp4_gemm.*  fp8_gemm.*  # CUTLASS block-scaled tensor-core GEMM (ch 13)
│       ├── linear.*            # the quantized Linear layer over those GEMMs (ch 13)
│       ├── attention.*         # the flash-attention kernels (ch 16–18)
│       ├── rope*, rmsnorm, ...     # kernels/ : the small reusable kernels (ch 15)
│       ├── modulation_mlp.*        # timestep → modulation vectors (ch 07)
│       ├── double_stream_block.*   # text+image joint-attention block (ch 08)
│       ├── single_stream_block.*   # fused QKV+MLP block (ch 09)
│       ├── flux_transformer.*      # the top-level 32-block model (ch 10)
│       ├── qwen_*.*                # the Qwen3 text encoder (ch 21)
│       ├── conv2d.* vae_*.*        # the VAE decoder (ch 19–20)
│       └── sampler.*               # the flow-matching scheduler & step (ch 02)
│
├── tools/
│   ├── generate.cu             # the end-to-end CLI: prompt → PNG (ch 23)
│   ├── f2k_convert.cu          # safetensors → F2K quantized weights (ch 12, 14)
│   ├── bench_attention.cu      # the attention microbenchmark (ch 17, 24)
│   ├── mma_unit.cu             # standalone PTX/ldmatrix validation (ch 18)
│   └── *.py                    # diffusers/HF reference dumpers for validation (ch 25)
│
├── tests/                      # one ctest per component; the correctness contract
├── third_party/                # cutlass, cudnn(via system), imgui, glfw, stb, json (submodules)
└── docs/                       # this course
```

A useful habit: when a chapter discusses a concept, open the named file beside it. The implementation chapters are written to be read *with the source*, not instead of it.

0.6 The one hardware fact that explains everything

We will repeat this until it is reflex. The GB10 offers on the order of **1 PFLOP/s of FP4 compute** but only **~273 GB/s of memory bandwidth**. The ratio of compute to bandwidth is enormous — thousands of FLOPs are available for every byte that can be moved.

Inference is not like training. In inference, each weight is used a small number of times per forward pass, so the workload is dominated by *reading the weights and activations from memory*, not by the arithmetic. On this machine that means:

1. **Shrink the bytes.** Store weights in 4-bit (NVFP4) or 8-bit (MXFP8) instead of 16-bit, and you move 4×/2× fewer bytes — a near-linear speedup on a bandwidth-bound workload. This is why Chapters 12–13 are about quantization.
2. **Keep data on-chip.** A kernel that re-reads the same data from HBM is wasting the scarcest resource. Flash

attention exists precisely to stop re-reading K and V from memory (Chapter 16).

3. **"Fast enough" = "saturates HBM."** The attention kernel in Chapter 17 is declared finished not when the tensor cores are busy but when it hits ~279 GB/s — essentially the memory ceiling. There is nothing left to win because the bytes simply cannot arrive faster.

4. **Unified memory changes the data-movement model.** CPU and GPU share the same 128 GB, cache-coherently. There is no `cudaMemcpy` of weights across PCIe; the model is mmap'd and the GPU reads it in place (Chapter 14). A 9B model in FP8 (~9 GB) or NVFP4 (~5 GB) simply fits, with room to spare.

If you ever wonder "why did they do it *that* way," the answer is usually "because the machine is bandwidth-bound." Chapter 11 turns this intuition into a roofline you can compute.

0.7 Building and running

The full build & toolchain details are Appendix C; here is the orientation version.

```
# configure + build (CUDA 13, cuDNN 9, CMake; targets sm_121)
cmake -S . -B build
cmake --build build -j

# run the whole pipeline from a prompt — single self-contained call, no Python
./build/generate --prompt "a cat on a skateboard" \
                 --res 1024 --precision fp8 --steps 4 --out cat.png

# correctness suite (each component has a golden test)
cd build && ctest
```

`generate` has three ways to supply text conditioning, which map to three points in the pipeline and are useful for isolating bugs:

- `--prompt "<text>"` — the full native path (tokenizer →

Qwen3 → transformer).

- `--tokens <file>` — skip the tokenizer; feed precomputed token ids.
- `--embeds <file>` — skip the encoder too; feed a precomputed `[512 × 12288]` conditioning tensor (used to compare against the reference diffusers pipeline, Chapter 25).

And `--precision {nvfp4,fp8}` selects the weight format (Chapter 12), `--res` the output resolution, `--steps` the number of denoise steps, `--seed` the RNG seed.

0.8 How the model gets onto disk

One preparatory step happens before any image is generated: the original FLUX.2-klein weights (distributed by Black Forest Labs as `.safetensors`) are **converted and quantized** into this project's own `.f2k1` container by `tools/f2k_convert.cu`. That tool reads each tensor, decides whether it is eligible for quantization, packs it into NVFP4 or MXFP8 with the appropriate block scales, and writes a memory-mappable file. The runtime then mmaps those files and never parses anything on the hot path. Chapters 12 (the formats) and 14 (the container) cover this; you only need to know now that it happened.

0.9 What to read next

- If you want the **theory** first: go to Chapter 01 (diffusion) and read Part I straight through. You will understand *why* the pipeline has the shape it does before seeing any CUDA.
- If you want the **implementation** first: jump to Chapter 11 (the Blackwell target) and Part III. You will see how the machine is used before learning the math it is computing.
- If you came for the **attention optimization** specifically (the headline engineering result): Part IV (Chapters 16–18) is

self-contained enough to read on its own, given this overview.

- If you came for **what you can do with it** beyond text-to-image: Part VIII (Chapters 26–28) adds the VAE encoder, image conditioning (img2img / inpainting / upscaling), and the streaming serving layer.

Either way, keep §0.6 in mind. Everything bends toward bandwidth.

Next: Chapter 01 — Diffusion from first principles.

Chapter 01 — Diffusion from First Principles

Goal of this chapter: derive, from scratch, why gradually adding noise to data and then learning to reverse that process gives you a generative model. We build the forward process, prove its closed form, derive the reverse process and the *score function*, connect the score to a *denoiser* via Tweedie's formula, and derive the training objective. By the end you will understand the machinery that FLUX's *flow matching* (Chapter 02) is a particular, streamlined instance of.

Prerequisites: multivariable calculus, basic probability (Gaussians, conditional densities, expectation), and comfort with linear algebra. No prior ML. Math is written in LaTeX; if your viewer doesn't render it, the prose around each equation states it in words.

Notation. x_0 is a clean data point (an image, or for us a latent). I is the identity matrix. $N(\mu,\Sigma)$ is a Gaussian. p_data is the (unknown) distribution we want to sample from. ∇_x is the gradient with respect to x. We work in R^d where d is the dimensionality of a latent (for us, tens of thousands).

1.1 The generative modeling problem

We have a dataset of samples $x_0 \sim$ p_data — millions of images. We want a procedure that produces *new* samples from the same distribution. We never get to see p_data itself; we only have samples.

The difficulty is dimensionality. A 1024×1024×3 image lives in R^3,145,728. The set of "plausible images" is a vanishingly thin, intricately curved manifold inside that enormous space. Sampling from it directly is hopeless: pick a random point and

with overwhelming probability you get noise, not a picture.

Historically there were several attacks on this problem — GANs (learn a generator network adversarially), VAEs (learn an encoder/decoder with a latent prior), autoregressive models (factorize p(x) = ∏*i p(x_i* | *x*<i)). Diffusion models took over image generation around 2020–2022 because they are **s table to train** and produce **high sample quality and diversity**. Their core idea is almost suspiciously simple, and the rest of this chapter is its derivation.

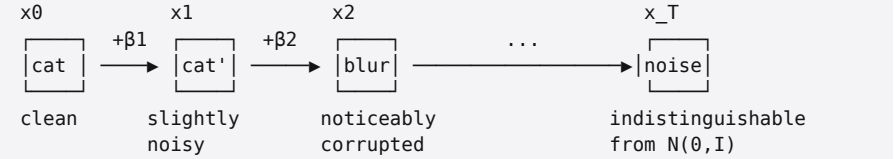
> **The idea in one line.** It is hard to jump from noise to an image in one step, but it is *easy* to take a tiny step from a slightly-noisier image toward a slightly-cleaner one. So: learn the tiny step, and take many of them.

1.2 The forward process: destroying structure on purpose

Define a sequence that progressively corrupts a clean sample x_0 with Gaussian noise over "time" t running from 0 (clean) to T (pure noise). In discrete form (the DDPM formulation), pick a *noise schedule* $\beta_1,\ldots,\beta_T \in (0,1)$ and define each step as

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_t,\ \varepsilon_t \sim N(0,I).$$

Equivalently, $q(x_t \mid x_{t-1}) = N(\sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$. Each step scales the previous value down slightly (the $\sqrt{1-\beta_t}$ factor) and injects a little fresh noise. The down-scaling is what keeps the variance from exploding; we will see in a moment that it keeps the total variance bounded at 1.



This **forward process is fixed** — there is nothing to learn

here. It is a known Markov chain that turns data into noise. The art is in *reversing* it.

1.2.1 The closed form: jumping to any time t in one shot

Applying the recursion t times looks expensive, but Gaussians compose, and the result telescopes into a clean closed form. Define

$$\alpha_t := 1-\beta_t,\ \bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s .$$

Claim. $x_t = \sqrt{(\bar{\alpha}_t)}\, x_0 + \sqrt{(1-\bar{\alpha}_t)}\, \varepsilon$, with $\varepsilon \sim N(0,I)$. Equivalently

$$q(x_t \mid x_0) = N\left(\sqrt{(\bar{\alpha}_t)}\, x_0,\ (1-\bar{\alpha}_t)\, I\right).$$

Proof (by unrolling two steps, then induction). Start from $x_t = \sqrt{(\alpha_t)}\, x_{t-1} + \sqrt{(1-\alpha_t)}\, \varepsilon_t$ and substitute $x_{t-1} = \sqrt{(\alpha_{t-1})}\, x_{t-2} + \sqrt{(1-\alpha_{t-1})}\, \varepsilon_{t-1}$:

$$x_t = \sqrt{(\alpha_t\, \alpha_{t-1})}\, x_{t-2} + \sqrt{(\alpha_t)}\sqrt{(1-\alpha_{t-1})}\, \varepsilon_{t-1} + \sqrt{(1-\alpha_t)}\, \varepsilon_t \text{sum of two independent Gaussians .}$$

The two noise terms are independent zero-mean Gaussians, so their sum is a single zero-mean Gaussian whose variance is the sum of the variances:

$$\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t) = \alpha_t - \alpha_t\alpha_{t-1} + 1 - \alpha_t = 1 - \alpha_t\alpha_{t-1}.$$

So $x_t = \sqrt{(\alpha_t\alpha_{t-1})}\, x_{t-2} + \sqrt{(1-\alpha_t\alpha_{t-1})}\, \bar{\varepsilon}$. This has exactly the same shape as the one-step rule with $\alpha_t \to \alpha_t\alpha_{t-1}$. Repeating down to x_0 replaces the product by $\bar{\alpha}_t = \prod_{s \le t} \alpha_s$, giving the claim. ■

Two consequences worth pausing on:

- **The variance is bounded.** $\mathrm{Var}[x_t \mid x_0] = (1-\bar{\alpha}_t)I$, and the signal coefficient is $\sqrt{(\bar{\alpha}_t)}$. As t grows $\bar{\alpha}_t \to 0$, so $x\_T \approx N(0,I)$ — pure noise, with no memory of x_0. This is the *variance-preserving* (VP) schedule: $(\text{signal})^2 + (\text{noise})^2 = \bar{\alpha}_t + (1-\bar{\alpha}_t) = 1$ at

every t.

• **Training is cheap.** To get a training pair at any noise level we do not simulate the chain; we sample x_0 from the data, sample one ε, pick a t, and compute x_t in closed form. One draw, any time level. This is the property that makes diffusion practical.

1.2.2 The signal-to-noise reparameterization

It is often cleaner to forget the discrete index and describe the corruption by a single scalar. Write $x_t = a_t\, x_0 + \sigma_t\, \varepsilon$ with $a_t = \sqrt{(\bar{\alpha}_t)}$ and $\sigma_t = \sqrt{(1-\bar{\alpha}_t)}$. The **signal-to-noise ratio** SNR(t) $= a_t^2/\sigma_t^2$ decreases monotonically from ∞ (clean) to 0 (noise). Everything that matters about a schedule is *how SNR sweeps from high to low*; the specific β_t values are just one way to pa rameterize that sweep. Flow matching (Chapter 02) chooses a different, simpler sweep — keep this reparameterization in mind, it is the bridge.

1.3 The reverse process: where the learning lives

We want to go backward: start from x_T ~ N(0,I) and undo th e noise step by step until we reach a clean x_0. The reverse of a Markov chain is also Markov, so we want $q(x_{t-1} \mid x_t)$. The problem: this density depends on the data distribution and is intractable.

But there is a beautiful fact. **Conditioned on the original x_0** , the reverse step *is* a tractable Gaussian. Using Bayes' rule,

$$q(x_{t-1} \mid x_t, x_0) = (q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0))/(q(x_t \mid x_0)),$$

and all three factors on the right are the Gaussians we already derived. Multiplying them and completing the square (a mechanical but tedious computation) gives another Gaussian:

$$q(x_{t-1} \mid x_t, x_0) = N\left(\tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I\right),$$

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\, x_0 + \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\, x_t, \quad \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t .$$

The catch is the dependence on x_0, which we do not have at sampling time — that is the whole point, we are *trying* to produce x_0. So the model's job is to **estimate the quantity that lets us recover $\tilde{\mu}_t$**. There are three equivalent things it could predict; they are reparameterizations of each other:

1. the original x_0 directly,
2. the noise ε that was added (the "ε-prediction" of DDPM),
3. the **score** $\nabla_{x_t} \log q(x_t)$ — the gradient of the log-density.

The score is the most theoretically illuminating, so we derive it next; FLUX ends up predicting a fourth, closely related quantity (a *velocity*), and Chapter 02 shows it is the same idea in a tidier coordinate system.

1.4 The score function and Tweedie's formula

The **score** of a distribution is $\nabla_x \log p(x)$ — a vector field pointing in the direction of increasing log-probability, i.e. "toward more plausible data." If you knew the score everywhere, you could sample by starting anywhere and walk ing uphill (with the right amount of noise — §1.6).

For the *conditional* forward density we can compute the score exactly. Since $q(x_t \mid x_0) = N(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\, I)$, its log is $-\tfrac{1}{2(1-\bar{\alpha}_t)}\lVert x_t - \sqrt{\bar{\alpha}_t}x_0\rVert^2$ + const, and so

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}\, x_0}{1-\bar{\alpha}_t} = -\frac{\varepsilon}{\sqrt{1-\bar{\alpha}_t}} = -\frac{\varepsilon}{\sigma_t},$$

using $x_t - \sqrt{\bar{\alpha}_t}x_0 = \sigma_t\, \varepsilon$. **The score is the added noise, negated and scaled.** This is the precise sense in which

"estimating the score" and "estimating the noise" are the same task — predicting ε and predicting ∇log q differ only by the known factor $-1/\sigma_t$.

Tweedie's formula: the denoiser is the score

There is a deeper identity connecting the score of the *margin al* $p_t(x_t) = \int q(x_t \mid x_0) p\_data(x_0)\, dx_0$ to the best possible denoiser. For a Gaussian-corrupted observation, **Tweedie's formula** states the posterior mean of the clean signal is

$$E[x_0 \mid x_t] = (1)/(\sqrt{(\bar{\alpha}_t)})\ (x_t + (1-\bar{\alpha}_t)\ \nabla_{x_t} log\ p_t(x_t)\ ).$$

Derivation sketch. Write $p_t(x_t) = \int N(x_t; a_t\, x_0, \sigma_t^2\, I)\, p\_data(x_0)\, dx_0$. Differentiate under the integral: $\nabla_{x_t}\, p_t = \int (a_t\, x_0 - x_t)/(\sigma_t^2)\, N(\cdot)\, p\_data\, dx_0$. Divide by p_t to turn the integrand's N p_data/p_t into the posterior $p(x_0 \mid x_t)$, and you get $\nabla_{x_t} log\ p_t = (a_t\, E[x_0 \mid x_t] - x_t)/(\sigma_t^2)$. Solve for $E[x_0 \mid x_t]$. ■

Read it slowly: **the optimal denoiser — the conditional mean of the clean image given the noisy one — is a simple algebraic function of the score.** A network trained to denoise is, automatically, a network that has learned the score. This is why the three prediction targets in §1.3 are interchangeable: x_0-prediction, ε-prediction, and score-estimation are linear reparameterizations of one anothe r. The model architecture (the MMDiT of Part II) is the *function approximator* for this denoiser/score; the choice of *what exactly it outputs* is a coordinate convention, and FLUX's choice is Chapter 02.

1.5 The training objective

We want a network $s_\theta(x_t, t)$ that approximates the marginal score $\nabla_{x_t} log\ p_t(x_t)$. We cannot evaluate that target — p_t is the intractable marginal. **Denoising score matching** (Vincent, 2011) rescues us: it proves that matching the *conditional*

score (which we *can* evaluate, §1.4) yields the same minimizer as matching the marginal score. Concretely, the two objectives

$$E_{x_t\sim p_t}\,\lVert s_\theta(x_t,t) - \nabla_{x_t}\log p_t(x_t)\rVert^2 \text{ and } E_{x_0,\,x_t\sim q(\cdot\mid x_0)}\,\lVert s_\theta(x_t,t) - \nabla_{x_t}\log q(x_t\mid x_0)\rVert^2$$

differ only by a constant independent of θ. *Why:* expand the marginal-score objective, and the cross term $E[\langle s_\theta, \nabla\log p_t\rangle]$ can be rewritten — using $\nabla p_t = \int \nabla q(\cdot\mid x_0)p_{data}$ — as $E_{x_0,x_t}[\langle s_\theta, \nabla\log q(x_t\mid x_0)\rangle]$, which is the cross term of the conditional objective. The squared-norm-of-target terms differ by a θ-independent constant. Same gradient, same minimizer.

Substituting the conditional score $\nabla\log q(x_t\mid x_0) = -\varepsilon/\sigma_t$ and reparameterizing the network to predict the noise directly, $\varepsilon_\theta(x_t,t) := -\sigma_t\, s_\theta(x_t,t)$, the objective collapses to the famous, almost trivial-looking DDPM loss:

$$L(\theta) = E_{x_0\sim p_{data},\,\varepsilon\sim N(0,I),\,t\sim U\{1,\dots,T\}}\,\lVert \varepsilon - \varepsilon_\theta(\sqrt{\bar\alpha_t}x_0 + \sqrt{1-\bar\alpha_t}\varepsilon_{x_t},\, t)\rVert^2$$

The entire training procedure is: *take a real image, noise it to a random level, ask the network to guess the noise, penalize the squared error.* That is all. There is no adversary, no sampling during training, no instability. This robustness is why diffusion won.

> **Conditioning.** For a text-to-image model the network also receives the prompt conditioning c (the Qwen3 embedding, Chapter 05): $\varepsilon_\theta(x_t,t,c)$. The derivation is unchanged; c is just an extra input. The model learns $\nabla_{x_t}\log p_t(x_t\mid c)$ — the score of images *given the prompt*.

1.6 Sampling: turning a score into images

With a trained score model, how do we generate? Two views,

both important.

Stochastic (ancestral) sampling. Plug ε_θ into the reverse Gaussian of §1.3: start at x_T~ N(0, I), and for t=T,...,1 draw x_{t-1}~ N($\tilde{\mu}_t$,β_t I) with $\tilde{\mu}_t$ reconstructed from the predicted noise. This is the original DDPM sampler; it takes many (hundreds of) steps.

The continuous view and the probability-flow ODE. Take the noising process to the continuous-time limit and it becomes a stochastic differential equation dx = f(x,t) dt + g(t) dW. Anderson's theorem gives the time-reversed SDE, wh ich involves the score:

$$dx = [f(x,t) - g(t)^2 \nabla_x log\ p_t(x)\ ]\ dt + g(t)\ d\bar{W}\ .$$

Remarkably, there is a **deterministic** ODE with the *same marginal densities* p_t at every time — the *probability-flow ODE*:

$$(dx)/(dt) = f(x,t) - tfrac12g(t)^2 \nabla_x log\ p_t(x).$$

Sampling is then just **numerically integrating an ODE** from t=T to t=0, using the learned score as the velocity field. Fewer, larger steps become possible with good integrators. This deterministic view is the doorway to flow matching: if all we need is a good velocity field whose flow carries N(0, I) onto p_data, maybe we can *learn that velocity field directly* and choose the straightest possible paths. That is exactly what FLUX does, and it is Chapter 02.

1.7 Where this lives in the code

This chapter is pure theory, but two of its objects appear directly in the runtime and are worth locating now so Part II/VII feel grounded:

- **The starting noise** x_T ~ N(0, I). In `tools/generate.cu`

the initial latent is drawn from a standard normal (not a uniform — rectified-flow models are trained against Gaussian noise, a subtlety that was once a bug; see Appendix B). The seed makes it reproducible.

• **The sampler / the reverse update.** `src/backend/cuda/sampler.{h,cu}` implements the scheduler and the per-step update `latent += dt · v`. In the score language of this chapter that update is one integration step of the reverse process; in the flow-matching language of the next chapter it is one Euler step of an ODE whose velocity field is the transformer's output. Same arithmetic, two derivations.

• **The network ε_θ / v_θ.** The entire FLUX transformer (`flux_transformer.cu`, Chapters 04–10) *is* the function approximator for the score/velocity. Everything in Parts II–IV is about evaluating that one function quickly.

1.8 Summary and what to carry forward

• A **fixed** forward process turns data into noise; its closed form $x_t = \sqrt{(\bar{\alpha}_t)}x_0 + \sqrt{(1-\bar{\alpha}_t)}\varepsilon$ makes training a one-draw operation at any noise level.

• The **reverse** process is where learning happens. The optimal reverse step is governed by the **score** $\nabla \log p_t$, which (Tweedie) is equivalent to the optimal **denoiser**, which is equivalent to **predicting the noise**.

• The **training loss** is a trivial denoising regression — predict the noise you added — and that simplicity is diffusion's superpower.

• **Sampling** is integrating the reverse SDE, or equivalently its deterministic **probability-flow ODE**. The deterministic view motivates flow matching.

The mental model to keep: *the network is a learned vector field that points noisy latents toward the data manifold (conditioned on the prompt); generation is following that field*

from noise to image. Chapter 02 replaces the noise-schedule machinery with the cleanest possible version of this field — straight-line paths — and shows that this is precisely what FLUX.2-klein computes, four steps at a time.

Exercises

1. **Variance preservation.** Verify $(\sqrt{(\bar{\alpha}_t)})^2 + (\sqrt{(1-\bar{\alpha}_t)})^2 = 1$ and explain in words why down-scaling the signal at each step is necessary for x_T to be standard normal regardless of x_0.
2. **Score ↔ noise.** Starting from $q(x_t \mid x_0)$, re-derive $\nabla_{x_t} \log q(x_t \mid x_0) = -\varepsilon/\sigma_t$ and state the constant relating s_θ and ε_θ.
3. **Tweedie in 1-D.** Let p_data be a single point mass at μ. Compute p_t, its score, and check Tweedie's formula returns $E[x_0 \mid x_t] = \mu$.
4. **ODE vs SDE.** Explain why the probability-flow ODE and the reverse SDE can share marginals yet produce different individual sample trajectories. Which one is reproducible given a fixed start, and which is not?

Next: Chapter 02 — Flow matching & rectified flow, where the noise schedule disappears and the velocity field becomes a straight line.

Chapter 02 — Flow Matching & Rectified Flow

> *Goal of this chapter:* derive flow matching — the framework FLUX.2-klein actually trains and samples with — and show that it is the deterministic, straight-line specialization of the score-based picture from Chapter 01. We build continuous normalizing flows, the flow-matching objective, the rectified-flow (straight-path) choice, and the exact schedule and Euler update implemented in `src/backend/cuda/sampler.{h,cu}`. We end with *distillation* — why this particular model needs only **four** steps.
>
> *Prerequisites:* Chapter 01 (score, probability-flow ODE), ODEs, change of variables for densities.

2.1 From "learn the score" to "learn the velocity"

Chapter 01 ended at the probability-flow ODE: there exists a deterministic velocity field $u_t(x)$ whose flow transports the noise distribution $N(0, I)$ at $t=1$ onto the data distribution at $t=0$, matching the diffusion marginals at every time. Sampling was "integrate that ODE."

That reframing invites a question: *if all we need is a velocity field that carries noise to data, why route through noise schedules, $\bar{\alpha}_t$, and Tweedie at all? Why not learn the velocity field directly — and pick the nicest field we can?*

That is **flow matching** (Lipman et al., 2022), and the "nicest field" is **rectified flow** (Liu et al., 2022): make the transport paths *straight lines*. Straight paths are the easiest to integrate, which is exactly why a distilled FLUX can sample in a handful of Euler steps.

2.2 Continuous normalizing flows in one page

A time-dependent velocity field $u_t : R^d \to R^d$ defines a flow φ_t by the ODE

$$(d)/(dt)\varphi_t(x) = u_t(\varphi_t(x)),\ \varphi_0(x)=x\ .$$

If you start a density p_0 and push every point along this flow, the density at time t, written $p_t = [\varphi_t]_\# p_0$, evolves according to the **continuity equation** (conservation of probability mass):

$$(\partial\ p_t)/(\partial\ t) + \nabla\cdot\ (p_t\ u_t\ ) = 0\ .$$

The pair (p_t, u_t) satisfying this is called a **probability path** generated by u_t. Our goal: find a u_t whose path connects a simple p_1= N(0, I) (noise) to p_0 = p_data. Then sampling is integrating the ODE from t=1 to t=0. Training is making a network v_θ(x,t) approximate $u_t(x)$.

> **Convention.** We follow the code: t=1 is **pure noise**, t=0 is **data**, and t coincides with the noise level σ in `sampler.h` (`ts_[0]=1`, `ts_[num_steps]=0`). Integration runs t:1→0, so the step size dt=σ_{next}-σ_cur is **negative** — matching `FlowMatchScheduler::dt()`.

2.3 The flow-matching objective and its conditional trick

The naïve objective is to regress onto the true field:

$$L\_FM(\theta) = E_t,\ x\sim\ p_t\ lVert\ v\_\theta(x,t)\ -\ u_t(x)\ rVert^2\ .$$

This is useless as written — we know neither u_t nor how to sample p_t. **Conditional flow matching** fixes it with the same move as denoising score matching in §1.5: build the path as a *mixture of simple per-sample paths* and regress onto the per-sample velocity.

Choose, for each data point x_0 and noise sample x_1, a **conditional path** $x_t = \psi_t(x_0, x_1)$ interpolating from x_0 (at t=0) to x_1 (at t=1), with a known **conditional velocity** $\dot{\psi}_t = (d)/(dt)\psi_t$. Then the **conditional flow-matching loss**

$$L\_CFM(\theta) = E\_t\sim U[0,1],\ x_0\sim p\_data,\ x_1\sim N(0, I)\ [\ lVert\ v\_\theta(x_t,t) - \dot{\psi}_t(x_0,x_1)\ rVert^2\ ]$$

has the **same gradient in θ** as L_FM against the marginal field u_t that the mixture of conditional paths induces. *Why:* the marginal velocity is the conditional-velocity posterior mean, $u_t(x) = E[\dot{\psi}_t \mid x_t = x]$, and regressing a function onto a target has the same minimizer as regressing onto that target's conditional mean (the classic "the mean minimizes MSE" identity). So matching the easy conditional velocity is equivalent to matching the impossible marginal one. This is the exact analogue of the DSM argument, and it is what makes the framework trainable.

Everything now hinges on one design choice: **which conditional path ψ_t?**

2.4 Rectified flow: straight lines are the whole trick

Pick the simplest possible interpolation — the straight line between data and noise:

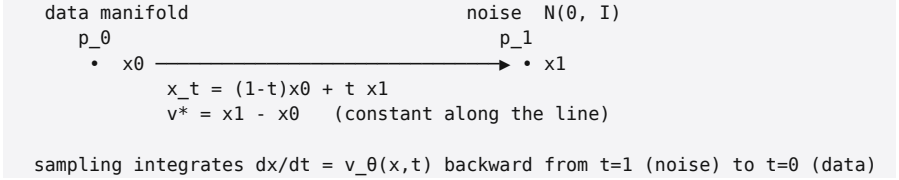
$$x_t = (1-t)\ x_0 + t\ x_1,\ \dot{\psi}_t = (d\ x_t)/(dt) = x_1 - x_0\ .$$

The conditional velocity is **constant in t** — it does not depend on where you are along the path, only on the endpoints. The training target could not be simpler:

$$L(\theta) = E_t,x_0,x_1\ lVert\ v\_\theta\ ((1-t)x_0+t\ x_1,\ t\ ) - (x_1 - x_0)\ rVert^2\ .$$

The network learns to look at a point on a noise↔data segment and the time, and predict the segment's direction.

This is **rectified flow**, and it is what FLUX trains.



Why straightness matters for us specifically

Recall §0.6: on the GB10 each transformer forward pass is expensive (bandwidth- bound over 9B weights). The number of forward passes is the number of ODE steps. For a curved velocity field you need many small Euler steps or a fancy integrator to control truncation error. **For a perfectly straight path a single Euler step is exact** — $x_0 = x_1 - 1\cdot(x_1-x_0)$. Real learned fields are not perfectly straight (the marginal field bends where many lines cross), but the straighter the paths, the fewer steps you need. Rectified flow is, quite literally, an effort to make the sampler cheap. Distillation (§2.8) pushes it to the extreme.

Relationship to Chapter 01's score

Rectified flow is not a different theory; it is the score picture in tidier coordinates. The straight-line interpolant is a Gaussian probability path $p_t(x \mid x_0)= N((1-t)x_0, t^2 I)$ — compare the VP form $a_t\, x_0+\sigma_t\varepsilon$ with $a_t=1-t$, $\sigma_t=t$. There is an exact linear map between the learned velocity v_θ and the score $\nabla \log p_t$; predicting one determines the other. Flow matching simply chooses the schedule $(a_t,\sigma_t)=(1-t,t)$ and the velocity parameterization because together they give straight conditional paths and a constant target. Chapter 01's generality (any schedule, any of three targets) collapses here into the single cleanest choice.

2.5 The timestep embedding the model consumes

The network must know *which* t it is being asked about. FLUX feeds t through a sinusoidal embedding, exactly as in `sampler.h::compute_timestep_embedding`:

$$emb(t)_i = cos(\tau\, e^{(-(log\ P)/(H)\ i)}),\ emb(t)_{H+i} = sin(\tau\, e^{(-(log\ P)/(H)\ i)}),\ i=0,\ldots,H-1,$$

with H=dim/2, max period P=10000, and — the detail that matters — the raw flow time scaled by 1000, $\tau = t \cdot 1000$ (the FLUX reference convention). This is the same Transformer positional-encoding trick (a bank of frequencies) applied to a scalar time. The resulting vector enters the **modulation MLP** (Chapter 07), which turns it into the per-block scale/shift/gate signals that tell every layer how aggressively to act at this noise level.

```
// sampler.h — cat(cos, sin) over a geometric frequency bank, t already ×1000
const float exponent = -log_max * float(i) / float(half);   // log_max = log(10000)
const float arg      = t * std::exp(exponent);
out[i]        = cos(arg);     // first half
out[half + i] = sin(arg);     // second half
```

2.6 The schedule: linear sigmas with resolution-aware shifting

A naïve schedule places the sampling times t_i uniformly: `FlowMatchScheduler`'s constructor builds t_i = 1 - i/N for i=0,…,N, i.e. the linear sweep 1, 1-tfrac1N, …, 0.

But there is a well-known wrinkle from the SD3/FLUX line of work: **the right distribution of timesteps depends on image resolution.** A larger image has more tokens (more spatial detail to resolve), and empirically benefits from spending *more* of its steps at high noise. FLUX implements this with *dynamic shifting* — remapping the linear sigmas through a monotone function whose strength scales with the

token count. This is `FlowMatchScheduler::flux2_dynamic`:

$$\mu = \beta_{base} + (\beta_{max}-\beta_{base})\cdot (S_{img}-S_{base})/(S_{max}-S_{base}),\ \sigma' = (e^{(\mu)})/(e^{(\mu)} + (tfrac1\sigma-1\ ))\ (\sigma>0),\ \sigma'=0\ (\sigma=0).$$

with defaults β_base=0.5, β_max=1.15, S_base=256, S_max =4096 (token counts). The exponential time-shift bends the schedule so larger S_img (e.g. 4096 tokens at 1024px) shifts mass toward high noise.

You can see the effect in a real run's log (1024px, 4 steps):

```
step 0  t=1.000 dt=-0.095     ← small first move (lots of structure still to commit)
step 1  t=0.905 dt=-0.145
step 2  t=0.760 dt=-0.247
step 3  t=0.513 dt=-0.513     ← the final, largest jump straight to t=0 (data)
```

Those non-uniform dts are the dynamic shift at work. Notice t never lands on a uniform grid; the steps get larger as noise decreases, because the late, low-noise part of a straight-ish path needs less resolution.

2.7 The sampling loop: one Euler step per call

With a velocity network and a schedule, sampling is **forward Euler on the ODE**. One step from t_i to t_i+1:

$$x\_t\_i+1 = x_t\_i + (t\_i+1-t\_i)\ v\_\theta(x_t\_i, t\_i, c) = x_t\_i + dt\_i \cdot v\_\theta(...),$$

with dt_i = t_i+1-t_i < 0. This is **literally** the runtime update: the transformer computes v_θ (the "velocity"/"v_pred"), and `axpy_bf16` performs `latent += dt · v` in place:

```
// sampler.h
bool axpy_bf16(void* y, const void* x, float alpha, size_t n, ...);  // y[i] += alpha * x[i]
// generate.cu denoise loop (schematic):
for (int i = 0; i < N; ++i) {
    v = transformer.forward(latent, cond, timestep_emb(sched.t(i)));
    axpy_bf16(latent, v, sched.dt(i), n);     // latent += dt_i * v
}
```

Every box in Chapter 00's denoise loop reduces to this: evaluate the 9B velocity field once, take one (variable-size) Euler step, repeat N times. The expensive part is the `transformer.forward` — which is why Parts II–IV exist — and the cheap part is this `axpy`.

> **Why N(0, I) for the initial latent.** The ODE starts at the noise endpoint $p_1 = N(0, I)$, so the initial latent must be drawn standard-normal. Using a uniform initialization (a real early bug, Appendix B) starts the integrator off the distribution the field was trained on, and the trajectory lands off-manifold. `generate.cu` draws $x_1 \sim N(0, > I)$ with the seeded RNG.

2.8 Distillation: why four steps

A base rectified-flow FLUX still needs a few dozen steps, because the *marginal* velocity field bends wherever many straight conditional paths cross (the field has to average them). **klein** is a *distilled* variant: a student network is trained so that a *few* of its Euler steps reproduce what many steps of a teacher produce — in effect re-straightening the trajectories so that 4 large steps suffice. Two fingerprints of distillation show up directly in this project:

- `guidance_embeds = false`**.** Classifier-free guidance (running the model with and without the prompt and extrapolating) doubles the cost per step. Distilled klein bakes guidance into the weights, so there is no CFG at inference and no guidance-scale input to the modulation MLP. (See the architecture config, Chapter 10.)
- **4 steps is the default.** `generate --steps 4`. You *can* raise it (the log above is a 4-step run), and §2.6's schedule still applies, but the model is tuned to look right at 4. More steps mostly sharpen fine texture; they will not fix, e.g., garbled text, which is a capability limit of the distilled model,

not an integration-error problem.

The cost of distillation is the well-known one: a 4-step distilled model trades a little fidelity and controllability for a ~10× cheaper sampler. For an interactive, bandwidth-bound, local image generator, that is exactly the right trade.

2.9 Where this lives in the code

Concept (this chapter) | Code
linear sigma sweep t_i = 1-i/N | FlowMatchScheduler ctor, sampler.h:46
resolution-aware dynamic shift | FlowMatchScheduler::flux2_dynamic, sampler.h:60
timestep sinusoidal embedding (t×1000, cat(cos,sin)) | compute_timestep_embedding, sampler.h:30
velocity field v_θ | FluxTransformer::forward (Chapters 04–10)
Euler update latent += dt·v | axpy_bf16, sampler.h:89; loop in generate.cu
initial $x_1 \sim N(0, I)$ | generate.cu latent init

The correctness of the whole loop is checked end to end by `tests/test_denoise_loop.cu` (Chapter 25), which runs the 4-step schedule on real weights and verifies the latent stays finite and well-scaled across steps.

2.10 Summary and what to carry forward

- Flow matching learns a **velocity field** $v_\theta(x,t)$ directly; sampling is integrating $dx/dt = v_\theta$ from noise ($t=1$) to data ($t=0$).
- The **conditional** flow-matching loss makes this trainable: regress onto the per-sample velocity of a chosen interpolation; the minimizer matches the intractable marginal field.
- **Rectified flow** chooses straight-line interpolations, so the target is the constant direction $x_1 - x_0$ and the paths are as cheap to integrate as possible. This is FLUX's training.
- FLUX adds a **resolution-aware dynamic-shift schedule** and, in klein, **few-step distillation** (no CFG, 4 steps) — both visible directly in `sampler.h` and the run logs.

• The runtime denoise loop is exactly: evaluate the transformer for v_θ, `latent += dt·v`, repeat. Everything else in the course is about making that one evaluation fast and correct.

With Parts-I theory in hand — *the model is a learned straight-line velocity field from noise to data, conditioned on the prompt* — we can now open the model itself. Chapter 03 covers the last piece of "where" the diffusion happens (latent space and the VAE), and then Part II dissects the velocity network box by box.

Exercises

1. **Constant target.** For the straight path $x_t=(1-t)x_0+tx_1$, confirm $\dot{\psi}_t=x_1-x_0$ and explain why a *constant-in-t* target is easier to learn and to integrate than ε-prediction under a curved VP schedule.
2. **One exact step.** Show that if v_θ equalled the conditional velocity exactly, a single Euler step from t=1 recovers x_0 with zero error. Why does this fail for the *marginal* field even when v_θ is perfect?
3. **Schedule shift.** Using `flux2_dynamic` with S_img=4096, compute μ and the remapped σ' for the linear $\sigma \in 1, 0.75, 0.5, 0.25$. Verify the steps get larger as $\sigma \to 0$, matching the run log in §2.6.
4. **Score from velocity.** Using $p_t(x \mid x_0)= N((1-t)x_0, t^2 I)$, write the score $\nabla_x \log p_t(x \mid x_0)$ and exhibit the linear map between it and the velocity x_1-x_0. Confirm this is consistent with §1.4.

Next: Chapter 03 — Latent diffusion & the VAE.

Chapter 03 — Latent Diffusion & the VAE

Goal of this chapter: explain *where* the diffusion of Chapters 01–02 actually happens. FLUX does not denoise pixels; it denoises a compressed **latent** produced by a variational autoencoder (VAE). We derive why latent-space diffusion is the right move, what a VAE is and what objective trains it, and the exact tensor plumbing of the FLUX VAE — channels, the 8× upsample, the patch-packing convention, and two normalization subtleties (`post_quant_conv` and a batch-norm de-normalization) that were real bugs before they were understood.

Prerequisites: Chapters 01–02; convolutions; KL divergence and the Gaussian.

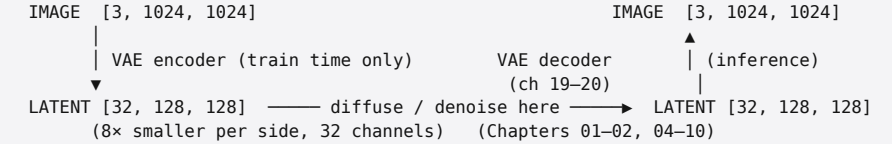
3.1 Why not diffuse pixels directly?

Chapter 00 noted a 1024^2 RGB image lives in R^3,145,728. Running a 9B-parameter transformer's attention over three million spatial positions is quadratically hopeless, and most of those pixels are perceptually redundant — a patch of blue sky carries little information per pixel. Two problems, one fix:

1. **Cost.** Attention is $O(S^2)$ in the number of tokens S (Chapter 16). Halving the spatial resolution cuts S by 4× and attention cost by 16×.
2. **Redundancy.** Natural images are highly compressible; the "interesting" degrees of freedom are far fewer than the pixel count.

Latent diffusion (Rombach et al., 2022 — the "Stable Diffusion" insight) puts both to work: first train an autoencoder that compresses an image into a small latent

and back, then run the *entire* diffusion process (Chapters 01–02) in that latent space. The expensive transformer never sees a pixel; it sees a latent that is ~8× smaller per side and a handful of channels deep. The VAE is trained once, separately, and frozen; diffusion is trained on its latents.



For us only the **decoder** runs at inference (we generate latents from noise and decode them); the encoder matters only conceptually and during the project's validation against diffusers (Chapter 25).

3.2 What a VAE is, briefly but honestly

An autoencoder is an encoder E and decoder D trained so D(E(x))≈ x. A plain autoencoder's latent space is an arbitrary, hole-riddled embedding — fine for reconstruction, bad as a space to *generate* in. A **variational** autoencoder regularizes the latent to be smooth and roughly Gaussian, so that nearby latents decode to plausible images and the space has no dead zones.

Formally, the VAE maximizes the **evidence lower bound** (ELBO) on log p(x):

$$\log p(x) \geq E_{z\sim q_\varphi(z \mid x)}[\log p_\psi(x \mid z)]_{\text{reconstruction}} - D_{KL}(q_\varphi(z \mid x) \,\|\, p(z))_{\text{regularization toward } N(0, I)}.$$

The encoder $q_\varphi(z \mid x) = N(\mu_\varphi(x), \sigma_\varphi^2(x))$ outputs a Gaussian per latent; the reconstruction term pushes the decoder to invert it; the KL term pulls the posterior toward a standard normal prior so the space stays well-behaved. Image VAEs add perceptual and adversarial losses on top for sharp recons

tructions, but the ELBO is the backbone. **For this course the takeaways are:** (i) the latent is *approximately* standard-normal-distributed, which is exactly the distribution rectified flow transports from (Chapter 02 — the noise endpoi nt N(0, I) lives in the *same space* as the latents), and (ii) the decoder D=p_ψ(x | z) is a deterministic-enough map from latent to pixels that we can treat it as a fixed function.

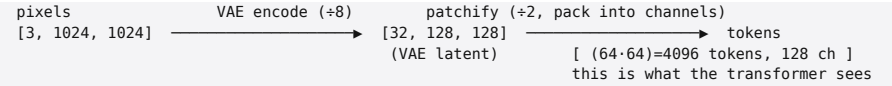
> A subtlety FLUX's latents care about: the encoder's output is **s caled and shifted** to standardize it (so its statistics match the diffusion prior). At decode time that transform must be inverted. §3.5 is exactly this, and getting it wrong was the "patch-grid" bug.

3.3 The FLUX VAE in numbers

From this project's decoder (`src/backend/cuda/vae_decoder.h`) and the patchify kernel, the concrete shapes are:

Quantity	Value
Latent channels C	32
Spatial compression	8× per side (latent $H_{lat}=H/8$)
Decoder input	[N, 32, H/8, W/8]
Decoder output	[N, 3, H, W]
Transformer patch size p	2
Transformer token channel dim	p^2 C = 4×32 = 128

There are therefore **two** spatial reductions stacked on top of each other, and keeping them straight is essential:



So at 1024px the transformer operates on 64×64 = 4096 image tokens, each a 128-dimensional vector — that is the `se q_img = 4096` you saw in Chapter 00 and the S=4608 (with 512 text tokens) of the attention benchmarks. The VAE's 8× and the patchify's 2× multiply to the 16× total between a pixel side and a token-grid side.

3.4 Patchify: turning a latent grid into a token sequence

A transformer consumes a *sequence* of token vectors, not a 2D grid. **Patchify** reshapes the [32, H/8, W/8] latent into [(H/16)(W/16), 128] tokens by carving the latent into 2×2 spatial patches and flattening each patch's 2×2×32 = 128 values into one token vector. The *order* of that flattening is a convention that must match the training pipeline exactly, or every token is internally scrambled. From `kernels/patchify.h`:

```
tokens[b, h_idx·W_p + w_idx,  c·p·p + ph·p + pw]  =  x[b, c, h_idx·p + ph, w_idx·p + pw]
       └──────── token index ────────┘  └─ channel within token ─┘
  c (latent channel) is the SLOWEST-varying axis in the packed dim; pw the FASTEST.
  W_p = W/p,  H_p = H/p,  packed channel dim = p·p·C = 128.
```

In words: the token at grid position (h_idx, w_idx) holds, for e ach latent channel c, that channel's 2×2 block laid out as (ph, pw)∈0,1², with channel c the outermost loop. This precise order — `c·p·p + ph·p + pw`, channel-slowest — matches diffusers' `_patchify_latents + _pack_latents`. An earlier version of this project used the channel-*fastest* order; the model still ran and produced an image, but a subtly wrong one, because each token's 128 numbers were permuted relative to what the weights expected. (Appendix B: "patchify channel order.") `unpatchify` is the exact inverse and runs after the final denoise step, before the VAE.

> **Why p=2 and not patch-in-the-transformer-only?** FLUX folds a 2×2 pixel-shuffle into the token construction so the transformer's sequence length is 1/4 of the latent's spatial size — another bandwidth/compute saving stacked on the VAE's. The decoder undoes the VAE's 8×; the patchify's 2× is undone by `unpatchify` in latent space.

3.5 Two normalization subtleties (and the bugs they caused)

Between "the transformer emitted a clean latent" and "the decoder produces pixels" sit two transforms that are easy to miss because they live *outside* the obvious decoder body. Both were real bugs in this project; both are now handled.

3.5.1 The batch-norm de-normalization

FLUX's pipeline standardizes the patch latent with a per-channel affine transform (stored as a `BatchNorm` over the 128-channel *packed* latent: `running_mean`, `running_var`) at encode time, and **inverts it before decoding**:

$$z\_denorm = z \cdot \sqrt{(Var+\varepsilon)} + mean,\ \varepsilon=10^{-4},\ Var\approx 3.1\ (\Rightarrow std\approx 1.76).$$

This is applied to the 128-channel patch latent *before* `unpatchify`. Each of a patch's four sub-pixels lives in a different one of the 128 channels and therefore gets a different mean/var — so **skipping this de-norm mis-scales the four sub-pixels of every 2×2 patch differently, producing a hard 2×2 checkerboard across the whole image.** That was the infamous "patch-grid" artifact; the fix loads `bn.running_{mean, var}` from the VAE file and applies the affine per channel before unpatchify (Appendix B, "VAE bn de-norm"). It is *structural*: more denoise steps never removed it, which is how we knew it wasn't an undercooking problem.

3.5.2 post_quant_conv

diffusers' `autoencoder_kl_flux2._decode` runs `z = post_quant_conv(z); decoder(z)` — a 1×1 convolution (32→32, a per-pixel channel remix plus bias) applied to the latent *before* the decoder body. It is a sibling of the `decoder.` weight prefix, not under it, so a decoder that starts at `decoder.conv_in` silently skips it. Omitting it turned sharp images into "mush." The decoder now folds it in: the `VAEDecoder::Config::post_quan`

`t_conv_name` names the tensor, the decoder builds a 1×1 conv and applies it at the top of `forward`, and it is skipped gracefully (identity) if the tensor is absent. The decisive diagnostic was feeding diffusers' *own* ground-truth latent through our decoder and getting mush — proving the gap was in the decoder, not the transformer trajectory (Chapter 25).

> **Lesson worth generalizing.** Reference pipelines hide affine/normalization steps in unobvious places (a sibling conv, a buried batch-norm). When porting, diff against the reference at the *latent* boundary, not just the final image — both these bugs were localized by comparing latents, not pixels.

3.6 The decoder, in outline

The full decoder is Chapters 19–20 (it is where the cuDNN-9 convolution story lives). For the latent picture, here is its shape, straight from `vae_decoder.h`, an 8× upsampler built from residual conv blocks:

```
latent [32, H/8, W/8]
  │  (post_quant_conv 1×1, §3.5.2)
  ▼
conv_in 32→512 (3×3)
mid_block:   resnet512 → self-attention(512) → resnet512
up_blocks[0]: 3× resnet512 → upsample ×2 → conv
up_blocks[1]: 3× resnet512 → upsample ×2 → conv          ┐ three ×2 upsamples
up_blocks[2]: resnet512→256, 2× resnet256 → upsample ×2  ┘ = 8× total
up_blocks[3]: resnet256→128, 2× resnet128   (no upsample)
conv_norm_out (GroupNorm) → SiLU → conv_out 128→3 (3×3)
  ▼
pixels [3, H, W]   (then de-norm §3.5.1 happens earlier, on the patch latent)
```

Three ×2 upsamples give the 8×. The mid-block even contains a *self-attention over spatial positions* (1 head, 512-dim) — the same attention machinery as the transformer, which is why the attention kernel of Part IV is written to also serve D=512 (Chapter 16). Everything else is convolution, group normalization, and SiLU.

3.7 Where this lives in the code

Concept | Code
patch packing c·p·p+ph·p+pw | kernels/patchify.{h,cu} (patchify_bf16/unpatchify_bf16)
latent→pixels decoder | vae_decoder.{h,cu} (Chapters 19–20)
post_quant_conv fold-in | VAEDecoder::Config::post_quant_conv_name
bn de-norm | applied in generate.cu to the 128-ch patch latent before unpatchify
spatial self-attention in mid-block | vae_attn.{h,cu} → attention.cu at D=512
correctness vs diffusers latent | tests/test_vae_decoder.cu, tools/diffusers_latent_dump.py (Chapter 25)

3.8 Summary and what to carry forward

• Diffusion runs in a **VAE latent space**, ~8× smaller per side with 32 channels, for both cost and redundancy reasons. The decoder is the only VAE part used at inference.

• A VAE's latent is regularized toward N(0, I) (the ELBO's KL term), which is *why* the rectified-flow noise endpoint and the latents share a space — the diffusion in Chapter 02 transports noise to latents, not to pixels.

• FLUX stacks a **2×2 patchify** on top of the VAE's 8×, giving a 16× pixel-to-token ratio; at 1024px that is 4096 image tokens of 128 dims each. The packing order (`c·p·p+ph·p+pw`, channel-slowest) must match exactly.

• Two easily-missed transforms — the **batch-norm de-normalization** and `post_quant_conv` — sit at the latent/decoder boundary and caused the patch-grid and mush bugs respectively. Diff at the latent boundary when porting.

That completes Part I: we know the *objective* (rectified flow), the *loop* (Euler on a velocity field), and the *space* (VAE latent). Part II opens the velocity network itself — the MMDiT transformer — beginning with how images become tokens an d the two-stream architecture that lets text and image attend to each other.

Exercises

1. **Token count.** Derive seq_img at 512px and 2048px from the 8× VAE and p=2 patchify. Confirm 1024px gives 4096, and relate to the S=4608 attention benchmark (what are the other 512 tokens?).
2. **Patch packing.** Write out the 128-vector for one token given a latent block, under both `c·p·p+ph·p+pw` (correct) and the channel-fastest order, and argue why the wrong order produces a coherent-but-degraded image rather than noise.
3. **Why the grid is structural.** Explain why omitting the per-channel bn de-norm yields a fixed 2×2 checkerboard independent of step count, whereas an under-trained sampler yields blur. (Hint: which channels map to which sub-pixel?)
4. **ELBO intuition.** If you removed the KL term entirely, what would happen to the latent space, and why would diffusing in it (and starting from N(0, I)) break?

Next: Chapter 04 — Diffusion transformers (MMDiT), where the latent tokens meet the text and the velocity network begins.

Chapter 04 — Diffusion Transformers (MMDiT)

Goal of this chapter: introduce the velocity network itself. We explain why a *transformer* replaced the U-Net for diffusion, what "tokens" are for an image model, and the **MMDiT** (Multi-Modal Diffusion Transformer) two-stream design that lets text and image attend to each other. This chapter is the map of Part II; Chapters 05–10 zoom into each component. We anchor everything to `src/backend/cuda/flux_transformer.{h,cu}` and the config drawn from `transformer/config.json`.

Prerequisites: Part I; the transformer (attention + MLP) at a conceptual level — Chapter 16 derives attention rigorously, but you need only "tokens attend to tokens" here.

4.1 From U-Net to transformer

The original latent-diffusion models used a **U-Net**: a convolutional encoder/ decoder with skip connections, denoising the latent at multiple spatial scales. U-Nets bake in locality (convolutions) and multi-scale structure (down/up sa mpling). They work, but they have two frictions for text-to-image: injecting text conditioning means bolting cross-attention onto a fundamentally convolutional backbone, and convolution's locality limits long-range interactions.

The **Diffusion Transformer (DiT)** (Peebles & Xie, 2022) threw out the U-Net and denoised the latent with a plain transformer over image patches. It scales cleanly, treats all positions uniformly, and — crucially for text-to-image — lets t ext and image live in the *same* attention, as just more tokens. FLUX is a descendant of this line, specifically of the **MMDiT** design introduced with Stable Diffusion 3.

> The velocity field v_θ(x,t,c) of Chapter 02 *is* this transformer. Its input is the noisy latent (as tokens) plus the timestep t plus the text conditioning c; its output is the velocity, same shape as the latent. Each of the 4 denoise steps is one forward pass of this network.

4.2 Tokens: the common currency

A transformer operates on a **sequence of token vectors** and learns interactions between them via attention. FLUX has two kinds of tokens, and the whole architecture is about how they mix.

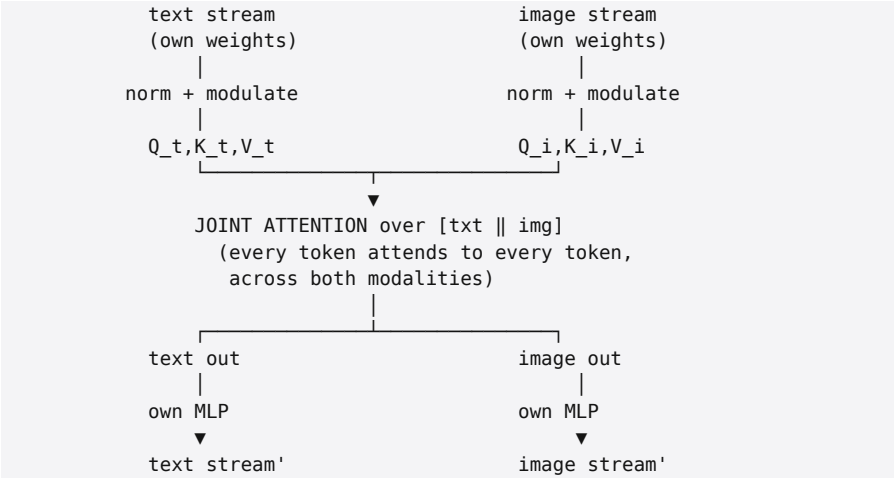
- **Image tokens.** From Chapter 03, the patchified latent is a sequence of S_img tokens (e.g. 4096 at 1024px), each a 128-dim vector. An *embedder* projects them to the model's hidden width.
- **Text tokens.** The Qwen3 encoder (Chapter 05) produces S_{txt}=512 conditioning vectors, each 12288-dim. A *context embedder* projects them to the same hidden width.

Once both are projected to the common hidden dimension d=4096, they are interchangeable as far as attention is concerned — image token 17 can attend to text token 4 and vice versa. This is what "multi-modal" means here: one attention operation over a *joint* set of text and image tokens.

4.3 The two-stream idea

A naïve DiT would simply concatenate text and image tokens and run identical transformer blocks over the union. MMDiT refines this: for the early, most important layers it keeps **two separate streams** — text and image each get their *own* projection weights, their own MLP, their own normalization — but they **attend jointly** (the attention is computed over the concatenation, so the streams exchange information), and

then split back apart. Each modality gets to develop its own representation while still being able to look at the other.



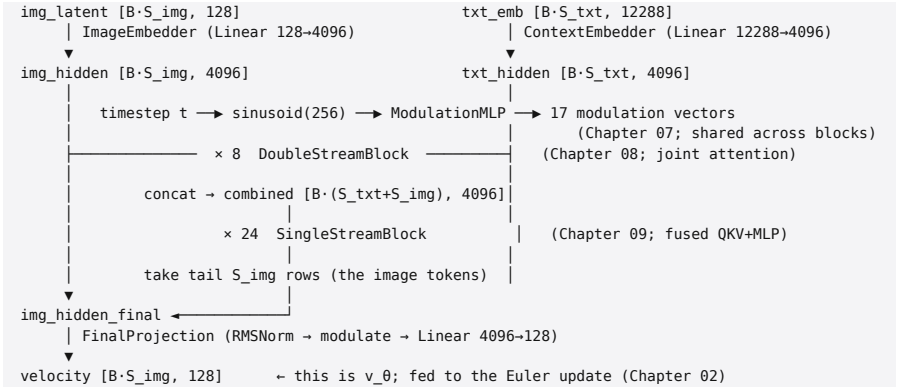
This is a **double-stream block** (Chapter 08). FLUX.2-klein uses **8** of them.

After the double-stream stage, the two streams are **concaten ated into one sequence** and processed by a stack of **singl e-stream blocks** — ordinary transformer blocks over the combined `[txt ‖ img]` sequence, sharing one set of weights for both modalities. FLUX.2-klein uses **24** of these. The intuition: the early layers benefit from modality-specific processing; once text and image are "aligned," a unified stream is more parameter-efficient. Single-stream blocks also use a **fused** QKV+MLP layout that we will see is a nice bandwidth win (Chapter 09).

After the single-stream stack, the **image** tokens (the tail of the combined sequence) are pulled back out and projected to the velocity. The text tokens have done their job — conditioning the image — and are discarded.

4.4 The full topology

Here is the complete network, adapted from the diagram that ships in `flux_transformer.h`:



Every arrow is a real component with its own chapter. Note the **concat/split** transitions (`concat_two_streams`, `take_tail` kernels) at the double→single boundary and at the output — these are pure data-movement kernels, no math.

4.5 The dimensions, exactly

From `flux_transformer.h::Config` and `transformer/config.json`:

Parameter | Value | Meaning
num_double_blocks | 8 | two-stream blocks
num_single_blocks | 24 | unified blocks → 32 layers total
n_heads | 32 | attention heads
head_dim | 128 | per-head dimension → this is the D=128 of Part IV
hidden d | 4096 | =32×128
ffn_dim | 12288 | MLP inner width (mlp_ratio=3.0)
t5_dim / joint_attention_dim | 12288 | context-embedder input (3 × Qwen3's 4096; Chapter 05)
in_channels | 128 | image-token channel dim (Chapter 03)
time_dim | 256 | timestep embedding width
axes_dims_rope | [32,32,32,32] | 4-axis RoPE (Chapter 06)
rope_theta | 2000 | RoPE base — not the usual 10000
RMSNorm eps | 1e-6 |
guidance_embeds | false | distilled klein: no classifier-free guidance (Chapter 02)

A few of these deserve a flag now, expanded later:

• `t5_dim = 12288` **is a misnomer.** It is not T5. FLUX.2-klein's text encoder is **Qwen3-8B**, and 12288 = 3 × Qwen3's hidden size 4096 — the context embedder consumes a *concatenation of three Qwen3 layer hidden states*. Chapter 05.

• `rope_theta = 2000` (not 10000) and **4-axis** RoPE are FLUX specifics that the position-encoding kernels must implement exactly. Chapter 06.

• **The single-stream blocks fuse their linears.** The QKV projection and the MLP in/out are stored as single large matrices (`to_qkv_mlp_proj` is `[36864, 4096]` = 9×4096; `to_out` is `[4096, 16384]` = 4×4096). This is deliberate — one big GEMM moves weights more efficiently than several small ones on a bandwidth-bound machine. Chapter 09.

4.6 A structural surprise: shared modulation weights

A timestep produces **modulation** signals — the per-layer scale/shift/gate that tell each block how hard to act at this noise level (AdaLN, Chapter 07). In many DiTs each block has its own modulation projection. In FLUX.2-klein the modulation projections are **shared across all blocks of a type**: there is one `double_stream_modulation_img.linear`, one `..._txt.linear`, one `single_stream_modulation.linear`, each emitting all the vectors that *every* block of that type consumes. (Confirmed by the tensor router: 8 double blocks × 16 tensors + 24 single blocks × 4 tensors + 9 globals = 233 tensors, with no per-block modulation linears — Chapter 14.) This is a distilled-model economy: it cuts parameters with little quality loss. The `ModulationMLP` (Chapter 07) computes all 17 modulation vectors once per forward pass from the single timestep embedding.

4.7 Construction and cost

`FluxTransformer`'s constructor builds 124 `Linear` instances (8 double-blocks × 12 + 24 single-blocks × 2 + embedders + modulation + final projection), each repacking its quantized weights into the tensor-core-friendly layout (Chapter 13). On the GB10 this is ~5.3 s one-time (the "transformer build" line in Chapter 00), and the forward pass over all 32 layers is the per-step cost in the denoise loop. The workspace — all the intermediate token buffers — is a few GiB at 1024px, which u nified memory absorbs without complaint (Chapter 11).

The whole thing is verified end to end by `tests/test_flux_tran sformer.cu` (runs a forward on real weights, checks finiteness and magnitudes) and, more stringently, by `tools/cmp_transfor mer.cu` against a diffusers reference at cos-similarity 0.963 in NVFP4 / 0.999 in MXFP8 (Chapter 25).

4.8 What each remaining Part-II chapter covers

- **05 — Text conditioning with Qwen3.** Where the `[512 × 12288]` comes from, the three-layer concatenation, and the context embedder.
- **06 — Rotary position embeddings.** RoPE, the 4-axis variant for 2D images, and why θ=2000.
- **07 — Modulation / AdaLN.** The timestep → 17 modulation vectors MLP and how a block consumes scale/shift/gate.
- **08 — Double-stream blocks.** The two-stream, joint-attention block in full, including the per-stream QK-norm.
- **09 — Single-stream blocks.** The fused QKV+MLP block and the split/concat plumbing.
- **10 — Assembling the transformer.** Embedders, the final projection, the stream transitions, and the top-level forward.

4.9 Summary and what to carry forward

• The velocity network is a **transformer over tokens**, not a U-Net; image patches and text vectors are both just tokens once projected to hidden width 4096.

• **MMDiT** runs **8 double-stream blocks** (separate text/image weights, *joint* attention) then **24 single-stream blocks** (unified, fused linears), then pulls the image tokens out and projects to the velocity. 32 layers total.

• Key FLUX specifics to remember: text encoder is **Qwen3** (so `t5_dim`=12288 is 3×4096), **4-axis RoPE** with θ=2000, **fused** single-stream linears, and **shared modulation** weights (a distilled-model economy).

• Everything maps to `flux_transformer.{h,cu}` and is the function we spend Parts III–IV making fast.

Chapter 05 starts at the left edge of the diagram: how a prompt becomes the `[512 × 12288]` conditioning that enters the context embedder.

Exercises

1. **Token budget.** At 1024px, S_img=4096 and S_{txt}=512. T hrough which blocks does the sequence have length 4096, and through which 4608? Relate to the attention benchmark shapes in Part IV.

2. **Two streams vs one.** Give one reason early layers benefit from modality-specific weights and one reason later layers don't need them. What does sharing weights in the single-stream stage save on a bandwidth-bound machine?

3. **Parameter accounting.** Using the dimensions table, estimate the parameter count of one double-stream block (12 linears) and one single-stream block (2 fused linears), and sanity-check that 8+24 of them approach ~9B.

4. **Shared modulation.** Why can modulation weights be

shared across blocks when attention/MLP weights cannot? (Hint: what does modulation depend on vs what attention depends on?)

Next: Chapter 05 — Text conditioning with Qwen3.

Chapter 05 — Text Conditioning with Qwen3

> *Goal of this chapter:* explain how a text prompt becomes the `[512 × 12288]` conditioning tensor that steers the diffusion. We cover why FLUX.2-klein uses a full **8-billion-parameter LLM (Qwen3-8B)** as its text encoder, how a *decoder-only causal* language model is repurposed as an *encoder*, the three-layer hidden-state concatenation that produces the 12288 dimension, the chat-template wrapping, and the context embedder that brings it into the model. The encoder's GPU *implementation* (grouped-query attention, q/k-norm, etc.) is Chapter 21; here we cover its *role*.
>
> *Prerequisites:* Chapter 04 (where conditioning enters); transformers at a conceptual level.

5.1 Why an 8B LLM is the text encoder

Earlier text-to-image models used CLIP text encoders (~100M params) or T5-XXL (~5B). FLUX.2-klein uses **Qwen3-8B** — a frontier-class large language model — as its text encoder. Why spend 8B parameters just to *read* the prompt?

Because prompt understanding is the bottleneck for instruction-following. A small text encoder produces embeddings that capture keywords but not composition, negation, counting, spatial relations, or world knowledge. A large LM has *already learned* these from language pretraining; its hidden states encode a rich, almost "thought-through" representation of the prompt. The diffusion transformer then has something far more informative to attend to. The cost is real (Qwen3-8B is bigger than some image backbones), but it runs **once** per image (Chapter 00) and its output is reused

across all denoise steps, so it is amortized.

> This is a general trend: as image models got better at *renderi ng*, the limiting factor became *understanding the request*, and the field responded by bolting on ever-larger language models as the front end.

5.2 A decoder-only LM used as an encoder

Qwen3 is a **decoder-only, causal** language model — the GPT family architecture. Normally it is run autoregressively to *generate* text: token by token, each position attending only to earlier positions (the causal mask). FLUX uses it differently: it feeds the *entire prompt at once* and reads out the **hidden states at every position** — the internal activations — rather than sampling output tokens. No generation happens; the LM is run as a single forward pass and its intermediate representations *are* the conditioning.

A few consequences of using a causal model this way:

- **Causal masking still applies.** Position i's hidden state summarizes tokens 0..i. The later positions therefore carry the most context, which matters for the layer-selection in §5.4.
- **A chat template is used** (§5.3), because Qwen3 was instruction-tuned and its representations are best-behaved when the input looks like the chat format it was trained on.
- **The padding tokens are encoded too.** The prompt is padded to a fixed length (512); diffusers feeds all 512 positions, padding included, with no attention mask into the *d iffusion* transformer (the image tokens simply attend to all 512, and the model learned to ignore padding). This is a quirk worth remembering when validating (Chapter 25).

5.3 The chat template

Qwen3 expects its instruction-tuned chat format. The prompt is wrapped exactly as `apply_chat_template([{role:user, content:PROMPT}], add_generation_prompt=True, enable_thinking=False)` produces (this is reproduced bit-for-bit by our native tokenizer, Chapter 22):

```
<|im_start|>user\n{PROMPT}<|im_end|>\n<|im_start|>assistant\n<think>\n\n</think>\n\n
```

The special tokens (`<|im_start|>`, `<|im_end|>`, `<think>`, `</think>`) are single token ids; the prompt and the literal words `user`/`assistant`/newlines are byte-level BPE'd. The trailing empty `<think>\n\n</think>` blocks (with `enable_thinking=False`) put the model in its "no chain-of-thought" mode. Feeding the *raw* prompt without this wrapper shifts the activations off the distribution Qwen3 was tuned on and measurably degrades conditioning — the template is not optional decoration.

The wrapped text is tokenized, padded/truncated to **512** tokens with the pad token (`<|endoftext|>`, id 151643), and fed to the encoder. (The padding side is right; the wrapper and padding are exactly what `tools/encode_prompt.py` and our native `BpeTokenizer` produce — Chapter 22.)

5.4 The three-layer concatenation → 12288

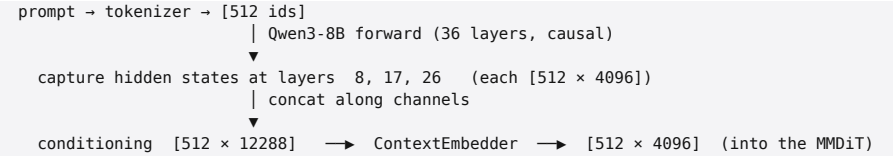
Here is the detail that explains the mysterious `t5_dim = 12288` from Chapter 04. Qwen3-8B has hidden size **4096** and **36** layers. FLUX does **not** use just the final hidden state; it extracts the hidden states at **three specific layers** and concatenates them along the channel dimension:

$$3 \times 4096 = 12288 \ .$$

So the conditioning is `[512 tokens × 12288]`, where each token's 12288 vector is `concat(h_layerA, h_layerB, h_layerC)` for three chosen layers. Using multiple layers gives the diffusion model a mix of representational depths — earlier

layers carry more surface/lexical features, later layers more abstract/semantic ones.

Which three layers? This is a genuine gotcha with an off-by-one. diffusers requests `output.hidden_states[k] for k in (9, 18, 27)`. But HF's `hidden_states` array is offset: `hidden_states[0]` is the *embedding* output, and `hidden_states[k]` for k≥1 is the output of *layer k-1*. So the actual layer outputs to capture are **layers {8, 17, 26}** (zero-indexed). This project's encoder captures exactly those (`capture_layers = {8, 17, 26}` in `generate.cu`). Getting this wrong (capturing {9,18,27}) silently shifts every conditioning vector by one layer — the kind of bug that produces *plausible but subtly wrong* images. (Appendix B; Chapter 21 for the capture mechanism.)



5.5 The outlier activations

A practical wrinkle: Qwen3's late-layer activations contain **large-magnitude outliers** — diffusers' actual `prompt_embeds` has `abs_max ≈ 22528` at the real-prompt (non-padding) positions, from activation spikes at chat-template tokens. These are not errors; they are how the model concentrates information. They matter for *quantization* (Chapter 12): a per-block scale that has to span a 22528 outlier could crush the small values sharing its block. The project investigated whether NVFP4 was destroying conditioning via these outliers and found it was *not* — the per-microblock E4M3 scales handle them correctly (Chapter 25, the text-attenuation investigation). Worth knowing the outliers exist when you read the quantization chapters.

5.6 The context embedder

The `[512 × 12288]` conditioning is not yet in the transformer's world; it must be projected to the hidden width 4096. That is the **context embedder**: a single `Linear` with weight `context_embedder.weight` of shape `[4096, 12288]`. It maps each text token's 12288-vector to a 4096-vector that lives in the same space as the image tokens, ready for joint attention (Chapter 08).

```
// conceptually, per text token:
txt_hidden[i] = context_embedder( conditioning[i] );   // [12288] → [4096]
```

This is the `ContextEmbedder` in `src/backend/cuda/embeddings.{h,cu}` (Chapter 10), and it is the entry point on the text side of the Chapter 04 diagram. From here the text tokens flow through the 8 double-stream blocks (developing their own rep resentation while the image attends to them) and into the single-stream stack.

5.7 Three ways to supply conditioning (and why)

`generate` can inject text at three points, which is invaluable for isolating bugs (and is how the text path was debugged — Chapter 25):

Flag | Skips | Use
`--prompt "<text>"` | nothing | the full native path (tokenizer → Qwen3 → embedder)
`--tokens <file>` | tokenizer | feed precomputed token ids (test the encoder)
`--embeds <file>` | tokenizer + encoder | feed a precomputed [512 × 12288] (test the transformer in isolation)

The `--embeds` path was the key debugging lever: by feeding diffusers' *own* known-good conditioning into our transformer, the team proved the text path's residual problems were in the C++ transformer (the modulation bug, Chapter 07/25), not in the encoder — because even perfect conditioning produced

the wrong image until the modulation fix landed. The native `--prompt` path (Chapter 22) later made the whole thing self-contained and ~350× faster than the PyTorch `--embeds` generation (0.28 s vs 98 s).

5.8 Where this lives in the code

Concept | Code
Qwen3 encoder (full impl) | qwen_encoder.{h,cu}, qwen_layer.*, qwen_attention.* (Chapter 21)
capture layers {8,17,26} | generate.cu qwen config
chat template + tokenization | bpe_tokenizer.{h,cpp}, encode_prompt.py (Chapter 22)
context embedder (12288→4096) | embeddings.{h,cu} ContextEmbedder (Chapter 10)
validation vs HF Qwen3 | tools/qwen3_golden.py, tests/test_qwen_golden.cu (cos ≥ 0.974/layer)
validation vs diffusers embeds | tools/diffusers_prompt_embeds.py, --embeds path

5.9 Summary and what to carry forward

- The text encoder is **Qwen3-8B**, a decoder-only causal LLM run as a single forward pass; its **hidden states** are the conditioning, not generated text.
- The conditioning is `[512 × 12288]`, where 12288 = **3 × 4096** from concatenating hidden states at **layers {8, 17, 26}** (mind the HF off-by-one).
- A **chat template** wraps the prompt; padding is to 512 and is fed through.
- Late-layer **outlier activations** (~22528) exist and matter for quantization but are handled correctly by NVFP4's per-microblock scales.
- The **context embedder** (`[4096, 12288]` Linear) projects conditioning into the hidden space; from there it joins the image tokens in the double-stream blocks.

Chapter 06 covers how *position* is encoded for these tokens — the 4-axis RoPE that distinguishes image token (h,w) from text token ell — before Chapter 07's modulation and the

block chapters.

Exercises

1. **Why hidden states, not logits.** Explain why FLUX reads Qwen3's hidden states rather than its output token probabilities. What information would the logits throw away?
2. **The off-by-one.** Given HF's convention (`hidden_states[0]`=embedding, `hidden_states[k]`=output of layer k-1), show that diffusers' `(9,18,27)` corresponds to layer outputs `{8,17,26}`. What would capturing `{9,18,27}` do?
3. **12288 decoded.** Connect `t5_dim=12288` (Chapter 04 config), `joint_attention_dim=12288`, and `3 × 4096`. Why is "t5_dim" a misnomer?
4. **Isolation logic.** You get a wrong image. Describe how `--embeds`, `--tokens`, and `--prompt` let you localize whether the fault is in the tokenizer, the encoder, or the transformer.

Next: Chapter 06 — Rotary position embeddings.

Chapter 06 — Rotary Position Embeddings

Goal of this chapter: derive RoPE from first principles — including the relative-position property that is its whole point — then extend it to the **4-axis** form FLUX uses to give 2D image patches and 1D text tokens distinct positions. We pin down the two pairing conventions present in this codebase (interleaved for the FLUX transformer, half-rotation for Qwen3), the table construction, and why θ=2000. Anchored to `kernels/rope_4axis.{h,cu}` and `kernels/rope.{h,cu}`.

Prerequisites: Chapter 04; attention's QK^→p (Chapter 16 derives it, but "scores are dot products of queries and keys" suffices); 2×2 rotation matrices, complex exponentials.

6.1 The problem: attention is position-blind

Attention computes scores from dot products q_i· k_j of query and key vectors. Nothing in that operation knows *where* tokens i and j are — permute the tokens and the set of pairwise scores is unchanged. But position is essential: "a cat on a dog" ≠ "a dog on a cat", and image patch (0,0) is not patch (63,63). We must inject position into the tokens.

The classic fix (original Transformer) adds a fixed sinusoidal vector to each token — *absolute* position, added to the content. RoPE (Su et al., 2021) does something more elegant: it **rotates** the query and key vectors by an angle proportional to position, so that the *dot product* of a query and key ends up depending only on their **relative** position i-j. Relative position is what attention actually wants (how far apart are these tokens), and RoPE bakes it into the geometry rather

than adding it as content.

6.2 RoPE in one pair of dimensions

Take a single 2D sub-vector (x_0, x_1) of a query (or key) at sequence position m. RoPE rotates it by angle mθ for some frequency θ:

$$\begin{pmatrix} x_0' \\ x_1' \end{pmatrix} = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} R(m\theta) \begin{pmatrix} x_0 \\ x_1 \end{pmatrix}.$$

It is cleanest in complex notation: identify (x_0,x_1) with $z = x_0 + i\, x_1$, and the rotation is just multiplication by a unit complex number, $z' = e^{i m\theta} z$.

The relative-position property. Let a query pair at position m be $q_m = e^{im\theta} q$ and a key pair at position n be $k_n = e^{in\theta} k$. The contribution of this pair to the attention score is the real part of $q_m \overline{k_n}$:

$$q_m \overline{k_n} = e^{im\theta} q \overline{e^{in\theta} k} = e^{i(m-n)\theta} q\bar{k}.$$

The phase depends only on m-n. So **after RoPE, the score from this pair depends only on the relative offset m-n and the (rotation-invariant) content interaction $q\bar{k}$.** No absolute position leaks in; translate both tokens by the same amount and the score is unchanged. This is exactly the inductive bias attention wants, achieved with zero added parameters and zero added content — purely a rotation.

6.3 RoPE across a full head: a bank of frequencies

A head has D dimensions, not 2. RoPE splits them into D/2 pairs and gives each pair its own frequency, geometrically spaced (the same idea as sinusoidal embeddings):

$$\theta_k = base^{(-2k/D)}, k = 0,1,\ldots,tfracD2-1 .$$

Low-k pairs rotate fast (encode fine, short-range position); high-k pairs rotate slowly (encode coarse, long-range position). At position m, pair k is rotated by m θ_k. The per-position rotation angles are precomputed into **cos/sin tables** of shape [seq, D/2] — exactly the tables the kernels ta ke.

> `base` **/ θ choice.** Qwen3 (text) uses base=10^6; the FLUX transf ormer uses **base=2000** (`rope_theta=2000`, not the usual 10000). A smaller base makes all frequencies higher → faster rotation per unit position. Image patch grids are small (e.g. 64×64) compared to LLM context lengths, so a base tuned for thousands-of-tokens text would barely rotate across a 64-wide grid; θ=2000 spreads the rotations sensibly over the patch ra nge. This single number must match the trained model exactly — it is wired as a parameter, not hard-coded to 10000.

6.4 Two pairing conventions (both live in this repo)

"Split D into pairs" hides a convention choice: *which* two coordinates form a pair. Two are common, and **this codebase contains both**, deliberately:

- **Interleaved** `(2k, 2k+1)` — adjacent coordinates are paired. This is the FLUX2 / diffusers convention (`apply_rotary_ emb` with `use_real_unbind_dim=-1`). Implemented in `rope_4axis.c u`:
- **Half-rotation** `(i, i+D/2)` — coordinate i is paired with the one D/2 away. This is the LLaMA / Qwen3 convention. Implemented in `rope.cu`:

The two are related by a fixed permutation of the channel order and are *not* interchangeable — applying the wrong one

rotates the wrong coordinate pairs and silently corrupts the geometry. The FLUX transformer's Q/K use the **interleaved** k ernel; the Qwen3 encoder (Chapter 21) uses the **half-rotatio n** one. (The header comment in `rope.cu` was once wrong about which it implemented — corrected after verifying against HF; Appendix B.)

6.5 From 1D to 4 axes: positioning a 2D image

Text is 1D (token index ell). An image is 2D (patch row h, patch column w). A single scalar position cannot distinguish patch (3,5) from patch (5,3). FLUX solves this with **multi-axis RoPE**: partition the head's D=128 dimensions into **4 axis-groups** and give each group its own position coordinate. From the config, `axes_dims_rope = [32, 32, 32, 32]` — four axes labeled (T, H, W, L):

axis	width	meaning	image token	text token
T	32	temporal/frame	0	0
H	32	patch row	h	0
W	32	patch column	w	0
L	32	sequence index	0	ell

Each axis applies ordinary RoPE within its 32 dims, using that axis's coordinate. The elegance: **image tokens carry position only in (H,W)**, while **text tokens carry position only in L** — they live in *orthogonal* position subspaces. So ani mage token at (0,h,w,0) and a text token at (0,0,0,ell) are positioned without interfering, yet share the same head and attend jointly (Chapter 08). The T axis is reserved for video/frames and is 0 here.

```
head_dim = 128  =  [ T:32 | H:32 | W:32 | L:32 ]
                      |      |      |      |
image patch (h,w):    0      h      w      0     → rotates in H,W subspaces
text token  l    :    0      0      0      l     → rotates in L subspace
```

This is built by `build_rope_4axis_tables(axes_dim,`

`token_positions, theta, ...)`: the host passes, for each token, its [T,H,W,L] coordinates; the helper computes the per-axis frequencies and bakes the per-token rotation into flat `[seq, head_dim/2]` cos/sin tables. The device kernel `rope_4axis_inplace_bf16` then doesn't need to know about axes at all — it just applies the interleaved rotation using the prebaked tables. **Three tables are built per forward pass**: one for the image-stream length (image positions), one for the text-stream length (text positions), and one for the combined single-stream sequence.

6.6 Where RoPE is applied in the model

RoPE rotates the **queries and keys** (not values) just before attention, inside each block:

- **Double-stream blocks (Chapter 08):** RoPE is applied to **both** the image Q/K (with image positions (0,h,w,0)) **and** the text Q/K (with text positions (0,0,0,ell)). An earlier version skipped RoPE on the text stream; adding it changed the image and was one of the fixes on the road to working text conditioning (Appendix B).
- **Single-stream blocks (Chapter 09):** applied to the combined sequence's Q/K with the combined position table (text-first, then image).
- **Qwen3 encoder (Chapter 21):** uses 1D half-rotation RoPE on its Q/K, with its own $\theta=10^6$, applied *after* the per-head q/k-norm (a Qwen3 quirk).

RoPE acts on Q and K only because the relative-position phase must enter the score q_i· k_j; values carry content that is summed by the (already position-aware) attention weights, so they are left unrotated.

6.7 Implementation notes

• **Tables are FP32, data is BF16.** The cos/sin tables are kept in FP32 for precision (the rotations compose multiplicatively over long sequences); the Q/K data is BF16. The kernel reads FP32 angles and writes BF16 rotated values in-place.

• **In-place, one CTA per row.** `rope_4axis_inplace_bf16` rotates each [head_dim] row in place; `batch_rows = batch · seq · n_heads`. It is a cheap, memory-bound elementwise kernel — negligible next to the GEMMs and attention.

• **Correctness.** `tests/test_rope_4axis.cu` checks the kernel's interleaved rotation against an FP32 host reference; `tests/test_rope.cu` does the same for the half-rotation variant. The 4-axis math (interleaved (2k,2k+1), per-axis concat layout, θ^(-2k/d_a) frequencies, positions img =(0,h,w,0) with flat index s=h· W+w, txt =(0,0,0,ell), single-stream text-first) was verified against diffusers during the text-path debugging (Chapter 25).

6.8 Summary and what to carry forward

• RoPE injects position by **rotating Q/K pairs by angles proportional to position**; the dot-product then depends only on **relative** position — the bias attention wants, with no parameters and no added content.

• A head uses a **bank of geometric frequencies** θ_k=base^(-2k/D); FLUX's base is **2000**, tuned to image-patch ranges (not 10000).

• Two pairing conventions exist here: **interleaved (2k,2k+1)** for the FLUX transformer, **half-rotation (i,i+D/2)** for Qwen3 — not interchangeable.

• **4-axis RoPE** (T,H,W,L) positions 2D image patches in (H,W) and text tokens in L, in orthogonal subspaces, so both can share joint attention; tables are prebaked per token by `build_rope_4axis_tables`.

• Applied to Q/K only, in every block (text stream included), and in the encoder.

Chapter 07 covers the other thing injected into every block — the **timestep**, via AdaLN modulation — completing the inputs a block needs before we assemble the blocks themselves in Chapters 08–09.

Exercises

1. **Relative position.** Re-derive $q_m \overline{k_n} = e^{i(m-n)\theta} q \bar{} k$ and state in words why this makes attention scores translation-invariant.
2. **Base intuition.** For D=32 and a 64-wide axis, compare the rotation of the slowest pair (k=D/2-1) across positions 0→63 under base 10000 vs 2000. Argue why 2000 is better for image grids.
3. **Convention swap.** You apply the half-rotation kernel to FLUX Q/K (which want interleaved). Which coordinate pairs get rotated, and why does the image come out wrong rather than catastrophically broken?
4. **Orthogonal axes.** Explain why putting image position in (H,W) and text position in (L) lets the two modalities share a head without their positions interfering. What would break if text tokens also had nonzero H?

Next: Chapter 07 — Modulation / AdaLN.

Chapter 07 — Modulation / AdaLN

Goal of this chapter: explain how the **timestep** (and, in non-distilled models, guidance) steers every layer of the transformer through **adaptive layer normalization (AdaLN)**. We derive what scale/shift/gate do, trace the `ModulationMLP` that turns a 256-dim timestep embedding into **17** modulation vectors, and dissect the `modulate` and `gated_residual` kernels. We end on the **scale/shift ordering bug** — the single most important bug in this project's history, and a perfect lesson in convention-matching. Anchored to `modulation_mlp.{h,cu}` and `kernels/modulation.{h,cu}`.

Prerequisites: Chapters 02 (timestep), 04 (block topology); layer norm.

7.1 The job: tell each layer "how noisy is it right now"

The transformer is called once per denoise step, at different noise levels t (Chapter 02). The *same weights* must behave differently at t≈1 (almost pure noise — commit broad structure) than at t≈0 (almost clean — refine fine detail). The network needs a per-step "knob" that re-tunes every layer's behavior based on t.

That knob is **modulation**: from the timestep embedding, compute a small set of vectors that *scale, shift, and gate* the activations inside each block. This is the mechanism by which a single set of weights implements a whole trajectory of behaviors across noise levels. In conditional models the same mechanism also folds in class/guidance signals; in distilled klein the only input is the timestep (`guidance_embeds=false`, Chapter 02).

7.2 AdaLN: adaptive layer normalization

Standard layer norm normalizes a token vector to zero mean / unit variance, then applies a *learned, fixed* affine γ x + β. **Adaptive** layer norm (AdaLN, from the DiT paper) replaces the fixed γ,β with values *predicted from a conditioning signal* — here, the timestep. A block's normalized activations are modulated as

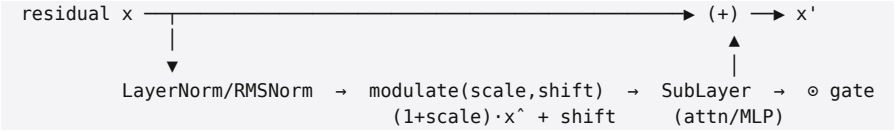
$$modulate(x; scale,shift) = (1+scale)\odot\hat{x} + shift,$$

where $\hat{x}$ is the normalized activation and scale,shift are per-channel vectors predicted from t. The "1+" means *zero* modulation is the identity (scale 0, shift 0 → pass-through), which makes initialization and zero-gate behavior clean.

AdaLN-**Zero** (what DiT/FLUX use) adds a third predicted vector, a **gate**, that multiplies the block's *output* before it is added back to the residual stream:

$$x \text{ arrow } x + gate\odot SubLayer\ (modulate(norm(x);\ scale,shift)\ ).$$

With gate=0 the sub-layer contributes nothing and the block is the identity — so at initialization every block is a no-op and the network starts as a clean residual highway, which stabilizes training. Each attention or MLP sub-layer gets its **own** (scale, shift, gate) triple.



This is the structure of *every* block in FLUX. A block has two sub-layers (attention, then MLP), so it consumes **two** (scale, shift, gate) triples = **6 modulation vectors** per stream.

7.3 Counting to 17

How many modulation vectors does one forward pass need?

Recall (Chapter 04) the modulation projections are **shared across blocks of a type**, so the `ModulationMLP` emits, once per forward pass, the full set consumed by every block:

```
Consumer | triples | vectors | weight shape
Double-stream image (attn + MLP) | 2 | 6 | double_stream_modulation_img.linear [24576, 4096] = 6×4096
Double-stream text (attn + MLP) | 2 | 6 | double_stream_modulation_txt.linear [24576, 4096]
Single-stream (one sub-layer pair) | 1 | 3 | single_stream_modulation.linear [12288, 4096] = 3×4096
Final norm-out (AdaLayerNormContinuous) | — | 2 | norm_out (scale, shift; no gate)
```

That is 6 + 6 + 3 + 2 = 17 modulation vectors. The double/single counts are *per block type* (shared across all 8 / all 24 blocks); the final projection (Chapter 10) uses a 2-vector (scale, shift only) modulation. These 17 vectors are e xactly the fields of `DoubleStreamBlock::Modulation`, `SingleStrea mBlock::Modulation`, and the final norm in the code.

7.4 The ModulationMLP

`ModulationMLP` (`modulation_mlp.{h,cu}`) computes all 17 vectors from the 256-dim timestep embedding:

```
timestep_emb [256]                        (from sampler.h, cat(cos,sin), t×1000)
     │ linear_1 → [4096]
     │ SiLU
     │ linear_2 → [4096]
     │ SiLU
     ├── proj_double_img → [24576]  = 6 × 4096   (img: scale/shift/gate × {attn, mlp})
     ├── proj_double_txt → [24576]  = 6 × 4096   (txt: same)
     ├── proj_single     → [12288]  = 3 × 4096   (single-stream triple)
     └── proj_norm_out   → [8192]   = 2 × 4096   (final scale, shift)
```

Six NVFP4/MXFP8 `Linear`s and a `silu_inplace_bf16` kernel. The two-layer MLP with SiLU gives the timestep a small nonlinear network before it fans out into the four projections. Each projection's output is sliced into its constituent 4096-vectors, which the blocks then index. On real weights this runs once per forward pass and its 17 outputs are tiny perturbations (trained modulation parameters have small magnitudes) — `ma`

x≈1.34, mean-of-means ≈0.07 — exactly what you expect for AdaLN deltas around the identity.

The structatic output is a struct of pointers into the projection buffers, one per modulation vector, matching the fields each block reads. (Implementation detail: the MLP runs at an internal batch of 128 rows to satisfy the Linear GEMM's M%128=0 constraint — Chapter 13 — and the real vectors live in row 0.)

7.5 The two kernels: modulate and gated_residual

The actual arithmetic is two tiny elementwise kernels (kernels/modulation.{h,cu}):

```
// modulate: (1 + scale) * x + shift,  scale/shift are [hidden] broadcast over rows
modulate(x, scale, shift):   x[r,c] = (1 + scale[c]) * x[r,c] + shift[c]

// gated_residual: y += gate * delta,  gate is [hidden] broadcast over rows
gated_residual(y, delta, gate):   y[r,c] += gate[c] * delta[r,c]
```

scale, shift, gate are BF16 [hidden=4096] vectors broadcast across all token rows. Both kernels are memory-bound and trivial; the interesting part is not the arithmetic but getting the *operands* right — which is the subject of §7.6. (modulate reaches ~0.004 max abs error vs an FP32 reference — pure BF16 rounding.)

7.6 The bug: scale/shift order (a case study in convention-matching)

This is the most instructive bug in the project, so we dwell on it.

A modulation projection emits a block of 3 × 4096 (or 2 × 4096) numbers that must be *sliced* into the named vectors. The question is the **order** of the slices. The original implementation sliced every Flux2Modulation output as [scale

`, shift, gate]`. But diffusers' `Flux2Modulation.split` actually emits `[shift, scale, gate]` — the destructuring in `transformer_flux2.py` is `(shift_msa, scale_msa, gate_msa)`, shift **first**.

With the wrong order, every block computed

(1+shift)⊙ˆx + scale instead of (1+scale)⊙ˆx + shift,

i.e. it swapped the multiplicative and additive roles of two vectors in **all 8 double + 24 single blocks**. The effect was subtle and devastating: the image came out *recognizable but barely prompt-dependent*. Quantitatively, before the fix a cat prompt vs a truck prompt differed by 0.12% in pixels while two random seeds differed by ~10% — the latent dominated the prompt 40-to-1. AdaLN is precisely the channel through which conditioning modulates the network, and corrupting it t hrottled the text path to a whisper.

The fix swapped the slice order to `[shift, scale, gate]` for the img/txt/single modulations. **Effect:** prompt influence (cat vs truck) jumped from 0.12% to **12.5%** — about 100× stronger — and became *dominant* over the latent. That one ordering change is what turned the pipeline from "produces an image that ignores the prompt" into "produces the requested image."

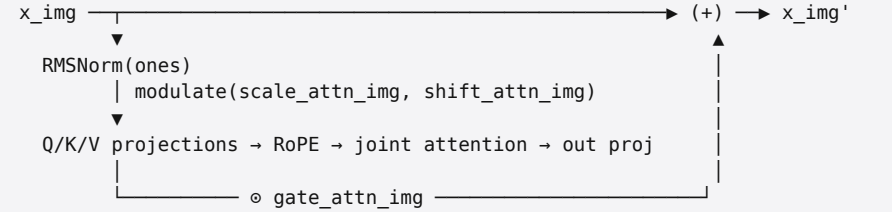
The twist: `norm_out` is **not** a `Flux2Modulation`. It goes through `AdaLayerNormContinuous`, whose convention *is* `scale, shift = chunk(emb, 2)` — **scale first**. So the final projection keeps scale-first while the block modulations are shift-first. Two different conventions in the same model, and you have to honor both. (Appendix B documents the full saga; Chapter 25 has the isolation method that found it.)

> **The general lesson.** When porting from a reference, the *ord er* of packed sub-tensors is a silent contract. A swap of two vectors will not crash, will not NaN, and will often produce *pla*

usible output — which is far more dangerous than an obvious failure. The only defense is to diff intermediate tensors against the reference (Chapter 25), not just eyeball the final image.

7.7 How a block consumes its modulation

To close the loop, here is how one double-stream image sub-layer (say attention) uses its triple, foreshadowing Chapter 08:



The same pattern repeats for the image MLP sub-layer (with `scale_mlp_img, shift_mlp_img, gate_mlp_img`), and symmetrically for the text stream — six vectors per stream, all sourced from the shared `ModulationMLP` output.

7.8 Where this lives in the code

Concept | Code
timestep → 17 vectors | modulation_mlp.{h,cu} (ModulationMLP), silu_inplace_bf16
(1+scale)·x+shift | kernels/modulation.{h,cu} modulate
y += gate·delta | kernels/modulation.{h,cu} gated_residual
slice order (shift,scale,gate) | modulation_mlp.cu (the fix)
norm_out scale-first | final projection (Chapter 10)
tests | tests/test_modulation.cu, tests/test_modulation_mlp.cu

7.9 Summary and what to carry forward

• The **timestep** steers every layer via **AdaLN**: predict per-channel **scale, shift, gate** and apply (1+scale)ˆx+shift then gate the sub-layer output into the residual. Zero modulation = identity.

• One forward pass needs **17** modulation vectors (6 img + 6 txt double, 3 single, 2 norm-out), produced once by the `ModulationMLP` and **shared across blocks**.

• The arithmetic is two trivial broadcast kernels (`modulate`, `gated_residual`); the difficulty is **operand order**.

• The **scale/shift swap** bug throttled prompt conditioning ~100×; fixing the slice order to `[shift, scale, gate]` (while keeping `norm_out` scale-first) is what made the model follow prompts. *Diff intermediates, not just images.*

With position (Chapter 06) and modulation (this chapter) in hand, we have every input a block needs. Chapters 08–09 assemble the two block types.

Exercises

1. **Why "1+scale".** Show that (1+scale)ˆx+shift with scale =shift=0 is the identity, and explain why that is a desirable in itialization for AdaLN-Zero blocks.

2. **Gate as a no-op switch.** Argue that gate=0 makes a block the identity, and why a network of identity-initialized residual blocks is easy to train.

3. **The 17.** Reconstruct the count 6+6+3+2=17 from the block topology of Chapter 04, and explain why double has 6-per-stream but single has only 3.

4. **The swap, quantified.** Given that AdaLN is the only conditioning channel, explain mechanistically why swapping scale↔shift would *attenuate* prompt dependence rather than, say, add noise. Why did the image stay coherent?

Next: Chapter 08 — Double-stream blocks.

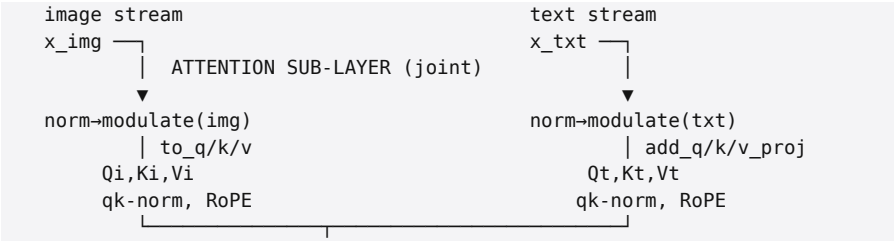
Chapter 08 — Double-Stream Blocks

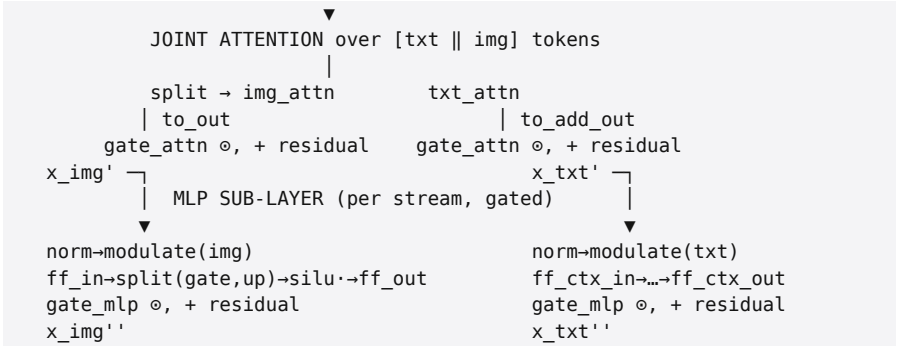
Goal of this chapter: assemble the first of FLUX's two block types in full. A double-stream block keeps **separate weights** for the text and image streams but couples them through **joint attention**. We walk the exact forward pass — norm → modulate → QKV → per-head QK-norm → RoPE → joint attention → projection → gated residual, then the gated MLP — name all 12 linears, and explain joint attention and QK-normalization. Anchored to `double_stream_block.{h,cu}` and `joint_attention.{h,cu}`.

Prerequisites: Chapters 06 (RoPE), 07 (modulation); attention conceptually (Chapter 16 for the kernel).

8.1 The shape of a block

Every transformer block is two residual sub-layers: an **attention** sub-layer and an **MLP** sub-layer, each wrapped in the AdaLN pattern of Chapter 07 (norm → modulate → sub-layer → gate → add). A *double*-stream block runs this structure **twice in parallel** — once for the image tokens with image weights, once for the text tokens with text weights — except that the attention sub-layer's QK^→p is computed **jointly** over the concatenation of both streams, so information crosses between them. FLUX.2-klein has **8** such blocks.





8.2 The attention sub-layer, step by step

Following `double_stream_block.cu` exactly, per stream:

1. Normalize (no-affine RMSNorm). `RMSNorm(x, ones)` — RMS normalization with a gain of all-ones, i.e. statistics-only. (Why RMSNorm and not LayerNorm: §8.5.) The learned affine that a normal norm would apply is *replaced* by the AdaLN modulate.

2. Modulate. `modulate(·, scale_attn, shift_attn)` — the AdaLN scale/shift for this stream's attention sub-layer (Chapter 07). Image uses `img_scale/shift_attn`, text uses `txt_scale/shift_attn`.

3. QKV projections. Three linears per stream produce queries, keys, values:

- image: `to_q`, `to_k`, `to_v` (weights `W_q/k/v`)
- text: `add_q_proj`, `add_k_proj`, `add_v_proj` (the `add_` prefix marks the text stream — the "additional" tokens added to the image's attention).

Each output is `[S × hidden]` and is viewed as `[S × n_heads × head_dim] = [S × 32 × 128]`.

4. Per-head QK-norm. Each head's query and key vectors are RMS-normalized with *learned* gains — `norm_q`/`norm_k` (image), `norm_added_q`/`norm_added_k` (text). This is **QK-normali**

zation (§8.4), a stabilizer that normalizes Q and K *per head* before the dot product. It uses the same `rmsnorm_bf16` kernel as elsewhere, called with `batch_rows = S × n_heads`.

5. RoPE. 4-axis rotary embedding (Chapter 06) on Q and K. **Both** streams get RoPE: image Q/K with image positions (0,h,w,0), text Q/K with text positions (0,0,0,ell). (The header comment in `double_stream_block.h` says "text gets NO RoPE" — that comment is **stale**; the `.cu` applies `rope_4axis_inplace_bf16` to `txt Q` and `txt K` with the text position table. Verifying the code over the comment is the right habit; adding text RoPE was one of the text-path fixes, Appendix B.)

6. Joint attention. The crux — §8.3. Produces `img_attn` and `txt_attn`.

7. Output projection + gated residual. `to_out` (image) / `to_add_out` (text) project the attention output back to hidden width, then `gated_residual(x, proj, gate_attn)` adds it into the stream scaled by the AdaLN gate.

8.3 Joint attention: where the streams talk

This is the only place text and image interact. Instead of attending within each stream separately, the block **concatenates** the two streams' Q, K, V and runs a single attention over the combined sequence of length S_{txt}+S_img, then splits the result back:

```
Q = [Q_txt ; Q_img]   K = [K_txt ; K_img]   V = [V_txt ; V_img]     (concat along tokens)
O = Attention(Q, K, V)          # every token attends to every token, both modalities
O_txt = O[:S_txt]    O_img = O[S_txt:]       (split back)
```

Because the attention matrix is over the union, an image token's output is a weighted sum over **all** text and image values, and vice versa — this is exactly how the prompt's content flows into the image representation. `joint_attention.cu` is a thin wrapper: it concatenates Q/K/V into workspace buffers, calls the single `Attention` kernel (Part IV) at the

combined length, and splits the output. The concat/split are one-CTA-per-row data-movement kernels; all the math is in the attention kernel, which is identical to the one used everywhere else. At 1024px the combined length is 512+4096 = 4608 — the S=4608 of the attention benchmarks (Part IV). cos-similarity of the wrapper vs an FP32 reference is 1.00000 for both output halves.

> Note the asymmetry that makes the two-stream design meaningful: the *attention* is shared (joint), but the *projections, norms, and MLP* are per-stream. Text and image develop distinct representations yet see each other through attention.

8.4 QK-normalization: why normalize queries and keys

Standard attention can suffer from attention-logit blow-up: if some query/key directions grow large, the QK^→p scores saturate the softmax and training destabilizes. **QK-norm** (used by Qwen3, FLUX, and others) RMS-normalizes each hea d's Q and K vectors *before* the dot product, bounding their magnitudes so the score scale stays controlled. Crucially these are **learned** per-head gains (`norm_q`, `norm_k`, etc.), unlike the no-affine RMSNorm at the block entry — the model can still set the effective temperature per head. QK-norm is applied **before RoPE** in this implementation (norm, then rotate), matching the reference. It is a small kernel call but a real correctness requirement: the learned gains are part of th e weights and skipping the norm changes the attention distribution.

8.5 The MLP sub-layer: a gated feed-forward

After the attention sub-layer updates the residual, each

stream runs an independent gated MLP (the AdaLN pattern again, with `scale_mlp/shift_mlp/gate_mlp`):

```
x → RMSNorm(ones) → modulate(scale_mlp, shift_mlp)
  → ff_in:   Linear → [2·ffn]            (a single fused projection)
  → split_half → (gate, up), each [ffn]  (split_half_bf16 kernel)
  → silu(gate) · up → [ffn]              (SwiGLU-style gating; silu_mul kernel)
  → ff_out:  Linear → [hidden]
  → gated_residual(x, ·, gate_mlp)
```

The MLP inner width is `ffn = 12288` (mlp_ratio 3.0). The first linear produces **twice** that (2×12288) and `split_half` cuts it into a *gate* and an *up* half; `silu(gate)·up` is the gated activation (the header calls it "GeGLU" but the activation is SiLU, i.e. a SwiGLU-style gate — what the code computes is `silu(gate)·up`); the second linear projects back to hidden. Text uses the parallel weights `ff_ctx_in`/`ff_ctx_out`. Gating MLPs like this consistently outperform a plain `Linear→act→Linear` at equal parameter budget, which is why modern transformers use them.

8.6 The twelve linears

A double-stream block owns **12** NVFP4/MXFP8 `Linear`s — six per stream:

	image weights	**text weights**
Q proj	to_q	add_q_proj
K proj	to_k	add_k_proj
V proj	to_v	add_v_proj
attn out	to_out	to_add_out
MLP in (→2·ffn)	ff_in	ff_ctx_in
MLP out (ffn→hidden)	ff_out	ff_ctx_out

plus 4 per-head QK-norm gains (`norm_q`, `norm_k`, `norm_added_q`, `norm_added_k`) and the modulation it *reads* (the shared `ModulationMLP` output, not owned by the block). 8 blocks × 16 tensors = the 128 double-block tensors the router accounts for (Chapter 14).

8.7 Correctness: the zero-gate identity

test

A beautiful property falls out of AdaLN-Zero (Chapter 07): if **all gates are zero**, both sub-layers contribute nothing and the block is the **exact identity**. `tests/test_double_stream_block.cu` exploits this: it runs a block with zeroed gates and checks the output equals the input **bit-for-bit** (`max_err = 0`). This single test proves the *entire wiring* is correct — every norm, modulate, QKV, RoPE, attention, projection, split, and residual connects properly, because any miswire would perturb the supposedly-untouched residual. A separate real-weights smoke test (loading `transformer_blocks.0` from the F2K file) checks the block produces finite, sanely-scaled outputs on actual data.

> The zero-gate identity test is a pattern worth stealing: design your residual components so that a known parameter setting makes them provably the identity, then test that exactly. It catches wiring bugs that numerical-tolerance tests miss.

8.8 Where this lives in the code

Concept	Code
the block	double_stream_block.{h,cu} (DoubleStreamBlock)
joint attention concat/split	joint_attention.{h,cu}
QK-norm	kernels/rmsnorm.{h,cu} (called per-head)
4-axis RoPE (both streams)	kernels/rope_4axis.*
MLP gate/up split	kernels/seq_ops split_half_bf16; swiglu/silu_mul
modulation it consumes	ModulationMLP (Chapter 07)
tests	tests/test_double_stream_block.cu, tests/test_joint_attention.cu

8.9 Summary and what to carry forward

- A double-stream block runs the AdaLN attention+MLP pattern **separately per stream** (own weights), but the attention's QK^→p is **joint** over `[txt ‖ img]`, which is how the prompt flows into the image.
- The attention path is: no-affine RMSNorm → AdaLN

modulate → QKV → **per-head QK-norm (learned) → 4-axis RoPE on both streams** → joint attention → out proj → gated residual.

• The MLP is a **SwiGLU-style gated** feed-forward (`ff_in`→split→`silu(gate)·up`→ `ff_out`), per stream.

• **12 linears + 4 norm gains** per block, ×8 blocks; modulation is shared.

• The **zero-gate identity test** proves the wiring bit-exactly. *Verify code over comments* — the "no text RoPE" comment is stale.

Chapter 09 covers the single-stream blocks — the other 24 layers — which fuse all of this into fewer, larger GEMMs.

Exercises

1. **Joint vs separate attention.** Explain what information an image token gains from *joint* attention that it could not get if text and image attended separately. Where in the diagram does the prompt enter the image stream?

2. **QK-norm placement.** Why is QK-norm applied *before* RoPE rather than after? What would rotating *then* normalizing change?

3. **Zero-gate proof.** Argue why zeroing all gates makes a double-stream block the exact identity, and why that makes a strong wiring test. What class of bug would it *not* catch?

4. **Fused MLP-in.** The MLP's first linear outputs `2·ffn` and is split into gate/up. Why is producing both halves in one GEMM better than two separate GEMMs on a bandwidth-bound machine? (Foreshadows Chapter 09/13.)

Next: Chapter 09 — Single-stream blocks.

Chapter 09 — Single-Stream Blocks

Goal of this chapter: the second block type — the 24 layers that do the bulk of the work. Single-stream blocks process the **combined** `[txt ‖ img]` sequence with **one** set of weights, and they **fuse** the QKV projection with the MLP input, and the attention output with the MLP output, into two big GEMMs computed in **parallel**. We walk the forward pass, explain the parallel attention+MLP formulation, and show why the fusion is a deliberate bandwidth optimization. Anchored to `single_stream_block.{h,cu}` and `kernels/qkv_mlp_split.*`.

Prerequisites: Chapters 07–08; the GEMM/`Linear` idea (Chapter 13 for depth).

9.1 What changes from double to single

After the 8 double-stream blocks, the text and image tokens are concatenated into one sequence of length S_{txt}+S_img and handed to **24 single-stream blocks**. Three things change:

1. **One stream, one set of weights.** No more separate text/image projections — both modalities flow through the same weights. (They are already "aligned" by the double-stream stage; Chapter 04.)
2. **Parallel attention + MLP.** Instead of two sequential sub-layers (attention *then* MLP), a single-stream block computes attention and the MLP **from the same normalized input, in parallel**, and merges them in a single output projection with **one** gated residual.
3. **Fused linears.** The QKV projection and the MLP-input projection are a single matrix; the attention-output and MLP-output projections are a single matrix.

This "parallel" transformer block (used by GPT-J, PaLM, and FLUX's single stream) trades a small amount of expressiveness for a large efficiency win: fewer, bigger GEMM s and one normalization instead of two.

9.2 The forward pass

From `single_stream_block.h`, in-place on the combined sequence `x` (`[B·S, 4096]`):

```
norm_x  = RMSNorm(x, ones)                      # statistics-only
mod_x   = modulate(norm_x, scale, shift)        # one AdaLN triple for the block
fused   = Linear(mod_x, W_qkv_mlp_proj)         # [B·S, 36864]   ← ONE big GEMM
q,k,v, gate,up = split(fused)                   # 4096,4096,4096, 12288,12288
q       = RMSNorm(q, norm_q) per head           # QK-norm (learned gains)
k       = RMSNorm(k, norm_k) per head
q,k     = RoPE(q,k)                             # 4-axis, combined positions
attn    = Attention(q, k, v)                    # [B·S, 4096]
mlp_act = silu(gate) · up                       # [B·S, 12288]   (SwiGLU gate)
concat  = [attn ‖ mlp_act]                      # [B·S, 16384]
delta   = Linear(concat, W_out)                 # [B·S, 4096]    ← ONE big GEMM
x       = gated_residual(x, delta, gate_mod)    # single residual
```

Read the structure: **one** normalize+modulate, **one** input GEMM that produces *everything* (the attention's Q/K/V *and* the MLP's gate/up), the attention and SwiGLU computed in parallel from those, **one** output GEMM that consumes *both* th e attention result and the MLP activation, and **one** gated residual. Compared to a double-stream block's two sub-layers with their two norms and two residuals, this is markedly leaner — appropriate for 24 repetitions.

9.3 The fused projections, decoded

The two weight matrices are large and their column/row splits encode the fusion. From the architecture (Chapter 04):

Input projection `to_qkv_mlp_proj` **:** `[36864, 4096]`**.** It maps a 4096-hidden token to 36864 outputs, sliced as:

$$36864 = 4096\_Q + 4096\_K + 4096\_V + 12288\_gate + 12288\_up = 3{\cdot}hidden + 2{\cdot}ffn = 9{\cdot}\,4096\ .$$

So a *single* GEMM produces the attention's Q, K, V **and** the MLP's gate and up. The `split` (in `kernels/qkv_mlp_split.*`) just views the contiguous output as those five tensors — no copy.

Output projection `to_out` **:** `[4096, 16384]`**.** It maps a 16384-wide concatenation back to hidden:

$$16384 = 4096_{attn\ out} + 12288_{mlp\ act} = hidden + ffn = 4 \cdot 4096\ .$$

So a *single* GEMM consumes both the attention output and the SwiGLU activation and produces the block's delta. The `concat` builds the 16384-wide input by placing the attention result and the MLP activation side by side (again a view/placement, not real math).

A single-stream block therefore owns just **2 linears** (plus 2 QK-norm gains and the shared modulation). 24 blocks × 4 tensors = the 96 single-block tensors the router accounts for (Chapter 14).

9.4 Why fuse? The bandwidth argument

This is the first place the Chapter 00 thesis — *everything bends toward bandwidth* — shows up in the architecture itself, not just the kernels.

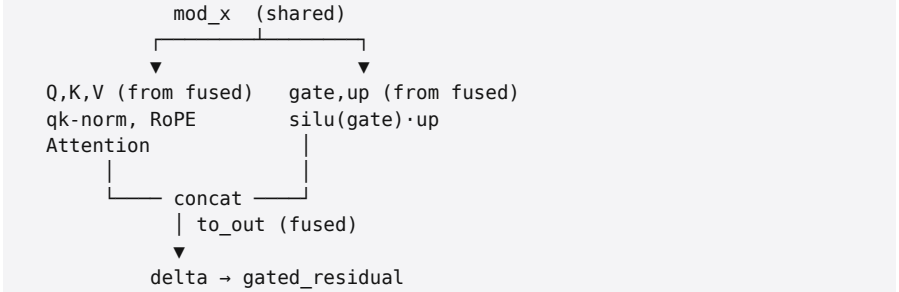
On a bandwidth-bound machine, a GEMM's cost is dominated by **reading its weight matrix from memory**. Two separate projections (say Q-proj and K-proj) read two weight tensors with two kernel launches, each paying launch overhead and each streaming its weights. Fusing them into one $[N_1+N_2, K]$ matrix reads the *same total bytes* but in **one** launch, with better memory-access contiguity and one set of fixed overheads. For the input projection, fusing five projections (Q, K, V, gate, up) into one GEMM means the **activation** `mod_x` is read **once** from memory instead of five times, and the output

is written as one contiguous block.

The activations are not free either: at S=4608, the 4096-wide `mod_x` is ~38 MB in BF16; reading it once vs five times is ~150 MB of saved traffic per block, per step. Across 24 blocks × 4 steps that adds up. The model was *trained* with these fused matrices (the weights are stored fused in the checkpoint), so we are not choosing to fuse — but the architecture chose to, for exactly this reason, and our job is to honor it: **don't split the fused weight before using it; feed the whole matrix to one GEMM.** (Chapter 13 builds the `Linear` that does this; the fused shapes are why `Linear` must handle large N efficiently.)

9.5 Parallel attention + MLP: the subtle part

In a *sequential* block, the MLP sees the residual *after* attention has updated it. In a *parallel* block, attention and MLP both read the **same** normalized input `mod_x`, and their results are merged only at the output projection. The two computations are independent until the final GEMM:



This loses the "MLP refines attention's output within the same block" interaction, but in a deep stack (24 blocks) the next block's attention sees the previous block's MLP output anyway, so the loss is minor — and the gain (one norm, one r

esidual, two big GEMMs, more parallelism) is real. It also makes the block a clean target for the kernel: the attention is exactly the D=128 kernel of Part IV, the SwiGLU is one `silu_mul`, and the rest is two GEMMs and elementwise ops.

9.6 Position and the combined sequence

RoPE here uses the **combined** position table (Chapter 06): the single-stream sequence is text-first then image, so positions are (0,0,0,ell) for the leading text tokens and (0,h,w,0) for the trailing image tokens, baked into one `[seq, head_dim/2]` table. The attention is over the whole combined sequence, so (as in the double-stream joint attention) image and text tokens continue to mix — but now through shared weights.

9.7 Correctness

Like the double-stream block, the single-stream block has a **zero-gate identity test**: with `gate_mod = 0` the block is the exact identity, and `tests/test_single_stream_block.cu` checks `max_err = 0` against the input, proving the fused split/concat, the parallel attention+MLP wiring, QK-norm, RoPE, and the residual all connect correctly. A real-weights smoke test loads `single_transformer_blocks.0` from the F2K file and checks finite, sanely-scaled output (`mean_abs ≈ 0.77`). This was, historically, the *first* complete MMDiT-block forward pass to run on real FLUX.2-klein weights on this custom CUDA path.

9.8 Where this lives in the code

Concept | Code
the block | single_stream_block.{h,cu} (SingleStreamBlock)
fused QKV+MLP split / attn‖mlp concat | kernels/qkv_mlp_split.{h,cu}
SwiGLU activation | kernels/silu_mul.* / swiglu.*
attention (D=128) | attention.cu (Part IV)
QK-norm, RoPE | kernels/rmsnorm.*, kernels/rope_4axis.*
tests | tests/test_single_stream_block.cu

9.9 Summary and what to carry forward

• Single-stream blocks (24 of them) process the **combined** sequence with **one** weight set and a **parallel** attention+MLP structure: one norm, one input GEMM, one output GEMM, one gated residual.

• The input GEMM is **fused** `[36864, 4096]` (= Q+K+V+gate+up); the output GEMM is **fused** `[4096, 16384]` (= attn‖mlp). `split`/`concat` are views, not copies.

• **Fusion is a bandwidth optimization**: read the shared activation once, stream one weight matrix per projection, one launch. Honor it — feed the whole fused matrix to one GEMM.

• Same correctness tools as the double-stream block: the **zero-gate identity** proves the wiring; a real-weights smoke test checks magnitudes.

Chapter 10 assembles the embedders, the 8+24 block stack, the stream transitions, and the final projection into the complete `FluxTransformer::forward`, closing Part II.

Exercises

1. **Decode the shapes.** From `[36864, 4096]` and `[4096, 16384]`, reconstruct the five input slices and two output slices, and verify 36864=9·4096 and 16384=4·4096.

2. **Parallel vs sequential.** State precisely what interaction a parallel block gives up versus a sequential one, and argue why 24 stacked blocks make the loss small.

3. **Bandwidth math.** Estimate the activation bytes saved per block by fusing the five input projections into one GEMM at S=4608, hidden 4096, BF16. Multiply by 24 blocks × 4 steps.

4. **One residual.** A single-stream block has one gated residual but does two things (attention + MLP). Where do both contributions enter that single residual? (Hint: the output

GEMM.)

Next: Chapter 10 — Assembling the transformer.

Chapter 10 — Assembling the Transformer

Goal of this chapter: connect every Part-II component into the complete velocity network — the embedders that bring image and text into hidden space, the modulation MLP that feeds all blocks, the 8 double + 24 single block stack with its stream transitions, and the final projection back to the latent. We trace the top-level `FluxTransformer::forward`, the construction cost, and the end-to-end validation. This closes the architecture; Part III makes it fast. Anchored to `flux_transformer.{h,cu}` and `embeddings.{h,cu}`.

Prerequisites: Chapters 04–09.

10.1 The three embedders

The transformer's hidden width is 4096, but its inputs and outputs are not. Three small `Linear`-based modules bridge the boundaries (`embeddings.{h,cu}`):

- **ImageEmbedder** — `x_embedder`, a `Linear 128 → 4096`. Projects each patchified image token (128 dims, Chapter 03) into hidden space. On real weights its output has `mean ≈ 0.13` — sane.
- **ContextEmbedder** — `context_embedder`, a `Linear 12288 → 4096`. Projects each Qwen3 conditioning token (Chapter 05) into hidden space.
- **FinalProjection** — the output bridge (§10.4): RMSNorm → AdaLN-modulate → `proj_out` `Linear 4096 → 128`. Maps the processed image tokens back to the latent channel dim so the result is a velocity in latent space.

These are the left and right edges of the Chapter 04 diagram. Everything between runs at hidden width 4096.

10.2 The top-level objects

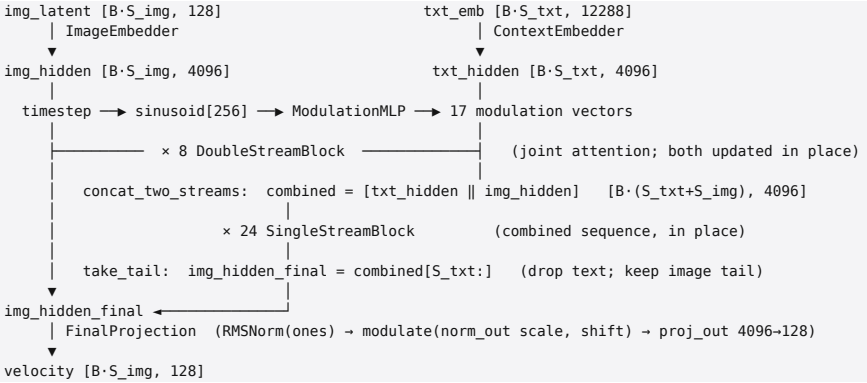
`FluxTransformer` owns, from `flux_transformer.h`:

```
 1 × ImageEmbedder          (128 → 4096)
 1 × ContextEmbedder        (12288 → 4096)
 1 × ModulationMLP          (timestep 256 → 17 modulation vectors; Chapter 07)
 8 × DoubleStreamBlock      (Chapter 08)
24 × SingleStreamBlock      (Chapter 09)
 1 × FinalProjection        (4096 → 128)
 2 × RoPE table sets         (double-stream lengths, single-stream length; Chapter 06)
```

The two RoPE table sets are built once per forward (they depend on S_img, S_{txt} and the patch grid `H_patches` × `W_patches`): an image table ((0,h,w,0)), a text table ((0,0,0,ell)), and the combined single-stream table (text-first). `Config` carries the shapes (`seq_img`, `seq_txt`, `H_patches`, `W_patches`, …) and the `precision` (NVFP4 or MXFP8, Chapter 12).

10.3 The forward pass, end to end

From the diagram that ships in `flux_transformer.h`:



Two pure data-movement kernels handle the stream transitions:

- `concat_two_streams_bf16` — at the double→single boundary, builds the combined sequence by placing text tokens first, then image tokens. (Order matters: it must matc h the combined RoPE table and the `take_tail` split.)

- `take_tail_bf16` — after the single-stream stack, extracts the last S_img rows (the image tokens). The text tokens have done their conditioning job and are discarded.

No new math lives in these — they are reshapes/copies — but their ordering is a contract (text-first) that the RoPE tables and the final split all depend on.

10.4 The final projection (and its scale-first modulation)

`FinalProjection` turns the processed image tokens into the velocity:

```
x → RMSNorm(x, ones)                        # no-affine
  → modulate(scale_out, shift_out)          # AdaLN, but scale-FIRST (see below)
  → proj_out: Linear 4096 → 128
```

This is `AdaLayerNormContinuous` in diffusers, and — as flagged in Chapter 07 — its modulation convention is **scale-first** (`scale, shift = chunk(emb, 2)`), unlike the *shift-first* `Flux2Modulation` of the blocks. There is no gate here (only 2 vectors, the `norm_out` pair from the ModulationMLP). Getting this one module's orde r *opposite* to the blocks' is required for correctness — a small but real trap. The output `[B·S_img, 128]` is the velocity v_θ that the sampler's Euler step consumes (Chapter 02). On real weights `FinalProjection` output has `mean ≈ 0.96` — sane.

10.5 What it costs to build and run

- **Construction (~5.3 s on GB10).** The constructor builds **124** `Linear` **instances** — 8 double × 12 + 24 single × 2 + 6 (embedders, modulation, proj_out) — and each repacks its quantized weights from the on-disk layout into the CUTLASS tensor-core layout (Chapter 13). With on-disk MXFP8 this is 5.3 s; the earlier construction-time quantization path took ~76 s (Chapter 12). All weights come from a `TensorRouter` over the F2K shards (Chapter 14).

• **Workspace.** All the intermediate token buffers — `img_hidden`, `txt_hidden`, the combined sequence, per-block Q/K/V/MLP scratch — total a few GiB at 1024px (e.g. ~2.5 GiB transformer + VAE separately), which unified memory absorbs (Chapter 11). The workspace is sized to the max over blocks and reused, since each block's intermediates die at its boundary.

• **Forward time.** All 32 layers + embedders + modulation + final projection is the per-step cost in the denoise loop (~1.6 s/step at 1024px FP8). Multiply by 4 steps for the ~6.5 s denoise of Chapter 00. The dominant sub-costs are the GEMMs (Part III) and the attention (Part IV).

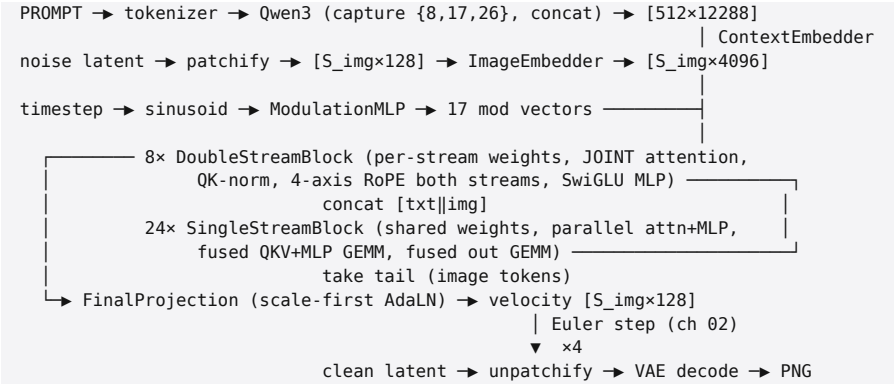
10.6 Proving the whole thing correct

Two levels of test, both essential:

• **Wiring (synthetic):** `tests/test_flux_transformer.cu` and the per-block zero-gate identity tests (Chapters 08–09) prove the residual highway and every connection. The denoise loop test (`tests/test_denoise_loop.cu`) runs the full 4-step schedule on real weights and checks the latent stays finite and well-scaled across steps.

• **Numerical (vs reference):** `tools/cmp_transformer.cu` compares a *single* transformer forward, on identical inputs (packed latent, prompt embeds, σ=1, 16×16 grid), against a diffusers reference produced by `tools/diffusers_block_dump.py`. Result: **cos-similarity 0.963 in NVFP4, 0.999 in MXFP8** — the NVFP4 error is purely directional and consistent with FP4 quantization noise (Chapter 12), not a structural bug. This test is what proved the architecture port *correct* (no RoPE/attention/wiring error) and localized the residual softness to quantization. (Chapter 25 is the full validation story.)

10.7 The complete picture (Part II in one diagram)



That is the entire FLUX.2-klein velocity network and its place in the pipeline. Everything in it has now been derived (Part I) and described (Part II). The remaining parts answer the engineering question: **how do you evaluate this function fast on a GB10?**

10.8 Summary and what to carry forward

• The transformer = **embedders** (image 128→4096, context 12288→4096) + a **ModulationMLP** + **8 double + 24 single** blocks + a **FinalProjection** (4096→128), with two **RoPE table sets** rebuilt per forward.

• **Stream transitions** are pure data movement: `concat_two_streams` (text-first) at the double→single boundary, `take_tail` to recover image tokens at the end — but their ordering is a contract.

• The **FinalProjection uses scale-first** AdaLN (opposite the blocks) and no gate.

• Built once (~5.3 s, 124 linears repacked), run per step; validated by zero-gate wiring tests and a **cos-0.963/0.999** comparison to diffusers.

Part II is complete: you can now explain every tensor from prompt to velocity. Part III turns to the machine — the Blackwell roofline, quantization, CUTLASS GEMM, the weight format, and the kernel library — i.e. how all of this is made to run in ~1.6 seconds per step.

Exercises

1. **Trace a token.** Follow one image token from `img_latent[128]` to `velocity[128]`, naming every module and the width at each stage.
2. **Transition contracts.** `concat_two_streams` places text first. List everything downstream that depends on this ordering (RoPE table, take_tail, …).
3. **Two AdaLN conventions.** Explain why the blocks are shift-first but `FinalProjection` is scale-first, and what tensor each reads.
4. **Build cost.** Why does the constructor build 124 linears, and why does on-disk MXFP8 cut construction from ~76 s to ~5.3 s? (Foreshadows Chapters 12–13.)

Next: Chapter 11 — The Blackwell target & the roofline, opening Part III.

Chapter 11 — The Blackwell Target & the Roofline

Goal of this chapter: turn "the machine is bandwidth-bound" from a slogan (Chapter 00) into a **computable** statement. We specify the GB10, build the roofline model, compute its ridge point at each precision, and then compute the arithmetic intensity of the three workloads that matter — the big transformer GEMMs, attention, and small-batch projections — to see *honestly* which are memory-bound and which sit near the compute ceiling. The conclusion is more nuanced than the slogan, and it is what justifies every choice in Part III–IV.

Prerequisites: Part II; basic GPU execution model (SMs, warps, HBM).

11.1 The GB10, precisely

```
 |
GPU | NVIDIA GB10 (Blackwell), compute capability sm_121
SMs | 48; ~99 KiB opt-in shared memory / block
Memory | 128 GB LPDDR5X, unified, ~273 GB/s, cache-coherent CPU+GPU (NVLink-C2C)
Tensor-core peak (theoretical) | ~1 PFLOP/s sparse FP4 (≈500 TFLOP/s dense FP4)
Tensor-core peak (measured here) | FP4 GEMM 252 TFLOP/s, FP8 GEMM 161 TFLOP/s (test_fp4_gemm/test_fp8_gemm, 2048³)
Host | 10× Cortex-X925 + 10× Cortex-A725 (aarch64)
```

Two facts shape everything. First, **unified memory**: the 128 GB is shared coherently between CPU and GPU, so model weights are mmap'd and read in place — no PCIe `cudaMemcpy`, no host/device staging on the hot path (Chapter 14). A 9B model in FP8 (~9 GB) or NVFP4 (~5 GB) simply *fits*, with tens of GB to spare for activations. Second, the **compute:bandwidth ratio is enormous** — even at the *measured* 252 TFLOP/s of dense FP4, the chip can do ~920 FLOPs for every

byte it can read from HBM. Whether a kernel is limited by compute or by memory depends on whether it does more or fewer than ~920 FLOPs per byte.

11.2 The roofline model

The roofline (Williams et al., 2009) is the right lens. Plot achievable performance P (FLOP/s) against **arithmetic intensity** I (FLOPs per byte of HBM traffic). Two ceilings bound any kernel:

$$P \le \min\left(P_{\max\_\text{compute ceiling}},\ B_{\max} \cdot I_{\text{memory ceiling}}\right),$$

where P_max is peak FLOP/s and B_max is peak bytes/s (273 GB/s). The two lines cross at the **ridge point**

$$I_{\text{ridge}} = \frac{P_{\max}}{B_{\max}}.$$

- If a kernel's intensity I < I_ridge, it is **memory-bound**: it cannot feed the tensor cores fast enough, and the only way to go faster is to move fewer bytes (quantize, fuse, keep data on-chip) or move them faster.
- If I > I_ridge, it is **compute-bound**: limited by FLOP throughput, and the lever is using a faster instruction (lower precision, sparsity, better tensor-core utilization).

Ridge points for the GB10 at the precisions we use (using **measured** peaks; the theoretical ones are ~2× higher and shift the ridge right):

precision	peak (measured)	I_ridge = peak / 273 GB/s
FP4 (NVFP4)	252 TFLOP/s	≈ 923 FLOP/byte
FP8 (MXFP8)	161 TFLOP/s	≈ 590 FLOP/byte
BF16 (CUDA cores / lower TC)	~tens of TFLOP/s	a few hundred FLOP/byte

Now we can classify the real workloads by computing their I.

11.3 Workload 1 — a big transformer GEMM

Take a hidden-to-hidden projection in the transformer at 1024px: Y = X W with M = 4608 tokens, K = N = 4096. FLOPs = 2MNK. Bytes = read X (MK· b_a) + read W (KN· b_w) + write Y (MN· b_a), with b_a=2 (BF16 activations) and b_w the weight byte width.

$$FLOPs = 2 \cdot 4608 \cdot 4096 \cdot 4096 \approx 1.55 \times 10^{11}.$$

With **FP4** weights (b_w = 0.5):

$$bytes = 4608 \cdot 4096 \cdot 2\_X = 37.7MB + 4096 \cdot 4096 \cdot 0.5\_W = 8.4MB + 4608 \cdot 4096 \cdot 2\_Y = 37.7MB \approx 8.4 \times 10^{7},$$

$$I = (1.55 \times 10^{11})/(8.4 \times 10^{7}) \approx 1845\backslash FLOP/byte.$$

That is **above** the FP4 ridge of ~923 — so at 1024px the big GEMMs are **compute-bound**, not memory-bound. The activations (not the 4-bit weights) dominate the byte count here because there are so many tokens. Lowering weight pre cision still helps (FP4 doubles the FLOP ceiling vs FP8, raising P_max), but more bytes won't be the lever for these.

> This refines Chapter 00's slogan. At high resolution the dense projections are compute-bound; the win from FP4 there is the *higher FLOP ceiling*, and the win from fusion (Chapter 09) is reading the shared activation once.

11.4 Workload 2 — attention

Attention's traffic is dominated by streaming K and V. For one head over S tokens at head-dim D, a flash kernel reads K and V once per query *tile* (Chapter 16), and the useful FLOPs are QK^→p + PV = 4 S^2 D per head. At S=4608, D=128, H=32, the benchmark (Chapter 17) measured the *final* kernel at **~279 GB/s and ~17.6 TFLOP/s** — i.e. it is pinned to the **memory ceiling** (279 ≥ 273 GB/s peak) and nowhere near any compute ceiling. Attention's intensity is low because each

byte of K/V is used for only a handful of multiply-adds before being discarded.

$$I_{attn} \approx (17.6\ \text{TFLOP/s})/(279\ \text{GB/s}) \approx 63\backslash\ \text{FLOP/byte} << 923 = I\_ridge.$$

Attention is decisively memory-bound. This is *why* Part IV is an exercise in moving fewer bytes and keeping K/V on-chip, and why the kernel is declared finished when it hits ~279 GB/s — there is no compute headroom to chase, only bytes to save, and the bytes are already saturating HBM.

11.5 Workload 3 — small-batch projections (the text encoder, low-res)

The Qwen3 encoder runs at S=512 tokens; many of its projections have small M. And at low image resolutions (256px, M=256 tokens) the transformer GEMMs shrink too. Redo §11.3 with M=256, FP4 weights:

$$\text{bytes} \approx 256{\cdot}4096{\cdot}2\_X = 2.1\text{MB} + 8.4\text{MB\_W} + 2.1\text{MB\_Y} \approx 1.26\times10^{7},\ I = (2{\cdot}256{\cdot}4096^{2})/(1.26\times10^{7}) \approx 682\backslash\ \text{FLOP/byte}.$$

Below the FP4 ridge — **memory-bound**, and now the **weight** bytes dominate (8.4 MB of W vs 4.2 MB of activations). Here 4-bit weights pay off directly: NVFP4 moves 4× fewer weight bytes than BF16, a near-linear speedup. The general rule: **sm all batch → weight-bandwidth-bound → quantization is a direct win; large batch → compute-bound → low-precision FLOP ceiling is the win.** Either way, lower p recision helps; the *mechanism* differs.

11.6 The honest synthesis

workload	regime at 1024px	dominant cost	the lever
big transformer GEMMs	compute-bound (just above ridge)	tensor-core FLOPs	low-precision FLOP ceiling (FP4>FP8), fusion
attention	memory-bound (at HBM peak)	K/V streaming	flash tiling, on-chip reuse (Part IV)

```
small-batch / low-res GEMMs | memory-bound | weight bytes | quantization (NVFP4)
VAE convolutions | memory-bound + kernel-selection | conv engine | cuDNN graph API, NHWC (Chapter 19)
weight loading at build | one-time, byte-bound | reading the model | mmap, on-disk quant (Chapter 14)
```

So Chapter 00's "everything bends toward bandwidth" is the right *design pressure* — the things that hurt (attention, weight loading, low-res/small-batch) are memory-bound, and the architecture's own choices (fused GEMMs, quantized weights) are bandwidth moves. But the rigorous statement is: **the workload is a mix, and the roofline tells you, per kernel, whether to cut bytes or raise the FLOP ceiling.** Both roads pass through low precision — which is why Chapter 12 is next.

11.7 Why low precision is the master key

Notice that **every** row of the table above is helped by quantization, for one of two reasons:

1. **Memory-bound kernels** move fewer bytes when weights/activations are smaller — a near-linear speedup right up to the point they become compute-bound.
2. **Compute-bound kernels** run on faster instructions: Blackwell's tensor cores do FP4 at ~2× the rate of FP8 and far above BF16, so dropping precision *raises the compute ceiling* P_max.

Quantization simultaneously slides a kernel down-and-right on the roofline (fewer bytes → higher I) **and** lifts the compute ceiling (faster instruction). The only cost is numerical accuracy — and Chapter 12 measures exactly how much, and why FP8 buys back most of what FP4 gives up.

11.8 Two more Blackwell-specific levers

- **Shared memory as the on-chip buffer.** The ~99

KiB/block opt-in smem is the staging area that makes memory-bound kernels fast: flash attention tiles K/V into it (Chapter 16–17), and the occupancy story of Chapter 17 is entirely about *fitting more CTAs per SM* by using less smem — a direct roofline move (more CTAs → more outstanding memory requests → closer to B_max).

• **Unified memory removes a whole category of traffic.** On a discrete GPU you'd budget PCIe transfers for the 5–9 GB model and the reference images; here there are none. The roofline's B_max is the *only* memory ceiling that matters, and there is no separate "get the data onto the device" phase.

11.9 Where this lives in the code / how to reproduce

• GEMM peaks: `tools/` builds `test_fp4_gemm` / `test_fp8_gemm` (Chapter 13), printing sustained TFLOP/s.

• Attention bandwidth/TFLOP/s: `tools/bench_attention.cu` (Chapter 17) prints GB/s and TFLOP/s per shape, plus the achieved occupancy.

• The device properties (SMs, smem, peak) are printed by `bench_attention` at startup. Appendix C lists the exact commands.

11.10 Summary and what to carry forward

• The **roofline** P≤min(P_max, B_max I) with ridge I_ridge = P_max/B_max classifies every kernel. For the GB10, I_ridge≈ 923 FLOP/byte at (measured) FP4.

• **Attention** (I≈63) and **small-batch/low-res GEMMs** (I≈680) are **memory-bound**; **big GEMMs at 1024px** (I≈1845) are **compute-bound**. The workload is a *mix*.

• **Low precision is the master key**: it cuts bytes (helps memory-bound) *and* raises the FLOP ceiling (helps

compute-bound). Cost = accuracy (Chapter 12).

• **Unified memory** removes transfer traffic entirely; **shared memory** and **occupancy** are the on-chip levers (Part IV).

With the roofline as our scoreboard, Chapter 12 introduces the numerical formats — BF16, FP8, NVFP4 — that let us trade precision for position on it, and measures the image-quality cost of each.

Exercises

1. **Ridge points.** Recompute I_ridge for FP4 and FP8 using the *theoretical* peaks (~500 / ~250 TFLOP/s). How far right does the ridge move, and which workloads cross from memory- to compute-bound?
2. **Resolution sweep.** Recompute the big-GEMM intensity I for M∈256, 1024, 4608 (256/512/1024px). At which resolution do the projections cross the FP4 ridge?
3. **Attention is stuck.** Given the measured 279 GB/s ≈ peak, explain why no amount of tensor-core cleverness speeds attention up, and what the *only* remaining levers are.
4. **Quantization on the roofline.** Draw (or describe) how switching a memory-bound GEMM from BF16 to NVFP4 weights moves its point on the roofline, and do the same for a compute-bound GEMM. Why does precision help both?

Next: Chapter 12 — Numerical formats & quantization.

Chapter 12 — Numerical Formats & Quantization

Goal of this chapter: understand the low-precision formats that let us trade accuracy for position on the roofline — **BF16, FP8 (E4M3/MXFP8), and NVFP4 (E2M1 + block scales)** — at the bit level, including *why* block scaling is necessary, the exact quantization procedure this project uses, and the **measured** image-quality cost of each. We end knowing why FP4 is fast-but-soft, why FP8 buys back almost all the quality, and which tensors are quantized at all. Anchored to `tools/f2k_convert.cu`, `tools/quant_sim.py`, and `linear.cu`.

Prerequisites: Chapter 11 (the roofline); IEEE floating point basics.

12.1 Floating point in one screen

A floating-point number is (-1)^s · (1.m) · 2^(e-bias): a sign s, a mantissa m (the fractional bits, giving **precision**), and an exponent e (giving **dynamic range**). For a fixed bit budget you trade range against precision by moving bits between e and m. The notation **ExMy** means x exponent bits, y mantissa bits (plus 1 sign bit).

format	bits	layout	mantissa bits	relative step (≈ 2^(-m))	role
FP32	32	E8M23	23	10^{-7}	references, scales
BF16	16	E8M7	7	≈0.8%	activations, "lossless" baseline
FP16	16	E5M10	10	0.1%	(not used here)
FP8 E4M3	8	E4M3	3	≈6–12%	MXFP8 weights/acts
FP8 E5M2	8	E5M2	2	≈25%	(more range, less precision)
FP4 E2M1	4	E2M1	1	≈50%	NVFP4 weights/acts

The key intuition: **mantissa bits set the relative quantization step.** BF16's 7 mantissa bits give ~0.8% steps — visually lossless, our baseline. Drop to FP8's 3 bits and steps grow to ~6–12%. Drop to FP4's *single* mantissa bit and

the step is ~50% — enormous. The whole game of low-precision inference is making that coarse grid survive by choosing the right *scale* for small groups of values.

> Note BF16 keeps FP32's 8 exponent bits (same range as FP32, just coarser mantissa) — which is why it is the friendly default for activations: it almost never overflows, it just rounds. FP16 trades range for precision and is more prone to overflow, so the ML world standardized on BF16.

12.2 The FP4 grid, concretely

E2M1 has 2 exponent bits and 1 mantissa bit. Enumerate it: the representable *magnitudes* are

$$0,\ 0.5,\ 1,\ 1.5,\ 2,\ 3,\ 4,\ 6$$

(times a scale), plus their negatives — **16 values total** in 4 bits. Look at the gaps: from 4 to 6 is a 50% jump; there is nothing between 4 and 6, nothing between 2 and 3. A weight of 5.0 must snap to 4 or 6 — a ~20% error on that element, and worst-case (a value at 5) the step is ~50% of the spacing. This coarseness is **fundamental to 1 mantissa bit**, not a bug. It is why pure FP4 images are soft (§12.6). E4M3, with 3 mantissa bits, has ~8× finer spacing within each binade (1.000, 1.125, 1.250, …), hence the ~6–12% steps.

12.3 Why a single scale is not enough: block scaling

A 4-bit code can only represent 16 distinct magnitudes. To cover a real weight tensor — whose values might span [-22528, 22528] (the Qwen3 outliers of Chapter 05!) — you multiply the grid by a **scale**: store $\hat{w} = \text{round}(w/S)$ in FP4 and recover $w \approx S\hat{w}$. But one scale for an entire matrix is a disaster: a single large outlier forces S huge, and then every *normal* value (~1) rounds to the nearest multiple of 6S — i.e. to **zero**.

The outlier survives; everything else is annihilated.

The fix is **block (a.k.a. microblock) scaling**: partition each row into small contiguous blocks and give *each block its own scale*. An outlier now only inflates the scale of its own little block; the other blocks keep fine scales and full precision. This is the central idea of NVFP4 and MXFP8.

```
row of weights:  [ w0 w1 ... w15 | w16 ... w31 | ... ]
                   └── block 0 ──┘ └ block 1 ─┘
each block: values in FP4 (or FP8) + ONE small scale (E4M3 or UE8M0)
plus one FP32 per-tensor scale on top (NVFP4)
```

The cost is the scales' own bytes — but a per-16 E4M3 scale adds only 8/16 = 0.5 bits per weight, so NVFP4 is effectively ~4.25 bits/weight, still ~3.5× smaller than BF16. The benefit is that block scaling makes 4-bit *usable* on real, outlier-laden weights.

12.4 The two formats this project uses

NVFP4 (Blackwell-native 4-bit)

- **Values:** E2M1 (the §12.2 grid), packed two-per-byte (`[N, K/2]`).
- **Block scale:** one **E4M3** scale per **16** consecutive elements (`[N, K/16]`).
- **Tensor scale:** one **FP32** scalar for the whole tensor.
- **Effective:** ~4.25 bits/weight; ~3.5× smaller than BF16.
- Dequant: w ≈ (FP32 tensor scale) · (E4M3 block scale) · (E2M1 value).

MXFP8 (the "microscaling" 8-bit)

- **Values:** E4M3, one byte each (`[N, K]`).
- **Block scale:** one **UE8M0** scale per **32** elements (`[N, K/32]`). UE8M0 is an *8-bit exponent-only* scale — a power of two, so dequant is a bit-shift, no mantissa.
- **Effective:** ~8.25 bits/weight; ~2× smaller than BF16,

~2× larger than NVFP4.

- Dequant: w ≈ 2^(UE8M0) · (E4M3 value).

Both are *block-scaled tensor-core formats* — Blackwell's MMA units consume the packed values and block scales directly (Chapter 13). The F2K container stores them in exactly this layout so the runtime mmaps and feeds them with no parsing (Chapter 14).

12.5 The quantization procedure

This project quantizes weights offline in `tools/f2k_convert.cu` (and `tools/quant_sim.py` mirrors it in PyTorch for validation). The procedure per block:

NVFP4 (block of 16): the E2M1 grid maxes at 6, so set the block scale so the block's largest-magnitude value lands at 6:

$$S\_block = quantize\_to\_E4M3\ ((max\_i\ |w\_i|)/(6)\ ),\hat{}w\_i = round\_to\_E2M1\ ((w\_i)/(S\_block)\ ).$$

MXFP8 (block of 32): E4M3 maxes at 448, and the scale is a pure power of two:

$$X = 2\hat{}(\ lceil\ log_2(max\_i|w\_i|\ /\ 448)\ rceil),\hat{}w\_i = round\_to\_E4M3\ ((w\_i)/(X)\ ).$$

The `[log2]` makes the scale a UE8M0 power of two (a shift). Both schemes are "absmax" calibrations — scale so the block's extreme value is representable, accept the rounding of the rest. (More elaborate calibrations — percentile clipping, GPTQ-style error feedback — are possible and were on the design table; the shipped converter uses absmax, which is already within FP4's noise floor for these weights.)

What gets quantized

Not every tensor is eligible. `f2k_convert` quantizes a tensor only if it is a rank-2 `.weight` with N%128=0 and K%64=0 —

the tensor-core tile constraints (Chapter 13). Everything else — biases, norm gains, embeddings, odd-shaped tensors — stays **BF16** passthrough. For Qwen3, `model.embed_tokens.weight` is explicitly kept BF16 (a `--keep-bf16` flag) and `lm_head` is dropped (unused as an encoder). So "the model is FP4" really means *its big matmul weights* are FP4; a long tail of small tensors stays BF16, which costs little and avoids quantizing things that don't tolerate it.

12.6 The measured quality cost

This is the heart of the chapter, and it is *measured*, not asserted (`tools/quant_sim.py` fake-quantizes the diffusers Flux2 transformer, weight-only, full pipeline, cat @ seed 42 / 4 steps):

format	weight rel-L2 error	image mean|Δ| vs BF16
NVFP4 (E2M1 + per-16 E4M3)	0.101	12.95%
FP8 (E4M3, per-row)	0.027	2.12%

FP8 is ~4× better per weight and ~6× better per image. End to end, the project's own CUDA pipeline (which also quantizes *activations*, not just weights) measures the transformer's single-forward cosine similarity against a diffusers BF16 reference (`tools/cmp_transformer.cu`, Chapter 25):

$$\textit{NVFP4: cos = 0.963,\ rel-L2 = 0.27} \longrightarrow \textit{MXFP8: cos = 0.99895,\ rel-L2 = 0.046 .}$$

Visually: the NVFP4 cat is soft/washed; the **MXFP8 cat is sharp**, essentially indistinguishable from BF16 (fur texture, facial features, lane markings all recovered). The FP4 "fuzziness" is **fundamental to 1 mantissa bit**, confirmed by the fake-quant study — it is not a kernel bug. (An earlier, long investigation chased the softness as a possible bug before this study settled it; Chapter 25.)

Resolution interacts with this. The FP4 softness is most

visible at low resolution; at 1024px the extra spatial detail masks much of it, so NVFP4 1024px images look quite good. But FP8 is the right default when quality matters, at the cost of ~2× the weight bytes (~5 GB → ~9 GB, fine in 128 GB) and ~2× the per-step GEMM time (still bandwidth-fine for the memory-bound parts; Chapter 11).

12.7 Tying back to the roofline

Recall Chapter 11: low precision helps memory-bound kernels (fewer bytes) *and* compute-bound kernels (faster instruction). Now we can be precise about the trade-curve:

- **NVFP4:** ~4.25 bits/weight, ~2× the FP8 FLOP ceiling. Best for *speed* — smallest model (~5 GB transformer), highest tensor-core rate (measured 252 TFLOP/s). Quality cost ~13% image error.
- **MXFP8:** ~8.25 bits/weight, ~161 TFLOP/s. ~2× slower and ~2× larger than NVFP4, but ~6× better image fidelity (cos 0.999). Best for *quality*.

`generate --precision {nvfp4,fp8}` lets you pick the point on this curve per run; the architecture and kernels are identical, only the `Linear`'s format differs (Chapter 13). The on-disk model exists in both NVFP4 (`transformer_f2k`) and MXFP8 (`transformer_mxfp8`) forms, produced by `f2k_convert --quant {nvfp4,mxfp8}`.

12.8 Where this lives in the code

Concept | Code
offline quantization (absmax, both formats) | tools/f2k_convert.cu (quantize_mxfp8, NVFP4 packing)
eligibility (N%128, K%64, .weight rank-2) | tools/f2k_convert.cu classifier
fake-quant quality study | tools/quant_sim.py
on-device activation quant | linear.cu (quant_for_A_kernel, quant_for_A_fp8_kernel)
precision selection | Linear::Precision, generate --precision
dtype codes (F8_E4M3=3, NVFP4=5, …) | f2k_format.h (Chapter 14)

12.9 Summary and what to carry forward

• **Mantissa bits = relative step.** BF16 ≈0.8% (baseline), FP8 E4M3 ≈6–12%, FP4 E2M1 ≈50% — the FP4 grid is the coarse 0,.5,1,1.5,2,3,4,6.

• **Block scaling** is what makes 4/8-bit usable: per-16 (NVFP4, E4M3 scale) or per-32 (MXFP8, UE8M0 scale) scales contain outliers so normal values keep precision. NVFP4 ≈4.25 b/w, MXFP8 ≈8.25 b/w.

• Quantization is **offline, absmax** (`f2k_convert`); only big rank-2 `.weight` matmul tensors are eligible, the rest stay BF16.

• **Measured cost:** NVFP4 ≈13% image error / cos 0.963; **MXFP8 ≈2% / cos 0.999**. FP4 softness is fundamental to 1 mantissa bit. Pick per run with `--precision`.

• Both formats help on the roofline (fewer bytes + higher FLOP ceiling); the curve is **speed (NVFP4) ↔ quality (MXFP8)**.

Chapter 13 shows how Blackwell's tensor cores actually *consume* these block-scaled formats — the CUTLASS GEMM and the `Linear` class built on it.

Exercises

1. **Enumerate E2M1.** Derive the 8 magnitudes 0,.5,1,1.5,2,3,4,6 from 2 exponent + 1 mantissa bits, and compute the worst-case relative rounding error.

2. **Outlier annihilation.** With one per-tensor scale and a single value at 22528 among values near 1, show that the near-1 values round to 0 in FP4. Then show a per-16 block scale rescues them.

3. **Byte budget.** Compute bits/weight for NVFP4 (E2M1 + per-16 E4M3 + per-tensor FP32) and MXFP8 (E4M3 + per-32 UE8M0). Confirm ~4.25 and ~8.25.

4. **Pick a format.** For (a) a quick 256px draft and (b) a final 1024px portrait, which precision and why? Tie your answer to §12.6's measured numbers and Chapter 11's roofline.

Next: Chapter 13 — Tensor-core GEMM with CUTLASS.

Chapter 13 — Tensor-Core GEMM with CUTLASS

Goal of this chapter: implement the matrix multiply that every `Linear` in the model reduces to, using Blackwell's **block-scaled tensor cores** via CUTLASS. We cover what tensor-core MMA is, how Blackwell consumes the NVFP4/MXFP8 formats of Chapter 12 *natively*, the CUTLASS type configuration we mirror from example 79a, the scale-factor layout that caused a subtle accuracy bug (and the CuTe fix), and the `Linear` class that wraps it — construction-time weight repacking, on-device activation quantization, and the alignment constraints. Anchored to `fp4_gemm.{h,cu}`, `fp8_gemm.{h,cu}`, `linear.{h,cu}`.

Prerequisites: Chapters 11–12; the GEMM Y=XW^→p; CUDA tiling at a high level (the PTX detail is Chapter 18).

13.1 Everything is a GEMM

Count the matmuls in one forward pass: each double-stream block has 12 linears, each single-stream block 2 (large, fused), plus embedders, modulation, final projection — 124 `Linear`s (Chapter 10), each a GEMM Y = X W^→p. The text encoder adds dozens more. **The GEMM is the workhorse**, and its throughput (Chapter 11: 252 TFLOP/s FP4, 161 FP8) sets the per-step cost. This chapter is how that throughput is reached.

A `Linear` computes, for M token rows, K input features, N output features:

$$Y_{[M,N]} = X_{[M,K]}\, W^{\rightarrow p}_{[K,N]} \; (+ \text{bias}),$$

with X the activations (BF16) and W the quantized weights (NVFP4 or MXFP8).

13.2 Tensor cores and MMA, briefly

A **tensor core** executes a small matrix multiply-accumulate in one instruction — `D = A·B + C` for fixed tile shapes (e.g. 16×8×16), with the inputs in a low-precision format and the accumulation in FP32. A GEMM kernel tiles the big matrices into these MMA-sized fragments, streams them through shared memory and registers, and accumulates. The arithmetic density is enormous: one instruction, hundreds of MACs. (The exact register/fragment layouts and the raw `mma.sync` PTX are Chapter 18, where we hand-write them for attention. Here CUTLASS generates them for us.)

Blackwell's 5th-gen tensor cores add **block-scaled** MMA: the instruction takes not just the packed low-precision operands but also their **per-block scale factors**, and applies the scales *inside* the MMA. This is the hardware counterpart of Chapter 12's block scaling — NVFP4 (E2M1 values + per-16 E4M3 scales) and MXFP8 (E4M3 values + per-32 UE8M0 scales) are *native operand formats*, not something we dequantize first. That is the whole reason these formats are fast: no separate dequant pass, the tensor core reads packed values + scales and produces FP32.

13.3 Why CUTLASS

Writing a peak-throughput Blackwell GEMM by hand means orchestrating TMA loads, multi-stage shared-memory pipelines, the block-scaled MMA atoms, and the epilogue — months of work to match the vendor library. **CUTLASS** (NVIDIA's open template library) already encodes these for every architecture, including the block-scaled Blackwell path. This project vendors CUTLASS and adapts its example `79a_blackwell_geforce_nvfp4_bf16_gemm` — the canonical GeForce/consumer Blackwell NVFP4→BF16 GEMM. (The

decision to use the templated CUTLASS path first, and keep h and-rolling `tcgen05` MMA as a later option, was deliberate; the hand-rolled path is exactly what Chapter 18 does for *attentio n*, where the access pattern is special enough to beat the library.)

The header `fp4_gemm.h` keeps the interface **CUTLASS-free** — all the template machinery lives in `fp4_gemm.cu` — so callers (blocks, the UI, tests) don't drag CUTLASS's heavy includes into every translation unit. `FP4Gemm` is a persistent handle: configure once with the shape, call `run()` each step.

13.4 The type configuration

The CUTLASS kernel is selected by a stack of template types. The ones that matter, mirrored from example 79a:

```
ArchTag        = cutlass::arch::Sm120;                      // covers sm_120 AND sm_121 (GB10)
OperatorClass  = cutlass::arch::OpClassBlockScaledTensorOp;  // block-scaled MMA
ElementA/B     = cutlass::nv_float4_t<cutlass::float_e2m1_t>;// NVFP4: E2M1 values + block scale
ElementC/D     = cutlass::bfloat16_t;                       // BF16 in/out
// cluster = 1×1×1 (GeForce), accumulation FP32
```

The FP8 variant (`fp8_gemm.cu`) swaps the operand type to `cutla ss::mx_float8_t<cutlass::float_e4m3_t>` (MXFP8: E4M3 values + UE8M0 per-32 scales), mirroring CUTLASS example 79c. A few notes:

- `Sm120` **is correct for sm_121.** CUTLASS's `Sm120` arch tag guards on `CUTLASS_ARCH_MMA_SM120_SUPPORTED || ..._SM121_SUPPORTED`, so the GB10 (sm_121) uses this path. (And the build must actually target `sm_121a` — getting the CMake CUDA arch wrong silently compiles for Turing and the MMA fails at runtime; Appendix C / a hard-won CMake lesson.)
- **Accumulation is FP32** even with 4-bit inputs — the products are tiny-precision but the sum is full-precision, which is what keeps GEMM error at the Chapter-12 noise floor rather than compounding.

Measured: **251.8 TFLOP/s** at 2048^3 NVFP4, **161 TFLOP/s**

FP8 — ~50% / ~64% of the relevant dense peaks with zero hand-tuning, bit-identical to CUTLASS's host reference. That validated the whole toolchain (CMake arch wiring, the block-scaled MMA path) before any model code existed.

13.5 The scale-factor layout — and a subtle bug

Here is the most instructive part of this chapter. The block scales are not stored in the obvious flat layout; CUTLASS uses an internal **tiled/swizzled** layout for the scale-factor tensors (`SFA` for the activation, `SFB` for the weight) so the MMA can fetch a tile's scales efficiently (TMA-aligned). When you write the quantized weights, you must write the scales into *that* layout, not a naïve one.

The project learned this the hard way. The first weight-quant code computed FP4 byte indices for the column-major B (weight) with a **hand-rolled formula** `byte_idx = k·(N/2) + (n/2)`, assuming a flat column-by-column packing. It produced *correct-looking* results on structured test inputs (all-ones, row-varying, K-axis-constant-within-block) but **uncorrelated output on random per-element weights**. The reason: the naïve formula matched CUTLASS's real layout only for column 0; for k>0 the stride was wrong, so CUTLASS re-read column-0 bytes for every K position. With weights constant along K (the structured tests) that looks fine — column 0 is representative — but with real K-varying weights the variation was erased and the output collapsed toward the all-ones answer.

The fix: never hand-roll sub-byte indexing under a CUTLASS layout. Route the write through CuTe, which *knows* the real layout:

```
auto stride_B = cutlass::make_cute_packed_stride(
```

```
    typename Gemm::GemmKernel::StrideB{}, {N, K, 1});
auto layout_B = cute::make_layout(cute::make_shape(N, K, 1), stride_B);
auto w_tensor = cute::make_tensor(
    cute::recast_ptr<cutlass::float_e2m1_t>(W_packed.data()), layout_B);
// assign through the tensor — CuTe handles packing/swizzle/tiling:
w_tensor(n, k, 0) = cutlass::float_e2m1_t(value / scale);
```

After the fix, random-data `Linear` hit **cos 0.989 / rel-L2 0.15** vs an FP32 reference — squarely within FP4 noise (Chapter 12). The lesson generalizes to *any* sub-byte format under a tiled layout: use `cute::recast_ptr` + `cute::make_tensor` and assign through the tensor; the same trap waits in the activation-quant kernel if it ever hand-indexes. (Appendix B has the full debugging trace; this is also why the structured-input tests *passed* while the model was wrong — a cautionary tale about test coverage.)

13.6 The Linear class

`Linear` (`linear.{h,cu}`) wraps `FP4Gemm`/`FP8Gemm` into the layer the model uses:

```
 y = activation(BF16) @ W^T(quantized) + bias(BF16, optional)
```

Two phases:

Construction (once). The weight is brought into the GEMM's expected operand layout. There are two source paths, unified by the `PreQuantNVFP4` descriptor (which, despite the name, carries either):

- **W_preq (NVFP4 from disk):** the F2K file already holds packed E2M1 `[N,K/2]` + E4M3 scales `[N,K/16]` (Chapter 14). Construction copies these into the CUTLASS SFB layout via the CuTe path of §13.5 — no quantization, just a relayout.
- **W_bf16 (MXFP8):** the F2K holds BF16 weights; construction **quantizes** them to MXFP8 (E4M3 + UE8M0) at build time, staging into the FP8 SFB layout. (There is *also* an on-disk MXFP8 path — `f2k_convert --quant mxfp8` — so construction can instead copy pre-quantized FP8, cutting

build time from ~76 s to ~5.3 s; Chapter 14.)

`make_weight_view` routes on the on-disk dtype: an `F8_E4M3` tensor → pre-quant copy; a `BF16` tensor → quantize-at-construction.

Forward (every step). The BF16 activation is **dynamically quantized on the device** into a matching FP4+SFA (or FP8) pair (`quant_for_A_kernel` / `quant_for_A_fp8_kernel` — a warp computes each row's block scales and packs the values), then `FP4Gemm::run()` executes, then bias is added. Activation quant is *per-forward* and on-device because activations change every step (unlike weights, quantized once). Quantizing activations too — not just weights — is why the end-to-end er ror (Chapter 12, cos 0.963 NVFP4) is a touch worse than weight-only fake-quant (12.95% image): both operands are low-precision.

13.7 The alignment constraints (and where they come from)

The Blackwell block-scaled GEMM tiles impose:

M\ (batch rows) % 128 = 0, N\ (out features) % 128 = 0, K\ (in features) % 64 = 0.

These are not arbitrary — they are the MMA/TMA tile sizes. Consequences felt throughout the model:

- **N%128, K%64** is exactly the eligibility test in `f2k_convert` (Chapter 12): a weight that doesn't tile stays BF16.
- **M%128** means the *token count* fed to a `Linear` must be a multiple of 128. This is why `seq_img` must be a multiple of 128 (Chapter 03's resolution validation), why the `ModulationMLP` pads to an internal M=128 and reads row 0 (Chapter 07), and why prompts are padded to 512 (Chapter 05). The constraint propagates from the MMA tile all the way up to the CLI's

allowed resolutions.

(The header notes these may later be relaxed with a padding workspace; for now the whole pipeline is arranged so they hold naturally.)

13.8 Where this lives in the code

Concept | Code
NVFP4 GEMM handle | fp4_gemm.{h,cu} (FP4Gemm), from CUTLASS ex 79a
MXFP8 GEMM handle | fp8_gemm.{h,cu} (FP8Gemm), from CUTLASS ex 79c
the layer | linear.{h,cu} (Linear, PreQuantNVFP4, Precision)
weight relayout (CuTe) | linear.cu construction
on-device activation quant | linear.cu quant_for_A_kernel / ..._fp8_kernel
smoke/perf/correctness | tests/test_fp4_gemm.cu, test_fp8_gemm.cu, test_linear*.cu

13.9 Summary and what to carry forward

- Every `Linear` is a GEMM Y=XW^→p; GEMM throughput sets the per-step cost.
- Blackwell's tensor cores consume **block-scaled NVFP4/MXFP8 natively** (scales applied inside the MMA, FP32 accumulate) — no separate dequant.
- We use **CUTLASS** (`Sm120` arch tag covers sm_121, `OpClass BlockScaledTensorOp`) rather than hand-rolling; measured 252/161 TFLOP/s, bit-exact to the reference.
- The scale-factor tensors use a **tiled/swizzled layout**; write them through `cute::recast_ptr` **+** `make_tensor`, never a hand-rolled byte index — the hand-rolled version passed structured tests but corrupted random data.
- `Linear` repacks the weight once (copy pre-quant NVFP4, or quantize BF16→MXFP8) and **dynamically quantizes activations on-device** each forward.
- The tile constraints **M%128, N%128, K%64** propagate up to resolution, sequence, and padding choices across the whole pipeline.

Chapter 14 covers where these quantized weights live — the

F2K on-disk format, the mmap loader, and the router that turns 233 tensor names into structured roles.

Exercises

1. **Why FP32 accumulate.** Argue why accumulating 4-bit products in FP32 (not FP4) keeps GEMM error at the per-element quantization floor rather than growing with K.
2. **The structured-test trap.** Explain precisely why the hand-rolled byte index passed all-ones and K-constant tests but failed on random weights. What test *would* have caught it, and why is it now `tests/test_linear.cu`'s random case?
3. **Constraint propagation.** Trace M%128=0 from the MMA tile to: the allowed resolutions, the modulation MLP's internal padding, and the 512 prompt length.
4. **Two weight paths.** Compare W_preq (NVFP4 copy) vs W_bf16 (MXFP8 quant at ctor) vs on-disk MXFP8 (FP8 copy) in build time and what each reads from disk. Why does on-disk MXFP8 cut build from 76 s to 5.3 s?

Next: Chapter 14 — Weights on disk: the F2K format, loader & router.

Chapter 14 — Weights on Disk: the F2K Format, Loader & Router

Goal of this chapter: follow the weights from a Black Forest Labs `.safetensors` download to device-ready tensors. We design **F2K**, a custom mmap-and-go container built for block-scaled quantization and unified memory; walk the multi-shard **loader**; and explain the **tensor router** that turns 233 opaque tensor names into the structured per-block roles the model consumes. Anchored to `src/common/f2k_format.*`, `f2k_model_loader.*`, `tensor_router.*`, and `tools/f2k_convert.cu`.

Prerequisites: Chapters 12–13 (the formats and why scale layout matters); Chapter 11 (unified memory).

14.1 Why not safetensors or GGUF?

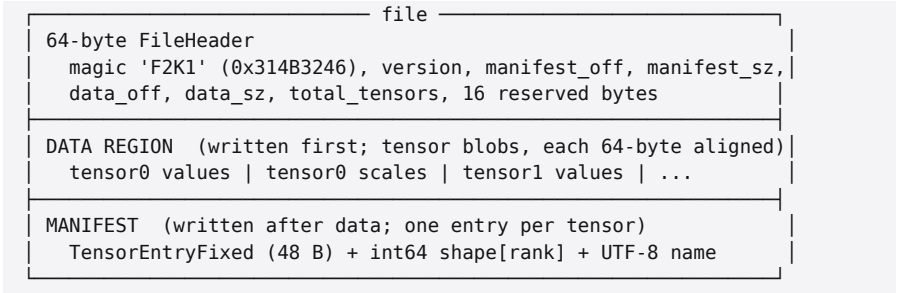
The weights ship as `.safetensors` (HF's format). Two existing formats were candidates for the runtime container, and both were rejected:

- **safetensors** has no first-class **quantization metadata**. There is no concept of per-microblock scales; stuffing the NVFP4 packed values + E4M3 block scales + per-tensor FP32 scale into its JSON header would lose the alignment guarantees the tensor cores need (Chapter 13's tiled scale layout).
- **GGUF** (llama.cpp) was the original plan, but it is tied to llama.cpp's k-quant types (Q4_K, Q5_K). On Blackwell we target **NVFP4/MXFP8 + tensor cores**, not k-quants, so GGUF's quant zoo is the wrong vocabulary, and its design assumes VRAM-constrained consumer GPUs — irrelevant with 128 GB unified (Chapter 11).

So the project defines **F2K**, optimized for exactly this stack:

store tensors in **GEMM-ready storage order**, 64-byte aligned, with native block-scale metadata, so the runtime can **mmap and go** — no parsing, no copy, no dequant on the load path.

14.2 The F2K v1 container



Design decisions, each with a reason:

- **Data first, manifest second, header patched last.** The converter can *stream* tensors to disk without knowing the total count up front; it appends the manifest and back-patches the header offsets on commit. Good for a one-pass converter.
- **64-byte alignment** (`F2K_DATA_ALIGN`) on every blob — matches cache lines and the tensor-core load alignment, so a tensor can be fed to a kernel directly from its mmap'd address.
- **Quantized tensors carry a separate scale blob.** A quantized tensor stores its packed values *and* a separate, 64-byte-aligned per-microblock scale blob with its own offset/size; the per-tensor FP32 scale lives in the fixed entry; the microblock size (16 for NVFP4, 32 for MXFP8) is recorded. This is the on-disk shape of Chapter 12's formats.
- **dtype codes are frozen.** `0 F16, 1 BF16, 2 F32, 3 F8_E4M3, 4 F8_E5M2, 5 NVFP4, 6 U8, 7 I8, 8 I32, 255 Unknown`. *Never renumber a shipped code*; append new dtypes at the

next free integer, and bump `F2K_VERSION` only when changing the semantics of an existing field (gate new reader paths on the version). The router in `Linear` (Chapter 13) switches on these codes to pick the weight path (F8_E4M3 → MXFP8 pre-quant copy; BF16 → quantize-at-ctor; NVFP4 → FP4 pre-quant).

A round-trip test (`tests/test_round_trip.cpp`) writes and re-reads tensors to prove the container is self-consistent.

14.3 Conversion: f2k_convert

`tools/f2k_convert.cu` is the offline pipeline (Chapter 12 covered the quant math; here the I/O):

1. Read each tensor from the source `.safetensors` (via `src/common/safetensors.*`).
2. Classify: rank-2 `.weight` with N%128=0, K%64=0 → quantize (NVFP4 or MXFP8 per `--quant`); everything else → BF16 passthrough. A `--keep-bf16 <substr>` flag forces specific tensors (e.g. Qwen3's `embed_tokens`) to stay BF16.
3. Write packed values + scale blob (NVFP4/MXFP8) or raw BF16 into the data region; record the manifest entry.

Real numbers: transformer shard 1 converts 9.13 GiB → **2.57 GiB (3.55×)** in ~77 s on the ARM cores (NVFP4), 113 tensors quantized + 42 passthrough, worst per-tensor `max_err 0.106`. The MXFP8 transformer is ~8.7 GiB (`--quant mxfp8`), and a BF16 passthrough F2K (~18 GiB) also exists for construction-time quantization. The model is split across **2 transformer shards**, **4 Qwen3 shards**, and **1 VAE file**.

> **Why an MXFP8 on-disk form matters.** Quantizing BF16→MXFP8 *at model build* took ~76 s every launch. Storing the result on disk (`--quant mxfp8`) lets the runtime *copy* pre-quantized FP8 instead of computing it, cutting transformer build to **5.3 s** (14×) with bit-identical output.

On-disk quant is the norm; the BF16 file is only needed if you want ctor-time quant.

14.4 The loader: mmap and go

`f2k_model_loader.*` is a thin multi-shard layer over the raw reader:

- **mmap, not read.** Each shard is memory-mapped. Because memory is **unified and coherent** (Chapter 11), the GPU can read weights at their mmap'd host addresses; there is no `cudaMemcpy` of the model, no device copy, no staging. The OS pages data in on demand.
- **Unified name→view.** It presents one `name → TensorView` lookup across all shards, with duplicate detection and per-tensor shard location. A `TensorView` is just `(ptr, dtype, shape, scale ptr, …)` into the mmap.
- **Fast.** The real model (233 tensors, 4.76 GiB on disk) "loads" in *milliseconds* — mmap is lazy; the cost is just mapping, not reading. The actual byte traffic happens when `Linear` construction touches each weight to repack it (Chapter 13).

This is the unified-memory payoff made concrete: "loading a 9B model" is an `mmap` and a name table, not a multi-second device transfer.

14.5 The router: names → structure

The loader gives `name → bytes`. But the model needs *structure*: "give me block 3's image Q-projection weight," not a string lookup. `tensor_router.*` parses the HF naming convention into roles:

```
transformer_blocks.3.attn.to_q.weight        → DoubleBlock[3].img_q
transformer_blocks.3.attn.add_q_proj.weight  → DoubleBlock[3].txt_q
single_transformer_blocks.7.attn.to_qkv_mlp_proj.weight → SingleBlock[7].qkv_mlp
context_embedder.weight                       → Globals.context_embedder
double_stream_modulation_img.linear.weight    → Globals.mod_double_img   (SHARED)
```

```
...
```

It produces a `GlobalTensors` struct + a `vector<DoubleBlockTensors>` (8) + `vector<SingleBlockTensors>` (24). On the real file **every required slot is filled and zero tensors are unrouted** — a strong structural check that the naming convention is fully understood. The decomposition confirms Chapter 04's accounting:

$$8\times \text{16\_double} + 24\times \text{4}_s\text{ingle} + \text{9\_globals} = 233 \text{ tensors.}$$

The router is also where the **shared-modulation** discovery lives (Chapter 04/07): there are no per-block modulation linears in the file — `double_stream_modulation_{img,txt}` and `single_stream_modulation` are single tensors shared across all blocks of their type. The router maps them into the globals, and every block reads the same modulation pointers. `tests/test_tensor_router.cu` asserts the full routing on the real model.

14.6 The path from disk to a running model

Putting Chapters 12–14 together, the lifecycle of one weight:

```
safetensors (BF16, on HF)
   │ f2k_convert (offline): classify → quantize (NVFP4/MXFP8) → pack + scales
   ▼
F2K shard on disk (packed values + block scales, 64-B aligned, dtype-coded)
   │ F2KModelLoader: mmap (ms), name→TensorView (unified memory, no copy)
   ▼
TensorRouter: name → GlobalTensors / DoubleBlock[i] / SingleBlock[i]
   │ FluxTransformer ctor: Linear repacks each view into CUTLASS SFB layout (ch 13)
   ▼
device-resident, tensor-core-ready Linear  (5.3 s for all 124, on-disk MXFP8)
```

No step parses JSON on the hot path, none copies the model across a bus, and the quant work is either offline (`f2k_convert`) or a one-time copy at construction. That is the F2K design goal — *mmap and go* — realized.

14.7 Where this lives in the code

Concept | Code
container format | src/common/f2k_format.{h,cpp} (FileHeader, TensorEntryFixed, F2KReader)
conversion | tools/f2k_convert.cu
multi-shard mmap loader | src/common/f2k_model_loader.{h,cpp}
name → role routing | src/common/tensor_router.{h,cpp}
safetensors reader (convert-time) | src/common/safetensors.{h,cpp}
tests | tests/test_round_trip.cpp, test_f2k_model_loader.cpp, test_tensor_router.cpp

14.8 Summary and what to carry forward

• **F2K** is a custom container because safetensors lacks block-scale metadata and GGUF assumes k-quants/VRAM limits. It stores **GEMM-ready, 64-B-aligned, block-scale-aware** tensors for **mmap-and-go**.

• Layout: 64-B header → data region (values + separate scale blobs) → manifest. **dtype codes are frozen**; bump `F2K_VERSION` for semantic changes.

• `f2k_convert` quantizes offline (3.55× shrink, NVFP4); an **on-disk MXFP8** form cuts model build 76 s → 5.3 s.

• The **loader mmaps** (ms; unified memory → no device copy); the **router** turns 233 names into `Globals + 8 DoubleBlock + 24 SingleBlock`, with **shared modulation** and zero unrouted tensors.

Chapter 15 finishes Part III with the small kernel library — the norms, activations, and data-movement ops that glue the GEMMs and attention together.

Exercises

1. **Why data-first.** Explain how writing the data region before the manifest lets the converter stream tensors without knowing the count, and what the header back-patch does.

2. **Frozen codes.** Why must dtype code 5 (NVFP4) never be reused for a different format, even in v2? What is the right way to add a new dtype?

3. **mmap economics.** Explain why "load the 9B model" is milliseconds here but the *first forward* touches real bandwidth. Where does the on-disk→device byte traffic actually happen?

4. **Routing as a check.** Why is "zero unrouted tensors + every slot filled" strong evidence the architecture is fully understood? What would a missing slot mean?

Next: Chapter 15 — The core kernel library.

Chapter 15 — The Core Kernel Library

Goal of this chapter: survey the small, reusable CUDA kernels that glue the GEMMs (Chapter 13) and attention (Part IV) together — the normalizations, activations, and data-movement ops. None is individually hard; together they are the connective tissue of every block, and they share a handful of patterns worth internalizing. We give the math for the norms and activations and the standard implementation shape. Anchored to `src/backend/cuda/kernels/*`.

Prerequisites: Chapters 04–10 (where each kernel is used); CUDA warp/block reductions.

15.1 The cast of kernels

From `src/backend/cuda/kernels/`:

kernel | op | used by
rmsnorm | RMS normalization | every block's norms + QK-norm (ch 08–09), Qwen3 (ch 21)
layernorm | mean/var layer norm | the centered LayerNorm in the model
groupnorm | grouped channel norm | VAE resnet/conv blocks (ch 19–20)
silu_mul / swiglu | SiLU(g)· u gated activation | MLP sub-layers (ch 08–09)
modulation | (1+scale)·x+shift, y+=gate·δ | AdaLN (ch 07)
rope / rope_4axis | rotary position | Q/K (ch 06)
patchify | latent grid ↔ token sequence | I/O of the transformer (ch 03)
embed_lookup | vocab gather | Qwen3 token embeddings (ch 21)
qkv_mlp_split / seq_ops | split/concat/take-tail | fused blocks, stream transitions (ch 09–10)
transpose_chw | NCHW↔NHWC | VAE conv layout (ch 19)
upsample | 2× nearest upsample | VAE decoder (ch 20)

These are all **memory-bound** (Chapter 11): they read and write activations with little arithmetic per byte. Individually they are negligible next to the GEMMs and attention — but there are many, in every block, so they must be clean and fused where cheap. The design philosophy is *write each once,*

reuse everywhere: the same `rmsnorm_bf16` serves the block-entry norm, the per-head QK-norm, and the Qwen3 enc oder, just called with different `batch_rows` and gains.

15.2 The shared implementation patterns

Almost every kernel here follows one of two shapes:

(a) One CTA (or warp) per row, with a reduction. Norms reduce over the feature dimension. The standard pattern: each thread accumulates a partial sum over its strided slice of the row, a **warp reduce** (`__shfl_xor`) combines lanes, a **bl ock reduce** (via a 32-slot shared scratch) combines warps, and the result is broadcast back. This is exactly the `warp_reduc e_sum/max` + `block_reduce` helpers that also appear in the attention fallback kernel (Part IV).

(b) One thread per element (elementwise). Activations, modulation, RoPE, patchify: a flat grid where each thread handles one (or a few) output elements.

Two conventions hold throughout:

- **BF16 storage, FP32 compute.** Inputs/outputs are BF16 (to move fewer bytes — Chapter 11), but reductions and transcendentals accumulate in FP32 to avoid precision loss. Norms especially need FP32 sums; a BF16 sum over 4096 elements would lose low bits.
- **Broadcast parameters are small BF16 vectors.** Scale/shift/gate and norm gains are `[hidden]` vectors broadcast across all rows (Chapter 07); RoPE tables are FP32 `[seq, D/2]` (Chapter 06).

15.3 The normalizations

Three normalizations appear, differing in *what* they normalize over.

RMSNorm — the dominant one in the transformer. It rescales a vector by its root-mean-square, with a learned per-channel gain γ (no mean subtraction, no bias):

$$RMSNorm(x)_i = (x_i)/(\sqrt{(tfrac1dΣ_j\ x_j^2 + ε)})\ γ_i,\ ε = 10^{-6}.$$

It is cheaper than LayerNorm (one reduction, no mean) and empirically as good for transformers, which is why modern models (LLaMA, Qwen3, FLUX) use it. In FLUX the block-entry RMSNorm uses an **all-ones** gain (statistics-only — the learned affine is replaced by AdaLN modulate, Chapter 07), while the **QK-norm** uses *learned* per-head gains (Chapter 08). Same kernel, different `gain` argument and `batch_rows = S·n_heads` for the per-head case.

LayerNorm — subtract the mean, divide by the std, affine:

$$LN(x)_i = (x_i - μ)/(\sqrt{(σ^2 + ε)})\ γ_i + β_i,\ μ=tfrac1dΣ_j\ x_j,\ \backslash$$
$$σ^2=tfrac1dΣ_j\ (x_j-μ)^2\ .$$

Used where the model specifies a centered norm. Two reductions (mean, then variance) instead of RMSNorm's one.

GroupNorm — used in the **VAE** (Chapter 19–20), where activations are [C, H, W] convolutional feature maps, not token vectors. It splits the C channels into G groups and normalizes over each group's channels *and* spatial extent:

$$GN(x)_{c,h,w} = (x_{c,h,w} - μ_g)/(\sqrt{(σ_g^2 + ε)})\ γ_c + β_c,$$

with μ_g,σ_g computed over all (c∈ g, h, w). The VAE uses GroupNorm-32. Its reduction is over a much larger set (channels × spatial), so the kernel reduces over a 3D extent per group.

15.4 The activations

SiLU (a.k.a. swish): SiLU(x) = x σ(x) = dfracx1+e^(-x). A smo oth, non-monotonic activation. Appears alone (`silu_inplace_b`

`f16`, in the modulation MLP, Chapter 07) and as the gate in:

SwiGLU — the gated MLP activation (Chapters 08–09). Given the MLP-input projection split into a *gate* half g and an *up* half u:

$$SwiGLU(g,u) = SiLU(g) \odot u \ .$$

`silu_mul`/`swiglu` fuse the SiLU and the elementwise multiply into one kernel — no intermediate write of SiLU(g). Gated activations consistently beat plain Linear→act→Linear at equal parameters, which is why the whole model's MLPs use them.

15.5 The data-movement kernels

These do no arithmetic — they rearrange — but their layouts are contracts (get one wrong and the model is subtly broken, as Chapter 03's patchify order showed).

- `patchify` / `unpatchify` — latent grid rightarrow token sequence, with the `c·p·p+ph·p+pw` packing (Chapter 03). One CTA per token.
- `qkv_mlp_split` / `split_half` — view a fused GEMM output as its constituent tensors (Q,K,V,gate,up or gate,up). No copy where contiguous; a placement copy for the `[attn ‖ mlp]` concat (Chapter 09).
- `seq_ops`: `concat_two_streams`, `take_tail` — the double→single stream merge (text-first) and the image-tail extraction (Chapter 10). One CTA per output row, striding the hidden dim.
- `transpose_chw` — NCHW ↔ NHWC for the VAE convolutions, because the fast cuDNN Blackwell conv engines require NHWC (Chapter 19). A tiled transpose.
- `upsample` — 2× nearest-neighbor spatial upsample in the VAE decoder (Chapter 20).
- `embed_lookup` — gather rows of the BF16 token-embedding

table by token id, for the Qwen3 encoder (Chapter 21).

15.6 Accuracy: small kernels, tight tolerances

Each kernel has a golden test against an FP32 host reference (`tests/test_*`): `test_rmsnorm`, `test_groupnorm`, `test_swiglu`, `test_modulation`, `test_rope`, `test_rope_4axis`, `test_patchify`, `test_upsample`, … Typical results are max-abs-error ~0.004 (pure BF16 rounding) — these kernels are *exact* up to the storage precision. Because they are simple and individually tested, debugging focuses on the hard parts (GEMM layout, attention, the architecture wiring), with the glue trusted. This is the payoff of the "write once, test once, reuse everywhere" discipline: the connective tissue is not where bugs hide.

15.7 Where this lives in the code

All under `src/backend/cuda/kernels/`: `rmsnorm`, `layernorm`, `groupnorm`, `silu_mul`, `modulation`, `rope`, `rope_4axis`, `patchify`, `embed_lookup`, `qkv_mlp_split`, `seq_ops`, `transpose_chw`, `upsample` (each `.h`/`.cu`), with `tests/test_<kernel>.cu` for the validated ones. The attention kernels live one level up in `attention.{h,cu}` and are Part IV.

15.8 Summary and what to carry forward

- The kernel library is the **memory-bound connective tissue**: norms, activations, and data movement, in every block, cheap individually but numerous.
- Two patterns: **row+reduction** (norms — warp→block reduce, FP32 accumulate) and **elementwise** (activations, RoPE, patchify). Convention: **BF16 storage, FP32 compute**; broadcast params are small BF16 vectors.
- **RMSNorm** dominates (statistics-only at block entry, learned-gain for QK-norm); **LayerNorm** where centered; **Gro**

upNorm-32 in the VAE.

- **SwiGLU** (`SiLU(g)·u`, fused) is the MLP activation.
- Data-movement kernels (`patchify`, `seq_ops`, `transpose_chw`, …) carry **layout contracts**, not math — and those contracts are where Chapter-03-style bugs live.
- Each is golden-tested to BF16 precision, so the glue is trusted and debugging focuses on GEMM/attention/wiring.

That completes Part III — the GPU foundations: the roofline (11), the formats (12), the GEMM (13), the weight pipeline (14), and the kernels (15). Part IV now goes deep on the one kernel that earned its own part: **attention**, the memory-bound heart of every block, taken from a naïve loop to peak HBM bandwidth.

Exercises

1. **RMS vs Layer.** Write both norms' reductions and explain why RMSNorm is cheaper and why "all-ones gain" makes it statistics-only. Why does FLUX pair that with AdaLN modulate?
2. **FP32 accumulate.** Estimate the error of summing 4096 BF16 squares in BF16 vs FP32 for an RMSNorm, and explain why the kernel accumulates in FP32.
3. **Fused SwiGLU.** Show the bytes saved by `silu_mul` fusing SiLU and the multiply vs computing `SiLU(g)` to a temp and multiplying separately, at S=4608, ffn=12288.
4. **A layout contract.** Pick `concat_two_streams` (text-first) and list everything downstream that breaks if you concatenate image-first instead (Chapter 6/10).

Next: Chapter 16 — Attention and the memory wall, opening Part IV.

Chapter 16 — Attention and the Memory Wall

Goal of this chapter: derive scaled dot-product attention, quantify *why* a naïve implementation is a memory-bandwidth disaster on the GB10, and derive the **online softmax** that lets attention be computed in a single streaming pass without ever materializing the S× S score matrix. This is the numerical foundation for the kernel journey of Chapter 17. Anchored to `attention.h` and the fallback kernel in `attention.cu`.

Prerequisites: Chapters 04/08–09 (where attention sits), 11 (roofline); softmax, basic numerical stability.

16.1 Scaled dot-product attention

Attention mixes a sequence of tokens by letting each token (the *query*) gather a weighted average of all tokens' *values*, weighted by query-key similarity. For queries Q∈ R^(S× D), keys K∈ R^(S× D), values V∈ R^(S× D) (one head, S tokens, head-dim D):

$$Attn(Q,K,V) = softmax\left(\tfrac{QK^{\top}}{\sqrt{D}}\right)_{P} \in R^{(S\times S)}\, V\,.$$

Step by step:

1. **Scores** S_ij = tfrac1√ D q_i· k_j — how much query i attends to key j. The 1/√ D keeps the dot products from growing with D (without it, variance scales with D and the softmax saturates).
2. **Weights** P_ij = softmax_j(S_ij) = dfrace^S_ijΣ_j' e^S_ij' — a probability distribution over keys, per query.
3. **Output** O_i = Σ_j P_ij v_j — query i's output is the P-weighted average of all values.

In the model this runs per head (H=32) and the head outputs

are concatenated. The 1/√ D is the `scale` in `Attention::Config`; QK-norm (Chapter 08) further stabilizes the scores before this.

16.2 The cost, and why it is a wall

The score matrix P is S× S. Two costs scale with S^2:

- **Compute:** QK^→p is $2S^2D$ FLOPs, PV another $2S^2D$ — total $4S^2D$ per head.
- **Memory (the killer):** materializing P is S^2 values. At 1024px the transformer's joint attention has S=4608, so P is $4608^2 \approx 21$M entries **per head**, × 32 heads. In FP32 that is ~2.7 GB *per layer*, written to and read back from HBM. The VAE mid-block attention is worse: S=16384 ⇒ $S^2 \approx 268$M.

A naïve "materialize P, then multiply by V" attention is therefore dominated by **writing and re-reading the S× S scores through HBM** — pure memory traffic, the exact thing the GB10 cannot afford (Chapter 11: attention measured at I≈63 FLOP/byte, deeply memory-bound). The arithmetic is cheap; the bytes are everything.

There is a second, subtler trap. The row kernel that *avoids* materializing P by recomputing — one CTA per query, looping over all keys — re-reads **all of K and V from HBM for every query**. That is $O(S^2 D)$ HBM traffic too, just in K/V reads instead of P writes. Either way, naïve attention re-touches $O(S^2)$ data in HBM.

```
naïve A: write P [S×S] to HBM, read it back ........  O(S²) HBM traffic
naïve B (row): re-read all K,V per query ............  O(S²·D) HBM traffic
GOAL: touch K,V once each, never materialize P ......  O(S·D) HBM traffic
```

The project's `attention_row_kernel` (the fallback in `attention.cu`) is naïve B: one CTA per (b,h,q), scores for that query held in smem, block-reduced softmax, then O=Σ P· V. It is correct and simple (cos 1.00000 vs FP32) and fine for short

sequences, but its K/V re-reads are the $O(S^2D)$ HBM cost that dominates at transformer scale. Eliminating those re-reads is what Chapter 17 is about — and it requires computing the softmax *without* having all the scores at once.

16.3 The obstacle: softmax needs the whole row

To tile attention — process keys in blocks, keeping only a block in fast memory — we hit a wall: **softmax is a global operation over the whole row of scores.** The denominato r Σ_j' e^S_ij' needs every key, and the numerically-safe form

P_ij = (e^S_ij - m_i)/(Σ_j' e^S_ij' - m_i), m_i = max_j' S_ij'

needs the **row max** m_i first (subtracting it prevents e^(large) overflow — without it, scores ~30 overflow FP32's exp). So the textbook algorithm is two passes over the keys: one to find m_i and the sum, one to weight V. Two passes means reading K (and V) twice — back to the memory wall.

The breakthrough — **online softmax** (Milakov & Gimelshein 2018; the heart of FlashAttention, Dao et al. 2022) — computes the softmax-weighted output in a **single pass** over the keys, maintaining running statistics and *correcting* them a s new blocks arrive.

16.4 Online softmax, derived

Process keys in order (or in tiles). Maintain three running quantities for a query:

- m — the max score seen *so far*,
- ell — the sum Σ e^(S_j - m) over keys seen so far (relative to the current m),
- O — the running weighted value sum Σ e^(S_j - m) v_j (also relative to the current m).

When a new key j with score s = S_ij arrives, the max may increase to m' = max(m, s). Everything accumulated so far was scaled by e^(-m); to put it on the new scale e^(-m') we multiply by the **correction factor**

$$corr = e^{(m - m')} \le 1 .$$

Then we fold in the new key. The single-key update is:

$$m' = max(m, s), \; ell' = ell \cdot corr + e^{(s - m')}, \; O' = O \cdot corr + e^{(s - m')} \, v_j .$$

Why this is exact. Define f_j = e^S_j - m_final for the final max. The true output is (Σ_j f_j v_j)/(Σ_j f_j). By induction, after each update $O = \Sigma_{j \le t} e^{(S_j - m_t)} v_j$ and $ell = \Sigma_{j \le t} e^{(S_j - m_t)}$ with m_t the running max — the correction $e^{m_{t-1} - m_t}$ exactly rebases the previous partial sums from scale m_{t-1} to m_t, because $e^{S_j - m_{t-1}} \cdot e^{m_{t-1} - m_t} = e^{(S_j - m_t)}$. At the end, divide: O_i = O/ell. No score is ever overflowed (always $s - m' \le 0$), and the whole row of scores is never stored — only (m,ell,O), which is O(D), not O(S).

```
for each key tile:
  load K,V tile into shared memory          (read each K,V byte ONCE)
  for each key j in tile:
    s    = scale · (q · k_j)
    m'   = max(m, s);   corr = exp(m - m')
    p    = exp(s - m')
    l    = l·corr + p
    O    = O·corr + p·v_j                    (rescale running output, add new)
    m    = m'
O_i = O / l                                  (normalize once at the end)
```

This is the algorithmic core of every fast attention kernel in Part IV. It turns attention from "materialize an S× S matrix" into "stream K/V once, keep O(D) state per query." The HBM traffic drops from $O(S^2)$ to O(S· D) — the goal of §16.2 — and the smem footprint is bounded by the tile, not by S (so even t he VAE's S=16384 fits).

16.5 Tiling: sharing K/V across queries

Online softmax removes the *re-read across passes*, but a per-query kernel still loads each K/V tile once **per query**. The second idea is to process a **block of queries together** so they **share** each K/V tile load from HBM. If B_M queries share a tile, K/V is read from HBM B_M× fewer times. This is why the fast kernels use query tiles (B_M=64 in the WMMA/mma kernels) — the K/V tile is streamed into shared memory once and reused by all 64 queries in the CTA before the next tile is loaded. Combined with online softmax, K and V are read from HBM essentially **once each**, which is the bandwidth-optimal lower bound. Chapter 17 is the story of realizing this efficiently on tensor cores.

16.6 The interface

`Attention::Config` is `{batch, seq, n_heads, head_dim, scale}` with tensors laid out `[batch, seq, n_heads, head_dim]` BF16 (the same layout Q/K/V come out of the block projections). One `forward(Q,K,V,O,stream)` call. The contract is identical acr oss all the kernel implementations of Chapter 17 — they are interchangeable behind this interface, which is exactly what let the project swap row → flash → WMMA → regO → mma.sync while keeping the rest of the model unchanged and re-running the same correctness test (`test_attention`, cos vs FP32). The same interface serves the transformer (D=128) and the VAE (D=512, S up to 16384).

16.7 Summary and what to carry forward

• Attention is O=softmax(QK^→p/√ D) V; the score matrix P is S× S, so both compute and (especially) memory scale as S^2.

• Naïve attention is a **memory wall**: either materialize P ($O(S^2)$ writes) or re-read K/V per query ($O(S^2D)$ reads). On the

GB10 attention is decisively memory-bound (I≈63).

• **Online softmax** computes the softmax-weighted output in **one streaming pass**, keeping running (m,ell,O) and rescaling by corr=e^(m-m') as new keys arrive — exact, overflow-safe, O(D) state, no P materialization.

• **Query tiling** makes a block of queries share each K/V tile load, driving HBM traffic to the O(S· D) lower bound — K/V read once each.

• All kernels sit behind one `Attention::forward` interface, so they are swappable and tested identically.

Chapter 17 takes these two ideas (online softmax + query tiling) and walks the five kernels that implement them with increasing efficiency — from the row fallback to a `mma.sync+ldmatrix` kernel pinned to peak HBM bandwidth.

Exercises

1. **The S² bill.** Compute the size of P (one head and all 32) at S=4608 in FP32 and BF16, and the HBM time to write+read it at 273 GB/s. Compare to the measured ~19.7 ms full attention call (Chapter 17).

2. **Online softmax exactness.** Prove by induction that after processing keys 1..t, O=Σ_j≤ t e^(S_j-m_t)v_j and ell=Σ_j≤ te^(S_j-m_t), and hence O/ell is the exact softmax output at the end.

3. **Overflow.** Show that without the running-max subtraction, a score of 90 overflows FP32 `expf`, and that online softmax never evaluates e^(>0).

4. **Tiling win.** With B_M queries per CTA sharing K/V tiles, write the HBM K/V read traffic as a function of S, D, B_M and explain why B_M=64 helps but B_M→∞ is limited by smem/registers (foreshadowing Chapter 17).

Next: Chapter 17 — The kernel journey.

Chapter 17 — The Kernel Journey

> *Goal of this chapter:* tell the full optimization story of the D=128 attention kernel — five implementations, each fixing the bottleneck the last one exposed — from a naïve row loop to a `mma.sync+ldmatrix` kernel pinned to **peak HBM bandwidth**. Every step has a measured number and a *reason*. The throughline is a lesson in finding the real bottleneck before optimizing: the biggest win came from diagnosing an **occupancy** limit that "obvious" optimizations would have missed. Anchored to `attention.cu` and `tools/bench_attention.cu`.
>
> *Prerequisites:* Chapter 16 (online softmax, tiling), 11 (roofline), 13 (tensor cores).

17.1 The scoreboard

All numbers are the GB10 at the real transformer shape **S=4608, H=32, D=128** (1024px joint attention), measured with `tools/bench_attention.cu`:

 # | kernel | per-call | HBM | vs prev | the idea
1 | attention_row_kernel | — (baseline B) | O(S²D) re-reads | — | one CTA per query
2 | flash_attn_kernel | (4.1× denoise vs row) | streams K/V | — | warp-per-query + online softmax
3 | flash_wmma_kernel | 56.5 ms | 97.7 GB/s (36%) | 1.45× denoise | tensor-core QK^T/PV
4 | flash_wmma_regO_kernel | 34.3 ms | 160 GB/s | 1.65× | register-resident O → 2 CTA/SM
5 | flash_mma_kernel | 19.7 ms | 279 GB/s (~peak) | 1.74× | mma.sync + ldmatrix

End to end, kernel 5 vs the committed kernel 3 took the **denoise loop from 8.41 s to 6.54 s** at 1024px (the difference is exactly 128 attention calls × the per-call saving). Total speedup #3→#5: **2.87×**. The rest of this chapter is *how* and, more importantly, *why each step*.

17.2 Step 1→2: stop re-reading K/V (flash)

The row kernel (Chapter 16, "naïve B") computes one query per CTA and re-reads all of K and V from HBM for each query — $O(S^2D)$ HBM traffic, the memory wall.

`flash_attn_kernel` applies Chapter 16's two ideas: **online softmax** (one pass, O(D) state) and **query tiling** (8 warps/CTA, one query per warp, so 8 queries share each K/V tile loaded into shared memory). K/V is read from HBM ~8× fewer times. Each lane owns D/32 dims of its query in registers; scores are a warp dot-reduce; the running (m,ell,acc) live in registers; the smem holds only the current `TILE_K` tile, so even the VAE's S=16384 fits. **Denoise 114 s → 28 s (4.1×)** at 1024px. This kernel is still the path for Dne128 (e.g. the VAE's D=512), where its register layout fits but the tensor-core tiling below does not.

17.3 Step 2→3: put the matmuls on tensor cores (WMMA)

The warp-per-query kernel computes q· k with warp shuffles — CUDA cores, not tensor cores. For D=128 the dot products are exactly what tensor cores are built for. `flash_wmma_kernel` is FlashAttention-2-style: **B_M=64 queries** per CTA (4 warps × 16), **B_N=48** keys per K/V tile, QK^→p and P V on tensor cores (`wmma`, BF16 MMA, FP32 accumulate), online softmax rescaling the FP32 O accumulator held in shared memory. (B_N=48 because 64 overflows the 99 KiB smem cap.) A non-obvious tuning detail: the **softmax epilogue** (the `exp`, the O-rescale, the PV add-back), not the matmul, was the bottleneck, so it is spread across all 32 lanes via flat indexing rather than left on the 16 row-owning lanes. **Denoise 28 s → 19.25 s (1.45×)**; per-call **56.5 ms**. This was the committed baseline at the start of the session.

17.4 The diagnosis that mattered: occupancy, not bandwidth

Here is the pivotal moment, and the methodological lesson of the whole chapter.

At 56.5 ms the WMMA kernel runs at **97.7 GB/s — only ~36 % of the GB10's 273 GB/s peak — and 6.2 TFLOP/s.** It is nowhere near the memory ceiling *or* a compute ceiling. So it is neither bandwidth-bound nor compute-bound; it is **utilizati on/occupancy-bound**. The "obvious" next optimizations — *b igger query tiles for more K/V reuse*, *persistent-CTA scheduling* — both assume a bandwidth/launch bottleneck the kernel **does not have**. They would have been wasted effort.

The real cause, found by smem accounting + `cudaOccupancyMa xActiveBlocksPerMultiprocessor`: the kernel uses **93 KiB of shared memory per CTA** (Ks + Vs + Qs + Ps + the FP32 Ss + Os + per-row stats). Against the 99 KiB opt-in cap that allows exactly **1 CTA per SM** — 4 warps on an SM that can hold ~48. With only one CTA resident there is **no latency hiding**: when the barrier-heavy softmax epilogue stalls, the SM has nothing else to run. The kernel is starved of parallelism, not bandwidth.

> **The lesson.** Measure *which* ceiling you are under before optimizing. The profiler (GB/s, TFLOP/s, occupancy) said "occupancy," and the lever that follows from "occupancy-bound" is **cut shared memory to fit more CTAs** — the opposite of "use bigger tiles." Chasing the intuitive-but-wrong lever is the most common way to waste optimization effort.

17.5 Step 3→4: register-resident O (regO)

To raise occupancy we must shrink smem. The biggest consumers are the FP32 output accumulator `Os[BM][D]` (32

KiB) and the staged queries `Qs[BM][D]` (16 KiB). `flash_wmma_reg O_kernel` removes both: hold the **O accumulator in registers** (the `wmma` accumulator fragments) and load **Q straight from global** into its fragments. Shared memory drops **93 KiB → 43 KiB → 2 CTAs/SM** (8 warps), doubling occupancy.

Result: **56.5 ms → 34.3 ms (1.65×)**, 97.7 → 160 GB/s, **bit-exact** (cos_vs_prod 1.00000). The win comes purely from latency hiding — two CTAs means when one stalls on the epilogue the other runs. Productionizing it took one fix: the final normalize/write loop wrote all 16 rows of a query tile unconditionally, which is an **out-of-bounds write for a partial last tile** (when $q_0+r \geq S$); guarding with `if (q0 + r < S)` made it correct for non-tile-aligned S (e.g. S=130 in `test_attention`).

The rescale tax. regO pays a price for hiding O in `wmma` fragments: `wmma` *hides the fragment's row→lane mapping*, so when online softmax needs to multiply each row of O by its correction factor, the kernel cannot index "row r" in registers. It must **store the fragment to smem, scale per row there, and reload it** — per K/V tile, per output column-tile. (The *PV add-back* is free — both accumulators share the hidden layout, so `ofrag.x[i] += acc.x[i]` works without knowing rows — but the per-row *rescale* does not.) This store/scale/reload is the "rescale tax," and removing it is the next step.

17.6 Step 4→5: mma.sync + ldmatrix (peak)

`wmma` is a convenience wrapper with opaque fragment layouts. Dropping to **raw** `mma.sync` **PTX** (Chapter 18) exposes the *documented* thread→register layout, which unlocks three things at

once:

1. **The rescale tax disappears.** With a known layout, each thread knows which two rows of O its accumulator registers hold, so the per-row correction is a plain **register multiply** — no smem store/reload.
2. **The softmax leaves smem.** Row max/sum become **register reductions across the 4 lanes that share a row** (`__shfl_xor`), so the score matrix never touches shared memory.
3. **No P round-trip.** Because P V's contraction dim (the keys) equals QK^→p's N dim, and the `mma` C-fragment and A-fragment share the same row/column mapping, the QK^→p result registers **repack directly** into the P V A-fragments (cast to BF16) — the probabilities never go to smem either.

The inner loop now touches shared memory **only** for the unavoidable K/V global→smem staging. Smem drops to 24 KiB.

But correctness-first cost speed first. The first `mma.sync` version loaded the K/V fragments with *manual scalar* `pack2` *reads* from smem — correct (bit-exact, all `test_attention` shapes incl. S=130), higher occupancy (3 CTA/SM), but **42.3 ms — slower than regO's 34.3.** The 96 scalar bank-conflict-prone smem reads per tile cost more than the rescale tax they removed. The unlock was `ldmatrix` — the warp-cooperative, conflict-free shared-memory→fragment load that `wmma` had been using internally all along. Swapping the manual loads for `ldmatrix.x2` (with the address recipes validated in Chapter 18) dropped it to **19.7 ms at 279 GB/s** — essentially the **memory ceiling** (279 ≥ 273 GB/s). The kernel is now bandwidth- bound; there is nothing left to win, because the K/V bytes cannot arrive faster.

```
 56.5 ms  WMMA           1 CTA/SM   97.7 GB/s   — occupancy-bound
 34.3 ms  regO           2 CTA/SM   160  GB/s   — + register O (rescale tax remains)
 42.3 ms  mma (manual)   3 CTA/SM   130  GB/s   — correct but scalar loads dominate
```

```
19.7 ms  mma + ldmatrix 2 CTA/SM   279  GB/s   — BANDWIDTH-BOUND (peak)  ✔ production
```

(Note the final kernel is *back* to 2 CTA/SM, lower than the manual version's 3 — register pressure rose — but it does not matter: at the memory ceiling, occupancy beyond what hides latency buys nothing.)

17.7 The methodology, distilled

The journey is a template for kernel optimization on a bandwidth-bound machine:

1. **Eliminate redundant HBM traffic first** (row→flash): the algorithm (online softmax + tiling) beats any micro-optimization of a wasteful one.
2. **Use the right compute units** (flash→WMMA): tensor cores for the matmuls.
3. **Measure which ceiling you are under** (WMMA diagnosis): GB/s vs peak, TFLOP/s vs peak, and `cudaOccupancyMaxActiveBlocks`. *Do not* optimize for a bottleneck you have not confirmed.
4. **Pull the right lever** (occupancy → cut smem → regO): the diagnosis dictates the lever; here "smaller smem," the opposite of the intuitive "bigger tiles."
5. **Validate every step against an independent oracle and bit-exactly where possible** (`test_attention` vs FP32, cos_vs_prod, the end-to-end image A/B): a faster wrong kernel is worthless, and reordered-summation BF16 drift (1.8% image diff mma-vs-regO) must be distinguished from bugs (both match FP32 at cos 1.0).
6. **Know when to stop** (279 GB/s ≈ peak): declare victory at the ceiling, not when the code stops being improvable in the abstract.

`bench_attention.cu` is the instrument: it prints per-shape latency, the K/V-reread HBM model in GB/s, TFLOP/s, and (via the probes) the achieved CTAs/SM — exactly the numbers

each decision above needed.

17.8 What didn't make the cut (and why)

• **Bigger query tiles / persistent CTAs** — rejected by the occupancy diagnosis (§17.4); they target a bottleneck the kernel doesn't have.

• `ldmatrix.x4` **to halve load instruction count**, **K/V double-buffering** — possible micro-opts, but the kernel is already at peak HBM bandwidth, so they cannot help a memory-bound kernel meaningfully. Noted, not pursued.

• **A bespoke kernel for D=512 (VAE)** — the warp-per-query flash kernel already serves it; its smem is too large for the B_M=64 tensor-core tiling, and the VAE decode is dominated by convolutions (Chapter 19), not attention, so it wasn't worth a special kernel.

17.9 Where this lives in the code

```
kernel | symbol in attention.cu
row fallback | attention_row_kernel
warp-per-query flash | flash_attn_kernel<DPT> (D∈{64,256,512})
WMMA (superseded) | flash_wmma_kernel<D> (kept for bench A/B)
register-O WMMA (superseded) | flash_wmma_regO_kernel<D> (kept for bench A/B)
production D=128 | flash_mma_kernel<D> (mma.sync + ldmatrix)
dispatch | Attention::forward (D=128 → mma; D∈{64,256,512} → flash; else row)
benchmark/probes | tools/bench_attention.cu, launch_*_probe
```

17.10 Summary and what to carry forward

• The kernel went **56.5 → 19.7 ms** (2.87×) by, in order: streaming K/V (flash), tensor cores (WMMA), **raising occupancy by cutting smem** (regO), and removing smem round-trips with `mma.sync`+`ldmatrix` (peak bandwidth).

• The decisive insight was a **profiling diagnosis**: WMMA was occupancy-bound (1 CTA/SM from 93 KiB smem), so the lever was *less* shared memory, not bigger tiles.

• The `mma.sync` kernel removes the **rescale tax**, the smem

softmax, and the **P round-trip** (QK^T registers repack into PV fragments); `ldmatrix` made its fragment loads cheap enough to hit **279 GB/s ≈ peak**.

• Method: eliminate redundant HBM → right compute units → **measure the ceiling** → pull the matching lever → validate bit-exactly → **stop at the ceiling**.

Chapter 18 opens the hood on the `mma.sync` and `ldmatrix` PTX — the exact fragment layouts and the standalone harness (`mma_unit.cu`) that validated them before they ever touched the attention kernel.

Exercises

1. **Why 36 % is the tell.** Explain why 97.7 GB/s (≈36 % of peak) and 6.2 TFLOP/s *together* imply occupancy-bound rather than bandwidth- or compute-bound.

2. **smem → occupancy.** From 93 KiB/CTA and the 99 KiB cap, derive 1 CTA/SM; from 43 KiB derive 2. Why does 2 CTA/SM hide the epilogue latency that 1 cannot?

3. **The rescale tax.** Explain why holding O in `wmma` fragments forces a smem store/scale/reload for the per-row correction, while the PV add-back is free. What property of `mma.sync` removes the tax?

4. **Manual vs ldmatrix.** The manual-load `mma` kernel had *higher* occupancy yet was *slower* than regO. Reconcile this with §17.4's "raise occupancy" lesson. (Hint: occupancy is necessary, not sufficient; what dominated?)

Next: Chapter 18 — `mma.sync` & `ldmatrix` up close.

Chapter 18 — mma.sync & ldmatrix Up Close

Goal of this chapter: open the hood on the two PTX instructions that made the peak-bandwidth attention kernel possible — the `mma.sync` tensor-core multiply-accumulate and the `ldmatrix` cooperative shared-memory load. We give the ex act thread→register fragment layouts, show how knowing them removes the rescale tax and the P round-trip (Chapter 17), and walk the **validation methodology** (`tools/mma_unit.c u`) that locked every layout bit-exactly *before* it touched the at tention kernel. This is the most hardware-level chapter; it is also a reusable recipe for hand-writing tensor-core kernels. Anchored to `attention.cu` and `tools/mma_unit.cu`.

Prerequisites: Chapter 17; CUDA warps/lanes, inline PTX, shared memory.

18.1 Why drop below wmma

`wmma` (the `nvcuda::wmma` C++ API) is convenient but treats fragments as **opaque** — you cannot know which register of which lane holds matrix element (r,c). That opacity is exactly what forced regO's *rescale tax* (Chapter 17): to scale "row r" of the O accumulator you had to round-trip it through shared memory. The raw PTX instruction `mma.sync` has a **documente d, fixed** thread→register mapping. Knowing it lets you operate on fragment elements *in registers* — rescale by row, repack one matmul's output into the next's input — eliminating the smem round-trips. The price is that you manage the layouts yourself, which is error-prone — hence §18.5's validation harness.

18.2 The instruction

We use the BF16 tensor-core MMA with FP32 accumulation, shape **m16n8k16**:

```
mma.sync.aligned.m16n8k16.row.col.f32.bf16.bf16.f32
   {d0,d1,d2,d3},        // D : 16×8 FP32 accumulator (also C, in-place)
   {a0,a1,a2,a3},        // A : 16×16 BF16, row-major  (4 regs × 2 bf16)
   {b0,b1},              // B : 16×8  BF16, col-major  (2 regs × 2 bf16)
   {d0,d1,d2,d3};        // C : accumulate into D
```

One warp (32 lanes) cooperatively computes D = A· B + C for a 16×8 tile contracting over K=16. `.row.col` means A is row-major, B is column-major. The operands and result are *di stributed across the warp's registers* per a fixed map. The project's wrapper (`attention.cu`):

```
__device__ void mma_m16n8k16(float (&d)[4], const uint32_t (&a)[4], const uint32_t (&b)[2]) {
    asm volatile(
      "mma.sync.aligned.m16n8k16.row.col.f32.bf16.bf16.f32 "
      "{%0,%1,%2,%3},{%4,%5,%6,%7},{%8,%9},{%0,%1,%2,%3};"
      : "+f"(d[0]),"+f"(d[1]),"+f"(d[2]),"+f"(d[3])
      : "r"(a[0]),"r"(a[1]),"r"(a[2]),"r"(a[3]),"r"(b[0]),"r"(b[1]));
}
```

The `"+f"` ties C and D to the same registers (accumulate in place). `a/b` are `uint32_t` each packing two BF16 values.

18.3 The fragment layouts (the crucial table)

Let `gid = lane/4` (0–7) and `t = lane%4` (0–3). The PTX ISA fixes:

A (16×16, row-major) — 4 regs, each 2 BF16:

```
 a0 = A[gid  ][2t], A[gid  ][2t+1]      (k = 0..7   region)
 a1 = A[gid+8][2t], A[gid+8][2t+1]
 a2 = A[gid  ][2t+8], A[gid  ][2t+9]    (k = 8..15  region)
 a3 = A[gid+8][2t+8], A[gid+8][2t+9]
```

B (16×8, col-major) — 2 regs:

```
 b0 = B[2t  ][gid], B[2t+1][gid]        (k = 0..7)
 b1 = B[2t+8][gid], B[2t+9][gid]        (k = 8..15)
```

C / D (16×8, FP32) — 4 regs:

```
 d0 = C[gid  ][2t]      d1 = C[gid  ][2t+1]
 d2 = C[gid+8][2t]      d3 = C[gid+8][2t+1]
```

Two facts in this table do all the work in the attention kernel:

• **C and A share the row map.** Both put row `gid` in their first pair of regs and row `gid+8` in the second. So a thread that computes C-row `gid` *also owns* A-row `gid` — which is why O (a C-fragment) can be rescaled in registers per row (§18.4) and why a QK^T result (C) repacks into a PV input (A) for free (§18.4).

• **The column index is** `2t`/`2t+1` for C — so the 4 lanes of a `gid`-group (t=0..3) collectively own all 8 columns of a row, letting a softmax row-reduction be a `__shfl_xor` over those 4 lanes (Chapter 17).

18.4 How the kernel exploits the layout

In `flash_mma_kernel` (Chapter 17), with D=128, B_N=48 keys/tile, B_M=64 queries/CTA (16 per warp):

O-rescale, in registers. O is a set of C-fragments `of[nt][4]`. After computing the per-row corrections `corr_lo` (row `gid`) and `corr_hi` (row `gid+8`), the rescale is just:

```
of[nt][0]*=corr_lo; of[nt][1]*=corr_lo;   // d0,d1 are row gid
of[nt][2]*=corr_hi; of[nt][3]*=corr_hi;   // d2,d3 are row gid+8
```

No smem, no reload — *the tax is gone*, because the layout tells each thread which rows its 4 accumulator regs hold.

Softmax in registers. The scores from QK^T are C-fragments. Row max/sum reduce over the 4 lanes sharing a row via `__shfl_xor_sync(mask, v, 1)` then `..., 2)` (which stay within the aligned group of 4). Scores never touch smem.

QK^T → PV repack, for free. PV's contraction dim is the keys = QK^T's N dim, and C's row map equals A's row map with the same `2t` column map. So the QK^T result registers (the probabilities `pcb`, cast to BF16) **are** the PV A-fragment, just packed two-per-uint32:

```
const uint32_t af[4] = {
   pack2(pcb[2*kt][0], pcb[2*kt][1]),    // A-row gid,   k=2t..  ← C d0,d1
   pack2(pcb[2*kt][2], pcb[2*kt][3]),    // A-row gid+8        ← C d2,d3
   pack2(pcb[2*kt+1][0], pcb[2*kt+1][1]),
   pack2(pcb[2*kt+1][2], pcb[2*kt+1][3]) };
```

No store of P to smem and back — the P round-trip regO/WMMA paid is eliminated. The inner loop's only smem traffic is the K/V global→smem staging.

18.5 ldmatrix: the cooperative fragment load

The one thing that *must* come from shared memory is the K/V tile (it arrives from global). Loading it into the `mma` B-fragment layout with manual scalar reads was *correct but slow* (Chapter 17: 42.3 ms — bank conflicts, 96 scalar reads/tile). `ldmatrix` is the instruction `wmma` uses internally: one warp-cooperative, conflict-free instruction loads four (or two) **8×8 BF16** tiles from shared memory *directly into the fragment layout* `mma` expects.

```
__device__ void ldmatrix_x2(uint32_t (&r)[2], const void* p) {
    uint32_t a = __cvta_generic_to_shared(p);
    asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0,%1}, [%2];"
                 : "=r"(r[0]),"=r"(r[1]) : "r"(a));
}
// + an x2.trans variant that transposes each 8×8 on load.
```

For `x2`, lanes 0–15 each supply the **address of one row** of the two 8×8 tiles; the hardware distributes the 128 loaded BF16 values across all 32 lanes' result registers in exactly the `mma` B layout. The `.trans` variant transposes each 8×8 — need ed when the source's row/column roles are swapped relative to what `mma` wants.

18.6 The address recipes — and how they were found

The hard part of `ldmatrix` is the **address each lane supplies**

, which depends on the source's memory layout. The project pinned the two recipes it needs *empirically*, by sweeping `(sel, trans)` in `mma_unit.cu` until the result matched a reference (§18.7). The winners:

QK^T — K as the B operand. K is in smem as `Ks[key][hd]` (row=key, col=hd). For QK^→p, B is K^→p, which in this storage is **column-major** B — so `ldmatrix` **non-trans**, with each lane addressing its key row and hd half:

```
// nt = key 8-tile, kt = hd 16-tile, D = head dim
ldmatrix_x2(bf, &Ks[(nt*8 + (lane&7))*D + kt*16 + ((lane>>3)&1)*8]);
```

PV — V as the B operand. V is in smem as `Vs[key][hd]` (row=key, col=hd). For PV, B is V with key as the contraction (row) and hd as N (col) — **row-major** B — so `ldmatrix` **trans**:

```
// nt = hd 8-tile, kt = key 16-tile
ldmatrix_x2_trans(bf, &Vs[(kt*16 + (lane&15))*D + nt*8]);
```

(Q stays a manual register load — it is loaded once, outside the K loop, so it is not on the hot path; PV's A operand comes from registers via the §18.4 repack.) Swapping these in for the scalar loads is what took the kernel from 42.3 ms to **19.7 ms at 279 GB/s** (Chapter 17).

18.7 The validation harness: mma_unit.cu

Hand-written PTX layouts are a minefield — a wrong index does not crash, it produces *plausible wrong numbers*. The project's defense was a **standalone, fast-iterating harness** that validated each layout against a CPU reference *before* it went into the kernel. `tools/mma_unit.cu` (built with a one-line `nvcc -arch=sm_121a`) does two things:

1. **Validate the `mma` primitive.** Load a known 16×16 A and 16×8 B with *manual* index-by-index fills (using the §18.3 table), run `mma_m16n8k16`, write C back via the C-layout, and compare to a CPU `A·B`. Result: **max-abs-error 0.00000** —

the layout table is exactly right.

2. **Sweep the** `ldmatrix` **recipes.** For each B operand (QK^T's col-major K source, PV's row-major V source), try `sel ∈ {0,1}` × `trans ∈ {0,1}` address recipes, run the `mma`, and print which combination reproduces the reference. Output:

This is *why the attention kernel was bit-exact on the first compile*: the matmul primitives and load recipes were proven in isolation, so the only thing left to get right in the kernel was the online-softmax bookkeeping (which the `test_attention` oracle then confirmed). Keep `mma_unit.cu` for any future tensor-core work — it is the cheapest possible way to settle a layout question.

> **The general technique.** When hand-writing tensor-core PTX: (1) write the layout table from the ISA, (2) validate the bare `mma` with manual fills vs a CPU reference, (3) *sweep* the `ldmatrix` address/transpose options against the same reference rather than reasoning them out, (4) only then build the real kernel, and (5) validate it against an independent oracle. Steps 2–3 take minutes and save hours of debugging a "plausible but wrong" full kernel.

18.8 Where this lives in the code

Concept	Code
mma.sync wrapper + layout comment	attention.cu mma_m16n8k16, pack2
ldmatrix wrappers	attention.cu ldmatrix_x2, ldmatrix_x2_trans
the production kernel using them	attention.cu flash_mma_kernel<128>
layout + recipe validation	tools/mma_unit.cu
CUTLASS reference for the PTX	third_party/cutlass/include/cute/arch/{mma_sm80,copy_sm75}.hpp

18.9 Summary and what to carry forward

- `mma.sync.m16n8k16.f32.bf16` computes a 16×8 tile per warp with a **fixed, documented** thread→register layout (the §18.3 table). Dropping below `wmma` exposes it.

• That layout has two gifts: **C and A share the row map** (so O rescales in registers and QK^T results repack into PV inputs for free), and **columns are** `2t` (so softmax reduces via `__shfl_xor` over 4 lanes). Together they remove the rescale tax, the smem softmax, and the P round-trip.

• `ldmatrix` is the conflict-free cooperative smem→fragment load; its address recipe depends on the source layout (non-trans for QK^T's col-major K, trans for PV's row-major V). Manual scalar loads were correct but 2× slower.

• `mma_unit.cu` validated the `mma` layout (max-err 0.0) and *swept* the `ldmatrix` recipes to a match before any kernel code — which is why the kernel was correct on the first compile. Reuse this harness for any tensor-core PTX work.

That closes Part IV — attention from the math (16) through the optimization journey (17) to the metal (18). Part V turns to the VAE decoder and the very different performance story of **convolutions** on Blackwell.

Exercises

1. **Read the layout.** For lane 5 (gid=1, t=1), name exactly which A, B, and C elements its registers hold, using §18.3.

2. **Free repack.** Show that the QK^T C-fragment register `d0` for a thread equals the PV A-fragment element that thread needs, and explain why this requires C and A to share the row map *and* the column convention.

3. **trans or not.** Given K stored `[key][hd]` and V stored `[key][hd]`, explain why QK^T's K-load is non-trans but PV's V-load is trans. (Hint: which axis is the contraction for each matmul?)

4. **The harness pays off.** A full attention kernel returns plausible-but-wrong numbers. Argue why isolating the `mma`+`ldmatrix` in `mma_unit.cu` localizes the bug faster than debugging the kernel directly. What class of bug does each step catch?

Next: Chapter 19 — Convolutions with the cuDNN 9 graph API, opening Part V.

Chapter 19 — Convolutions with the cuDNN 9 Graph API

> *Goal of this chapter:* the VAE decoder is convolutions, and getting them onto Blackwell's tensor cores was a 14× story with a sharp lesson. We cover what a conv is (and why it is "just" a GEMM), why the *legacy* cuDNN API silently misses the fast Blackwell engines, the two compounding causes (a stale library link and the legacy API itself), and the cuDNN 9 **graph API + NHWC** rewrite that took VAE decode from **18.7 s to 1.35 s** at 1024px. Anchored to `conv2d.{h,cu}`.
>
> *Prerequisites:* Chapters 11 (roofline), 13 (GEMM, tensor cores), 03 (the VAE).

19.1 Convolution is a GEMM in disguise

A 2D convolution slides a filter bank W∈ R^(C₀ut× C_in × R× S) over an input x∈ R^(C_in× H× W), producing y∈ R^(C₀ut× H'× W') where each output pixel is a dot product of the filter with a local input patch. The classic way to see it as a matmul is **im2col**: gather every C_in× R× S input patch into a column , stack the columns into a matrix X̃ of shape [(C_inRS)×(H'W')], reshape W to [C₀ut×(C_inRS)], and then y = WX̃ is an ordinary GEMM. Modern libraries use *implicit GEMM* (they never materialize X̃; they index it on the fly), but the point stands: **a conv is a GEMM**, and on Blackwell it wants the same tensor cores as Chapter 13.

The FLUX VAE uses only 3×3 and 1×1 convolutions, single-stride, dilation 1, no grouping (`conv2d.h`). 1×1 convs are literally per-pixel matmuls; 3×3 are the im2col GEMM above with a 9-patch gather.

19.2 The symptom: convolutions 170× too slow

When the VAE decoder was first built on cuDNN, a single 128-channel 1024^2 3×3 convolution took **~510 ms**. For reference, PyTorch does the same conv in ~3 ms, and the eventual fixed version here does it in ~29 ms. At ~510 ms/conv the whole VAE decode was **18.7 s** at 1024px — larger than the entire denoise loop. Something was dispatching to a slow, non-tensor-core kernel.

19.3 The diagnosis: two compounding causes

Cause 1 — a stale cuDNN link. CMake linked the *unversioned* dev symlink `libcudnn.so`, which on this box pointed at cuDNN **8.9** (the apt package). cuDNN 8.9 **predates Blackwell** — it has no sm_121 convolution engines at all, so it falls back to a generic kernel. The system *also* had cuDNN **9.23** installed (`libcudnn.so.9` plus `engines_{precompiled,runtime_compiled,tensor_ir}.so.9`), and the headers in `/usr/include` already reported `CUDNN_MAJOR 9` — so the code compiled against 9's headers but *linked* 8.9's runtime, a silent mismatch. **Fix:** link `/usr/lib/aarch64-linux-gnu/libcudnn.so.9` explicitly in CMake.

Cause 2 — the legacy API. Linking cuDNN 9 alone did *not* fix it: the legacy `cudnnConvolutionForward` path stayed ~510 ms even on 9. On Blackwell, the fast tensor-core conv engines are reachable **only** through the cuDNN 9 **backend graph API**, and **only with NHWC** layout. The legacy API and NCHW both route to slow fallback kernels. So the fix was not just "newer cuDNN" but "newer cuDNN, *graph API*, *NHWC*."

> **The lesson.** A 170× performance bug had *two* causes, neither of which threw an error: a build-system link pointing

at the wrong runtime, and an API that "works" but never touches the fast path. Both are invisible to correctness tests (the math was right the whole time) — only a *performance* measurement against a known-good reference (PyTorch's 3 ms) revealed the gap. Always benchmark a new dependency path against a reference implementation, not just check that it produces correct output.

19.4 The cuDNN 9 graph API

The modern cuDNN interface is a **dataflow graph**: you describe the operation as a graph of tensor nodes and op nodes, hand it to a **heuristic** that proposes **engine configurations** (concrete kernels + tile/stage choices), pick one, and **execute** it with a workspace. It is more verbose than the one-call legacy API, but it is the only way to reach the autotuned, architecture-specific engines.

The rewritten `conv2d.cu`:

1. **Builds an op-graph** with **NHWC** tensors and a **KRSC** filter (the layouts the Blackwell engines want).
2. Asks the heuristic in `ENGINEHEUR` **mode A** for engine configurations. The heuristic returns them **sorted by predicted performance**, so `config[0]` **is the Blackwell tensor-core engine**. Benchmarking *all* configs picked the same engine but cost ~13 s of construction; taking `config[0]` directly cuts construction to **~0.47 s** with the identical kernel.
3. **Executes** the plan with a device workspace.

```
build graph (NHWC in/out, KRSC filter, conv node)
  → heuristic (ENGINEHEUR mode A) → configs sorted by predicted perf
    → take config[0]  (the tensor-core engine; no full autotune sweep)
      → build execution plan + workspace
        → execute(x_nhwc, W_krsc, y_nhwc)
```

19.5 Keeping an NCHW interface

The rest of the VAE (resnet blocks, group norm, upsample — Chapter 20) and the patchify/transpose conventions are all **NCHW**. Rather than ripple NHWC through everything, `Conv2d` keeps an **NCHW interface** and transposes internally:

```
x [N,C,H,W]  --nchw_to_nhwc-->  x_nhwc  --cuDNN graph-->  y_nhwc  --nhwc_to_nchw-->  y [N,C',H',W']  (+ bias)
```

The weight is reordered **KCRS → KRSC once at construction**. The transposes use the existing `transpose_chw` kernel (Chapter 15); they cost some bandwidth but are far cheaper than the kernel-selection win. The workspace is `x_nhwc + y_nhwc + plan_workspace` — ~5.1 GiB at 1024², which unified memory absorbs (Chapter 11). Callers and the other VAE kernels are unchanged.

19.6 The payoff

measurement	before (legacy/8.9)	after (graph API/9/NHWC)	speedup
one 128-ch 1024² 3×3 conv	~510 ms	~29 ms	~18×
VAE decode @ 1024px	18.7 s	1.35 s	13.8×
VAE forward @ 128px	314 ms	10 ms	31×
total 1024px image	~133 s	~30 s	(this + attention + denoise)
construction (engine pick)	(n/a)	0.47 s (config[0]) vs 13 s (full sweep)	28×

Correctness held throughout: `tests/test_conv2d.cu` shows cos 1.00000 vs an FP32 reference, and the 1024px image is bit-identical to the legacy path up to BF16 rounding (mean Δ 0.055/255 — a *different conv kernel*, same math). After this fix the VAE decode (1.35 s) is no longer a bottleneck; the denoise loop (Part IV) is the dominant cost again.

19.7 Why the VAE conv was memory-bound-ish too

On the roofline (Chapter 11), these convs are large (high channel counts, big spatial extent) but not as compute-dense as the transformer's huge GEMMs, and the NHWC transposes add traffic — so the win is partly *reaching the tensor cores at*

all (Cause 2) and partly *moving the right bytes in the right layout*. The deeper point: even with perfect math, you can sit at a tiny fraction of peak if you are on the wrong kernel/layout/library — performance is a property of the *whol e dispatch path*, not just the algorithm.

19.8 Where this lives in the code

Concept	Code
conv wrapper (graph API, NHWC, NCHW interface)	conv2d.{h,cu}
NCHW↔NHWC transpose	kernels/transpose_chw.*
cuDNN 9 link	CMake (libcudnn.so.9, not the .so symlink)
correctness	tests/test_conv2d.cu
users	vae_resnet.*, vae_decoder.* (Chapter 20)

19.9 Summary and what to carry forward

• A convolution **is a GEMM** (im2col / implicit GEMM); on Blackwell it wants the same tensor cores as `Linear`.

• The VAE convs were **170× too slow** from **two silent causes**: CMake linked **cuDNN 8.9** (pre-Blackwell) via the unversioned symlink, *and* the **legacy API** never reaches the fast engines even on cuDNN 9.

• The fix is the cuDNN 9 **backend graph API + NHWC + KRSC**, picking the heuristic's `config[0]` (tensor-core engine) without a full autotune sweep. `Conv2d` keeps an NCHW interface and transposes internally.

• Result: **VAE decode 18.7 s → 1.35 s (13.8×)**, 128px forward 31×, bit-identical output. *Benchmark new dependency paths against a reference — correctness tests w on't reveal a wrong-kernel dispatch.*

Chapter 20 assembles these convs into the full decoder — resnet blocks, group norm, upsampling, the mid-block spatial attention, and the `post_quant_conv`/de-norm boundary from Chapter 03.

Exercises

1. **Conv as GEMM.** For a 3×3, C_in=512→ C$_o$ut=512 conv at 128×128, write the im2col matrix shapes and the GEMM dimensions M,N,K. Is it above or below the FP-relevant ridge (Chapter 11)?

2. **Two silent bugs.** Explain why neither the stale cuDNN link nor the legacy API produced *wrong* output, and why only a performance comparison to PyTorch revealed them. What test would catch this in CI?

3. **config[0] vs sweep.** Why does taking the heuristic's first config give the same kernel as a full benchmark sweep here, and when might it *not*? What did the sweep cost?

4. **Layout tax.** The NCHW interface transposes to NHWC and back per conv. Estimate that traffic at 512ch 128^2 BF16 and argue why it is worth paying for the tensor-core engine.

Next: Chapter 20 — Assembling the VAE decoder.

Chapter 20 — Assembling the VAE Decoder

Goal of this chapter: build the full decoder that turns the clean [32, H/8, W/8] latent into an [3, H, W] image. We cover the ResnetBlock primitive, the mid-block **spatial self-attention** (which reuses the Part-IV attention kernel at D=512), upsampling, the assembled architecture, the `post_quant_conv` / batch-norm boundary (Chapter 03), the workspace strategy, and the decisive correctness test. Anchored to `vae_decoder.{h,cu}`, `vae_resnet.{h,cu}`, `vae_attn.{h,cu}`.

Prerequisites: Chapters 03 (latent/VAE role, the bugs), 15 (GroupNorm/SiLU/ upsample), 16–18 (attention), 19 (conv).

20.1 The decoder's job and shape

From Chapter 03: the transformer emits a clean latent, `unpatchify` turns it back into a [32, H/8, W/8] grid, and the decoder expands it **8×** spatially to [3, H, W] RGB. The expansion is a stack of residual convolution blocks with three 2× upsamples (2×2×2 = 8×), a self-attention at the lowest resolution, and a final projection to 3 channels. All the heavy lifting is the convolutions of Chapter 19; this chapter is how they are wired.

20.2 The ResnetBlock primitive

The decoder is mostly **ResnetBlocks** (`vae_resnet.h`), the standard pre-activation residual conv block:

```
residual = GroupNorm32(x) → SiLU → conv1 (3×3, may change channels)
         → GroupNorm32     → SiLU → conv2 (3×3, channels stay)
skip     = x               if C_in == C_out
         = conv_shortcut(x) (1×1)   if C_in != C_out
y        = skip + residual
```

Two GroupNorms (Chapter 15, 32 groups, FP32-accumulated stats), two SiLUs, two 3×3 convs (Chapter 19), and a residual add. The `conv_shortcut` is a 1×1 conv that appears only when a block changes channel count (e.g. 512→256), projecting the skip path to match. Everything is BF16 with FP32 accumulation inside GroupNorm and the cuDNN GEMM. This one primitive, instantiated at various channel counts, fills the mid-block and all four up-blocks.

20.3 The mid-block spatial attention

At the lowest resolution (deepest, 512 channels) the decoder has a **self-attention over spatial positions** — the same attention math as the transformer, but here the "tokens" are the H× W spatial locations and the "channels" are the head di mension. It is **1 head, head-dim 512** (`vae_attn.h`): a GroupNorm, then `to_q`, `to_k`, `to_v` projections, the attention, and a `to_out.0` projection, added back as a residual.

This is *why the Part-IV attention kernel was written to also serve D=512* (Chapter 16–17). At the VAE's resolutions the sequence length S=H× W can be huge — **16384** at the mid-block for 1024px output — so this attention is the S=163 84, D=512 shape in the benchmarks. It uses the **warp-per-q uery flash kernel** (not the D=128 mma kernel, whose tensor-core tiling doesn't fit D=512's smem) — online softmax keeps its smem bounded by the tile, so even S=16384 fits. The mid-block attention lets the decoder model long-range spatial structure that pure local convolutions cannot.

20.4 The assembled architecture

From `vae_decoder.h`, the full decoder (latent `[32, H/8, W/8]` → image `[3, H, W]`):

```
(post_quant_conv 1×1, 32→32)          # §20.5
conv_in            32 → 512  (3×3)
mid_block:    Resnet512 → SelfAttn(1 head, 512) → Resnet512
up_blocks[0]: 3× Resnet512        → Upsample ×2 → conv 512→512
up_blocks[1]: 3× Resnet512        → Upsample ×2 → conv 512→512
up_blocks[2]: Resnet512→256(shortcut), 2× Resnet256 → Upsample ×2 → conv 256→256
up_blocks[3]: Resnet256→128(shortcut), 2× Resnet128                (no upsample)
conv_norm_out (GroupNorm32) → SiLU → conv_out 128 → 3  (3×3)
image [3, H, W]
```

The three `Upsample ×2` stages (in up-blocks 0, 1, 2) give the 8× total; up-block 3 has no upsample (the spatial size is already H, W after three doublings from H/8). Channels taper 512 → 512 → 512 → 256 → 128 → 3 as resolution rises — the usual decoder shape (more channels where the map is small, fewer where it is large). `Upsample` is the nearest-neighbor 2× kernel (Chapter 15) followed by a 3×3 conv.

20.5 The latent/decoder boundary (recap from Chapter 03)

Two transforms at the input boundary are easy to miss and were real bugs:

- `post_quant_conv` — a 1×1 conv (32→32, per-pixel channel remix + bias) that diffusers runs *before* the decoder body (`autoencoder_kl_flux2._decode`: `z = post_quant_conv(z); decoder(z)`). It is a **sibling** of the `decoder.` prefix, not under it, so a decoder starting at `conv_in` skips it — and skipping it turned sharp images to "mush." It is now folded into `VAEDecoder` (`Config::post_quant_conv_name`), applied at the top of `forward`, and skipped gracefully if absent.
- **The batch-norm de-normalization** — applied to the 128-channel *patch* latent *before* `unpatchify` (so, in `generate.cu`, not inside the decoder): z· √(Var+ε)+mean, inverting the encoder's standardization. Skipping it caused the 2×2 patch-grid checkerboard. (Full treatment: Chapter 03; war story: Appendix B.)

20.6 The workspace strategy

The decoder threads activations through a small number of reused device buffers. The classic trick is **ping-pong**: two buffers, each conv/resnet reads one and writes the other, swapping each step, plus scratch for the upsample and the per-sub-component working set (the cuDNN plan workspace, the attention scratch). The arena is sized to the **maximum** over stages and reused, since a stage's intermediates die at its boundary. At 128px the total is ~176 MiB; at 1024px it is a few GiB (dominated by the cuDNN conv workspace and the large early feature maps), well within unified memory.

20.7 Performance and the decisive correctness test

With the cuDNN graph-API convs (Chapter 19), the decoder is fast: **128px forward ~10 ms** (was 314 ms), **1024px VAE decode ~1.35 s** (was 18.7 s) — no longer a bottleneck.

Correctness is established two ways:

- **Component tests:** `test_vae_resnet`, `test_vae_attn`, `test_groupnorm`, `test_upsample`, `test_conv2d` each check a piece against an FP32 reference.
- **The decisive end-to-end test:** feed diffusers' **own ground-truth latent** (dumped via `tools/diffusers_latent_dump.py`, `output_type="latent"`) through *our* decoder and compare to diffusers' decoded image. Result: a **sharp, photoreal cat matching diffusers**. This is the test that proved the decoder correct *and* isolated the earlier "mush" to the missing `post_quant_conv` — because feeding a known-good latent removed the transformer trajectory from suspicion, localizing the fault to the VAE. (`tests/test_vae_decoder.cu`; Chapter 25 for the methodology.)

The pattern (Chapter 25): to debug a multi-stage pipeline, **inj**

ect a known-good intermediate from a reference at the stage boundary. Feeding diffusers' latent into our decoder split "is the latent wrong?" from "is the decoder wrong?" in one experiment.

20.8 Where this lives in the code

Concept	Code
the decoder	vae_decoder.{h,cu} (VAEDecoder)
resnet primitive	vae_resnet.{h,cu} (ResnetBlock)
mid-block attention (D=512)	vae_attn.{h,cu} → attention.cu (flash, D=512)
convs (graph API)	conv2d.{h,cu} (Chapter 19)
group norm / SiLU / upsample	kernels/groupnorm.*, silu_*, upsample.*
post_quant_conv	VAEDecoder::Config::post_quant_conv_name
bn de-norm	generate.cu (on the patch latent, pre-unpatchify)
tests	tests/test_vae_decoder.cu (+ component tests)

20.9 Summary and what to carry forward

• The decoder expands the [32,H/8,W/8] latent to [3,H,W] via **ResnetBlocks** (GroupNorm→SiLU→conv ×2 + shortcut), **three 2× upsamples** (=8×), a **mid-block spatial self-attention** (1 head, D=512, reusing the Part-IV flash kernel at S up to 16384), and a final GroupNorm→SiLU→conv to 3 channels.

• Two boundary transforms — `post_quant_conv` (folded in) and the **bn de-norm** (in `generate.cu`) — are essential and were the "mush" and "patch-grid" bugs.

• A **ping-pong workspace** sized to the max stage keeps memory modest.

• With graph-API convs, **decode is ~1.35 s at 1024px**; correctness was nailed by decoding **diffusers' own latent** — the inject-a-known-good-intermediate technique.

That completes Part V. Part VI covers the text front-end: the Qwen3 encoder implementation (Chapter 21) and the native BPE tokenizer (Chapter 22) that made the pipeline self-contained.

Exercises

1. **Channel/resolution taper.** Explain why decoders use *more* channels at low resolution and *fewer* at high, and trace the 512→256→128→3 taper against the three upsamples.

2. **Why attention in a conv net.** What can the mid-block self-attention model that stacked 3×3 convs cannot, and why is it placed at the *lowest* resolution (think S=H· W cost)?

3. **The boundary bugs.** Describe how `post_quant_conv` (missing) and the bn de-norm (missing) each corrupt the image differently (mush vs grid), and why feeding diffusers' latent localized the former.

4. **Workspace.** Sketch a ping-pong schedule for `conv_in → resnet → attn → resnet` and argue why two buffers + scratch suffice despite the channel changes.

Next: Chapter 21 — The Qwen3 encoder, opening Part VI.

Chapter 21 — The Qwen3 Encoder

Goal of this chapter: implement the text encoder whose *role* we covered in Chapter 05 — the 8B Qwen3 LLM run as a single forward pass to produce the `[512 × 12288]` conditioning. We cover the Qwen3-8B architecture (grouped-query attentio n, per-head q/k-norm, its own RoPE), the layer and stack implementation, the capture-layer mechanism, and the validation against HuggingFace. Anchored to `qwen_encoder.{h, cu}`, `qwen_layer.*`, `qwen_attention.*`, `embed_lookup.*`.

Prerequisites: Chapters 05 (the role), 06 (RoPE), 08 (attention/QK-norm), 13 (Linear), 15 (RMSNorm/SwiGLU).

21.1 Qwen3-8B at a glance

From `text_encoder/config.json`:

param	value
layers	36
hidden	4096
query heads	32
kv heads	8 (grouped-query attention, group size 4)
head_dim	128
FFN	SwiGLU, intermediate 12288
RMSNorm eps	1e-6
rope_theta	1e6 (note: ≠ the transformer's 2000)
attention_bias	false (no biases on q/k/v/o)
vocab	151936

It is a standard decoder-only LLM (the LLaMA/Qwen lineage): token embedding, then 36 identical transformer layers, then a final RMSNorm. We run it as an **encoder** — one forward pass over the padded prompt, capturing hidden states (Chapter 05), no generation. Three things distinguish it from the FLUX transformer of Part II and are worth care: **grouped-query attention**, **per-head q/k-norm**, and its **own RoPE c onvention** ($\theta=10^6$, half-rotation).

21.2 Grouped-query attention (GQA)

Standard multi-head attention has one K and one V per query head. **GQA** shares each K/V head across a *group* of query heads: Qwen3 has 32 query heads but only **8 KV heads**, so each KV head serves **4** query heads (group size 32/8 = 4).

Why: K and V are the memory-heavy operands (in autoregressive decoding they form the KV cache; here they are the bandwidth cost of the projections and the attention re ads). Cutting KV heads 4× shrinks the K/V projection weights and the K/V bytes streamed in attention (Chapter 11 — attention is memory-bound), for a small quality cost. The query projection stays full-width (32 heads); only K and V are narrowed.

In the kernel (`qwen_attention.cu`), each query head h attends to the K/V of head h/4:

```
query heads:  0  1  2  3 | 4  5  6  7 | ... | 28 29 30 31
kv heads:     └──  0  ──┘ └──  1  ──┘  ...  └──   7   ──┘
head h (query) uses K,V from kv-head (h / 4)
```

The attention is **causal** (decoder-only): query position i attends only to keys 0..i (Chapter 05 — later positions carry the most context, which is why the capture layers matter). `tests/test_qwen_attention.cu` validates the GQA causal kernel at the Qwen3 shape (B=1, S=128, H_q=32, H_kv=8, D=128) against an FP32 host reference (max err 0.001).

21.3 A Qwen3 layer

`qwen_layer.cu` implements one of the 36 layers (pre-norm transformer with SwiGLU):

```
# attention sub-layer
h    = RMSNorm(x, in_norm_gain)                 # eps 1e-6
q    = Linear(h, Wq) → [S, 32·128]              # no bias
k    = Linear(h, Wk) → [S,  8·128]              # GQA: 8 kv heads
v    = Linear(h, Wv) → [S,  8·128]
q    = RMSNorm(q, q_norm_gain) per head         # per-head q-norm  ┐ BEFORE RoPE
k    = RMSNorm(k, k_norm_gain) per head         # per-head k-norm  ┘ (Qwen3-specific)
```

```
q,k  = RoPE(q, k)                                  # half-rotation, θ=1e6
a    = GQA_causal_attention(q, k, v)               # head h → kv head h/4
x    = x + Linear(a, Wo)                           # residual, no bias
# MLP sub-layer
h    = RMSNorm(x, post_norm_gain)
x    = x + SwiGLU_MLP(h)                            # gate/up/down, intermediate 12288
```

Notes:

- **Per-head q/k-norm before RoPE.** Like the FLUX blocks (Chapter 08) Qwen3 RMS-normalizes each head's Q and K — but with its *own* learned gains, and applied **before** RoPE. This ordering (norm, then rotate) is part of the Qwen3 spec; the same `rmsnorm_bf16` kernel is reused with `batch_rows = S · n_heads`.
- **RoPE: half-rotation, θ=10⁶.** Qwen3 uses the **LLaMA-style (i, i+D/2) pairing** (`rope.cu`, Chapter 06) — *not* the interleaved (2k,2k+1) of the FLUX transformer — and base 10^6. Using the wrong pairing or base silently corrupts the conditioning. (This is exactly why the codebase keeps both RoPE kernels.)
- **No biases.** `attention_bias=false`: the q/k/v/o projections have no bias terms.
- **SwiGLU MLP.** Same gated structure as the FLUX MLP (Chapter 15), intermediate 12288.

21.4 The encoder stack and capture

`qwen_encoder.cu` assembles the full model:

```
token_ids [512] → embed_lookup(embed_tokens)  → [512, 4096]   (BF16 gather, Chapter 15)
  → Layer 0 → Layer 1 → ... → Layer 35
       with hidden states captured at layers {8, 17, 26}        (Chapter 05)
  → (final RMSNorm — used for the standard LM output, not needed for capture)
capture: concat( h_8, h_17, h_26 )  → [512, 12288]               → ContextEmbedder
```

The **capture mechanism** is the encoder's whole purpose: `Config::capture_layers` (`= {8, 17, 26}`) records which layer *outputs* to keep, and the encoder concatenates those three `[512, 4096]` tensors along the channel axis into the `[512, 12288]` conditioning (Chapter 05's 3×4096). The capture indices `{8,17,26`

`}` are the corrected HF off-by-one of diffusers' `{9,18,27}` (Chapter 05). `embed_lookup` is a simple BF16 row-gather from the (BF16-kept) `embed_tokens` table; `lm_head` is unused (we are an encoder, not a generator).

21.5 Quantization and the outliers

The encoder's weights are NVFP4 in the F2K file (`qwen3_f2k/`, 4 shards, 4.8 GiB), **except** `embed_tokens` which is kept BF16 (a `--keep-bf16` flag in `f2k_convert`, Chapter 12/14) — embedding tables tolerate quantization poorly and are looked up, not mat mul'd. The NVFP4 quant quality across shards is the same as the transformer (worst `max_err 0.144`). Recall the **outlier activations** (~22528) from Chapter 05: they appear in the late layers, and the per-microblock scales handle them (Chapter 12/25) — the encoder's conditioning is correct despite them.

21.6 Validation against HuggingFace

The encoder is checked against the real HF Qwen3-8B, not just internally:

- `tools/qwen3_golden.py` runs HF Qwen3 on a fixed prompt and dumps per-layer hidden states; `tests/test_qwen_golden.cu` runs *our* encoder and compares.
- Result: **cosine similarity ≥ 0.974 from layer 0 through 34**, magnitude ratios within ~25%. The dominant error source is NVFP4 quantization (worst per-tensor `max_err 0.14`), not an implementation bug.
- A subtle HF gotcha was caught here: `output_hidden_states=True` returns `n_layers+1` states — `hs[0]` is the embedding, `hs[i]` is the output of layer i-1, and `hs[N]` is *post-final-norm*. Comparing our layer-N-1 output against `hs[N]` (post-norm) drops cos to ~0.69 from the gamma rotation; comparing against `hs[N-1]` gives ≥0.974. (Same off-by-one family as the

capture indices.)

Performance: **4 shards load + build in ~10.5 s** (one-time), **~50 ms forward at seq=128**, ~77 MiB workspace. At seq=512 (the real prompt length) it is the ~0.28 s text-encod e of Chapter 00 — negligible next to the denoise loop, and ~350× faster than the PyTorch path it replaced.

21.7 Where this lives in the code

Concept	Code
token embedding gather	kernels/embed_lookup.{h,cu}
GQA causal attention	qwen_attention.{h,cu}
one Qwen3 layer	qwen_layer.{h,cu}
36-layer stack + capture	qwen_encoder.{h,cu}
RoPE (half-rotation, θ=1e6)	kernels/rope.{h,cu} (Chapter 06)
validation vs HF	tools/qwen3_golden.py, tests/test_qwen_golden.cu
in-pipeline use	generate.cu (--prompt/--tokens, capture {8,17,26})

21.8 Summary and what to carry forward

- The text encoder is **Qwen3-8B** run as a single forward pass; we implement embed → **36 layers** → capture.
- **GQA** (32 query / 8 KV heads, group 4) shrinks the memory-heavy K/V; attention is **causal**.
- Each layer: pre-RMSNorm → q/k/v (no bias) → **per-head q/k-norm before RoPE** → **half-rotation RoPE, θ=1e6** (its own convention, *not* the FLUX transformer's) → GQA → o-proj → residual → RMSNorm → SwiGLU.
- Capture layer outputs **{8, 17, 26}** (corrected HF off-by-one) and concat to `[512, 12288]`; `embed_tokens` stays BF16, rest NVFP4.
- Validated vs HF at **cos ≥ 0.974/layer** (NVFP4 noise dominates); ~10.5 s build, ~50 ms forward at seq 128. Mind HF's `hidden_states` off-by-one when validating.

Chapter 22 covers the last non-CUDA piece — the native byte-level BPE tokenizer that feeds this encoder and made the pipeline self-contained.

Exercises

1. **GQA bookkeeping.** With 32 query and 8 KV heads, write the query→kv-head map and compute the parameter and K/V-byte savings vs full 32-head MHA at hidden 4096.

2. **Two RoPEs.** Contrast the encoder's RoPE (half-rotation, θ=1e6) with the FLUX transformer's (interleaved, θ=2000, 4-axis). Why does the codebase keep both kernels, and what breaks if you swap them?

3. **The HF off-by-one (twice).** Explain how the *same* indexing convention causes both the capture-layer shift ({9,18,27}→{8,17,26}) and the golden-comparison trap (`hs[N]` is post-norm).

4. **Encoder vs generator.** List what we *don't* run (sampling, KV cache, lm_head) and why running Qwen3 as an encoder is cheaper than as a chatbot.

Next: Chapter 22 — A native byte-level BPE tokenizer.

Chapter 22 — A Native Byte-Level BPE Tokenizer

> *Goal of this chapter:* implement the one remaining non-CUDA piece — a from-scratch Qwen2/3 byte-level BPE tokenizer in C++ — so that `generate --prompt "…" --out x.png` is a single self-contained call with **no Python**. We cover byte-level BPE, the bytes-to-unicode map, the GPT-2/Qwen pretokenizer regex (and why it must be hand-rolled), the merge algorithm, special-token splitting, the chat template, and the bit-exact validation against HuggingFace. Anchored to `src/common/bpe_tokenizer.{h,cpp}` and `tests/test_tokenizer.cpp`.
>
> *Prerequisites:* Chapters 05 (the chat template, capture role), 21 (the encoder this feeds).

22.1 Why bother (the motivation)

Before this, tokenization ran in Python: `tools/encode_prompt.py` (or, worse, the full `diffusers_prompt_embeds.py` which loaded all 15 GB of Qwen3 in PyTorch just to produce embeddings — ~98 s with the GPU idle, Chapter 25). The encoder itself is native CUDA (Chapter 21); the tokenizer was the last Python dependency between a prompt and a PNG. Porting it makes the binary self-contained:

```
./generate --prompt "Rocket on a boat" --res 1024 --precision fp8 --out rocket.png
#  100% C++/CUDA: BpeTokenizer → Qwen3 encoder → transformer → VAE → PNG
```

Tokenization is ~2 s (mostly process/encoder-independent), versus 98 s for the PyTorch embeds path — and there is no second runtime to install or keep in sync.

22.2 What a byte-level BPE tokenizer is

BPE (byte-pair encoding) builds a vocabulary by repeatedly

merging the most frequent adjacent symbol pair, learned offline. At inference, tokenizing means greedily applying those merges to split text into known subword tokens. **Byte-level** BPE (GPT-2's variant, which Qwen uses) operates on the raw **UTF-8 bytes** of the text, so *any* string is representable (no unknown tokens) — every byte maps to a base symbol, and merges build up from there.

The pipeline for a piece of text:

```
text → NFC normalize → pretokenize (regex) → for each chunk:
       bytes → byte-to-unicode map → BPE merge → vocab lookup → token ids
```

Plus a wrapper that splits out **special tokens** and applies the **chat template**. The five files needed are in the HF tokenizer dir: `vocab.json` (token→id, 151643 entries), `merges.txt` (151387 ranked pairs), `added_tokens.json` (26 special tokens), `tokenizer_config.json`, `chat_template.jinja`.

22.3 The bytes-to-unicode map

The vocabulary's keys are not raw bytes — they are strings of *printable* Unicode characters, because GPT-2's BPE was defined over a reversible byte→char map that avoids control characters and spaces in the vocab. The map (`bytes_to_unicode`): printable ASCII bytes map to themselves; the rest map to codepoints starting at 256. So byte `0x20` (space) → `Ġ`, byte `0x0A` (newline) → `Ċ`, etc. `bpe_tokenizer.cpp` builds the 256-entry table at load:

```
// printable ranges map to themselves; others to 256, 257, ...
for (b in 0x21..0x7E, 0xA1..0xAC, 0xAE..0xFF) bs.push_back(b), cs.push_back(b);
for (b in 0..255 not already in bs) bs.push_back(b), cs.push_back(256 + n++);
byte_encoder_[bs[i]] = utf8_encode(cs[i]);   // each byte → a UTF-8 string
```

So a pretoken's raw UTF-8 bytes become a sequence of these mapped-char strings (e.g. " dog" → `Ġ d o g`), which is the space BPE operates in and the space `vocab.json` keys live in (e.g. `"Ġdog"`).

22.4 The pretokenizer regex (and why it's hand-rolled)

Before BPE, text is split into chunks by a fixed regex (the GPT-2/Qwen "Split" pretokenizer). The pattern is:

```
(?i:'s|'t|'re|'ve|'m|'ll|'d) | [^\r\n\p{L}\p{N}]?\p{L}+ | \p{N} |
 ?[^\s\p{L}\p{N}]+[\r\n]* | \s*[\r\n]+ | \s+(?!\S) | \s+
```

In words, tried in order: contractions (`'s`, `'re`, …); an optional leading non-letter then a run of letters (" dog"); a **single** digit (so "100" → three tokens); optional space + a run of punctuation; whitespace ending in newlines; trailing whitespace; any whitespace.

The trap: this uses `\p{L}` (Unicode letter) and `\p{N}` (Unicode number), which **C++** `std::regex` **does not support**. Options were a heavy ICU/Unicode-regex dependency or a hand-rolled scanner. The project **hand-rolls** it (`pretokenize_and_bpe`): decode the text to codepoints (tracking byte offsets), and at each position try the seven alternatives *in order*, classifying each codepoint with **ASCII-precise** predicates — `is_letter` = `[a-zA-Z]` or any non-ASCII codepoint, `is_number` = `[0-9]`, `is_space`, `is_nl`. This is **bit-exact for English prompts** (and very close otherwise — e.g. an em-dash classifies as a "letter," but because it is space-delimited it still chunks identically, as the validation confirms). The seven alternatives map directly to seven branches in the scanner; getting their *order* and the greedy/optional semantics right is the whole job.

22.5 The BPE merge

For each pretoken chunk, BPE merges its mapped-char symbols using the ranked merges:

```
// symbols = one mapped-char string per byte of the chunk
while (symbols.size() > 1) {
    find the adjacent pair (a,b) with the LOWEST merge rank   // merge_rank_["a b"]
    if none is mergeable: break
    merge it (a,b) → "ab", removing the second
```

```
}
for each remaining symbol: ids.push_back(vocab_[symbol]);
```

Lowest rank = earliest-learned = most frequent merge. Iterate until no adjacent pair appears in `merges.txt`. Every final symbol is guaranteed to be in `vocab.json` (base bytes are all there), so the lookups never miss. The merges are loaded from `merges.txt` keyed by `"left right"` (the `#version` header line is skipped).

22.6 Special tokens and the chat template

The chat template (Chapter 05) wraps the prompt:

```
<|im_start|>user\n{PROMPT}<|im_end|>\n<|im_start|>assistant\n<think>\n\n</think>\n\n
```

The special tokens (`<|im_start|>`=151644, `<|im_end|>`=151645, `<think>`=151667, `</think>`=151668, ...) are **single ids** that must be matched *literally* and emitted directly, while the gaps between them are byte-level-BPE'd. `encode()` scans the string, at each position checking the 26 added tokens **longest-first**; on a match it flushes the pending text buffer (pretokenize+BPE) and emits the special id; otherwise it accumulates bytes. `encode_for_flux()` builds the template, calls `encode()`, and **right-pads/truncates to 512** with the pad token (`<|endoftext|>`=151643) — exactly matching `tools/encode_prompt.py` and the encoder's expected input (Chapter 21).

(NFC normalization is specified by the tokenizer but is a no-op for ASCII prompts; it is noted as a known limitation for exotic inputs.)

22.7 Bit-exact validation

A tokenizer that is "almost right" silently shifts conditioning. The defense is a **bit-for-bit** comparison against the Python HF tokenizer (`tests/test_tokenizer.cpp`, ctest `tokenizer`): for several prompts, check that `encode_for_flux(prompt, N)`

equals the reference id vector exactly. The reference cases deliberately exercise the hard parts:

```
"a cat on a skateboard"                          → 17 ids  PASS
"My dog Rocket, who is an Akita, ... Oklahoma"    → 34 ids  PASS
"It's 3 cats & 2 dogs (100% good) — don't stop!"  → 33 ids  PASS   (em-dash, contractions,
                                                                    single-digit splits, '&', '!')
+ a padding check (pad to 40 with id 151643)              PASS
ALL OK
```

The third case is the stress test: the em-dash (multi-byte UTF-8), the `'t`/`'don` contraction handling, single-digit tokenization of "100" (→ `1`,`0`,`0`), and `&`/`!` punctuation — all reproduced exactly. Passing these (and the real Rocket prompts) means the native tokenizer is interchangeable with HF's for this model.

22.8 The payoff, end to end

With the tokenizer native, the single-call path works (Chapter 23): tokenize (~2 s, no model load) → Qwen3 encoder (Chapter 21) → transformer → VAE → **PNG via stb**. The slow PyTorch embeds path (98 s, 15 GB, GPU idle) is now optional/legacy; the images are indistinguishable (the encoder is the same NVFP4 Qwen3 either way — the ~8% pixel difference vs the diffusers-embeds path is the NVFP4 encoder vs FP32, a valid alternate composition, Chapter 25). The only non-CUDA logic left in the hot path is this pure-C++ tokenizer, and it is ~2 s of mostly-fixed cost.

22.9 Where this lives in the code

Concept | Code
the tokenizer | src/common/bpe_tokenizer.{h,cpp} (BpeTokenizer)
bytes-to-unicode, vocab/merges load | BpeTokenizer::load
hand-rolled pretokenizer | BpeTokenizer::pretokenize_and_bpe
BPE merge | BpeTokenizer::bpe_segment
special-token split + template | BpeTokenizer::encode, encode_for_flux
validation | tests/test_tokenizer.cpp (ctest tokenizer)
reference dumper | the Python HF tokenizer (one-off, to produce ref vectors)
in-pipeline use | generate.cu --prompt path

22.10 Summary and what to carry forward

• A **byte-level BPE** tokenizer makes any string representable; tokenizing is: NFC → **regex pretokenize** → **bytes-to-unicode** → **BPE merge** → vocab lookup, wrapped by special-token splitting and the chat template.

• The pretokenizer regex uses `\p{L}/\p{N}` which `std::regex` lacks, so it is **hand-rolled** with ASCII-precise classifiers — bit-exact for English prompts.

• BPE merges by repeatedly combining the **lowest-rank** adjacent pair; special tokens are matched **longest-first** and emitted as single ids; the output is **right-padded to 512** with `<|endoftext|>`.

• Validated **bit-for-bit** against HF on contraction/digit/punctuation/em-dash cases, making it a drop-in for the Python tokenizer and completing the **self-contained** `--prompt → PNG` path.

That closes Part VI. Part VII integrates everything: the end-to-end pipeline (Chapter 23), the performance methodology that found every bottleneck (Chapter 24), and the validation methodology that made the port trustworthy (Chapter 25).

Exercises

1. **Byte-level coverage.** Explain why byte-level BPE never needs an `<unk>` token, and what the bytes-to-unicode map buys over operating on raw bytes directly.

2. **Pretokenizer order.** For `" 2"` (space then digit) and `" dog"` (space then letters), trace which of the seven alternatives fires and why the leading space ends up attached (or not). Tie to the reference ids (`2`→ space+`2`; `dog`→ one token).

3. **Why bit-exact matters.** Argue why an "almost correct"

tokenizer is more dangerous than an obviously broken one, with reference to Chapter 05's conditioning role. What would a 1-token shift do?

4. **The hand-roll trade.** What does the ASCII-precise pretokenizer get wrong for non-ASCII punctuation, and why does the em-dash test still pass? When would you need a full Unicode-property implementation?

Next: Chapter 23 — The end-to-end pipeline, opening Part VII.

Chapter 23 — The End-to-End Pipeline

Goal of this chapter: read `generate.cu` start to finish and see how every component of Parts I–VI is orchestrated into one call from a prompt to a PNG. We trace argument parsing, model construction, the three conditioning paths, latent initiali zation, the denoise loop, the latent→pixel boundary, and image output — with the timing of each stage. Anchored to `to ols/generate.cu`.

Prerequisites: all prior chapters (this is the integration).

23.1 The shape of generate

`generate` is the conductor; it owns no math, only orchestration. Its job:

```
parse args → load+build models → get conditioning → init noise →
  denoise loop (call transformer ×N) → latent→pixels (VAE) → write image
```

The single self-contained invocation (Chapters 22, 0):

```
./generate --prompt "a cat on a skateboard" --res 1024 --precision fp8 \
           --steps 4 --seed 0xCAFEBABE --out cat.png
```

Flags: `--prompt`/`--tokens`/`--embeds` (the three conditioning entry points, §23.3), `--res` (output resolution), `--precision {nvfp4,fp8}` (Chapter 12), `--steps`, `--seed`, `--out` (`.png`→PNG via stb, else PPM, Chapter 22/15), plus `--decode_latent` for VAE-isolation debugging.

23.2 Construction

```
1. derive shapes from --res:  H_LAT = W_LAT = res/8,  patch grid = res/16,
       seq_img = (res/16)²  (validated: res%16==0 && seq_img%128==0)   ← the M%128 chain (Ch 13)
2. load transformer F2K shards (transformer_mxfp8/ for fp8, transformer_f2k/ for nvfp4)
       → TensorRouter → FluxTransformer ctor  (~5.3 s, 124 Linears repacked; Ch 10/13/14)
3. load VAE F2K → VAEDecoder ctor  (~0.5 s; Ch 20)
4. if --prompt/--tokens: load Qwen3 F2K (4 shards) → QwenEncoder ctor
       (~10.5 s; capture_layers={8,17,26}; Ch 21)
```

```
5. build the RoPE tables (img/txt/combined) from seq_img, seq_txt, H/W_patches (Ch 6/10)
6. allocate device workspaces (transformer ~2.5 GiB + VAE ~3–5 GiB at 1024px; Ch 11)
```

All weights are mmap'd (no device copy; Chapter 14); "construction" cost is the one-time weight repack into the tensor-core layout, not I/O.

23.3 Getting the conditioning (three paths)

```
if --embeds <file>:   load precomputed [512×12288] BF16 directly  → d_txt
elif --prompt "<t>":  BpeTokenizer.encode_for_flux(t, 512)  → ids  → QwenEncoder.forward → d_txt
elif --tokens <file>: read [512] ids from file              → QwenEncoder.forward → d_txt
else:                 random buffer (noise images; smoke only)
```

The `--prompt` path (Chapter 22 tokenizer → Chapter 21 encoder) is the native, self-contained one (~0.28 s text encode). `--tokens` skips the tokenizer; `--embeds` skips the encoder too (feeding diffusers' known-good conditioning — the key debugging lever, Chapter 25). All three produce the same `d_txt [512×12288]` that the ContextEmbedder consumes (Chapter 5/10).

23.4 Latent initialization

```
d_latent ← N(0, 1) sampled with the seeded RNG, shape [seq_img × 128]
```

Standard normal, **not uniform** — the rectified-flow ODE starts at the Gaussian noise endpoint $p_1 = N(0, I)$ (Chapter 02); a uniform init starts off the trained distribution and lands off-manifold (a real early bug, Appendix B). The seed makes runs reproducible and lets you reroll composition (`--seed`).

23.5 The denoise loop

The heart, ~6.5 s of the run at 1024px (Chapter 0):

```
FlowMatchScheduler sched = FlowMatchScheduler::flux2_dynamic(n_steps, seq_img);  // Ch 2
for (int i = 0; i < n_steps; ++i) {
    auto temb = compute_timestep_embedding(sched.t(i) * 1000, 256);   // Ch 2
    mod = ModulationMLP.forward(temb);                                // 17 vectors, Ch 7
    v   = FluxTransformer.forward(d_latent, d_txt, mod, rope_tables); // velocity, Ch 4–10
    axpy_bf16(d_latent, v, sched.dt(i), n);                           // latent += dt·v, Ch 2
}
```

Each iteration is one Euler step of the ODE: build the timestep embedding for this t, expand it into the 17 modulation vectors, evaluate the 9B velocity network, and take the (variable-size, dynamic-shift) step. The transformer forward is the expensive part (Parts III–IV); `axpy` is trivial. After 4 steps `d_latent` is the clean latent.

23.6 The latent→pixel boundary

```
1. bn de-normalize d_latent (the 128-ch patch latent):  z·√(var+ε)+mean   (Ch 3/20)
2. unpatchify:  [seq_img × 128] → [32, H_LAT, W_LAT]                      (Ch 3)
3. VAEDecoder.forward:  [32,H/8,W/8] → [3, H, W]   (applies post_quant_conv; Ch 19/20)
```

The bn de-norm (on the 128-channel patch latent, *before* unpatchify) and `post_quant_conv` (inside the decoder) are the two boundary transforms whose absence caused the patch-grid and mush bugs (Chapter 3/20; Appendix B). Order matters: de-norm → unpatchify → decode.

23.7 Output

```
copy pixels to host → map [-1,1] → [0,255] → write_image(out_path)
       .png → stbi_write_png (RGB8)   else → PPM P6
```

`write_image` dispatches on the extension (Chapter 22/15). Done — a PNG you can open in VS Code.

23.8 The full timing ledger (1024px, FP8, 4 steps)

stage	time	chapter
transformer build	~5.3 s	10, 13, 14
VAE build	~0.5 s	20
Qwen3 build (—prompt/—tokens)	~10.5 s	21
text encode (tokenize + encoder)	~0.28 s	21, 22
denoise loop (4 steps)	~6.5 s	2, 4—18
VAE decode	~1.3 s	19, 20
total image (excl. builds)	~8 s	—

Builds are one-time per process; an interactive UI (ImGui, the project's other half) amortizes them across many images. The

per-image cost is dominated by the denoise loop, and within it the GEMMs (Part III) and attention (Part IV) — which is why those got the optimization effort. Compare to the project's start: ~133 s/image before the attention (2.87×) and cuDNN-conv (13.8×) work; ~30 s after, and ~8 s/image once the encoder/tokenizer went native and builds are amortized.

23.9 Isolation modes built into the conductor

`generate`'s flags double as debugging instruments — the conductor is built to let you cut the pipeline at any stage and inject a known-good piece (Chapter 25):

- `--embeds` — bypass tokenizer+encoder (test the transformer alone).
- `--tokens` — bypass the tokenizer (test the encoder).
- `--decode_latent <file>` — bypass the transformer entirely, decode a given latent (test the VAE alone — this is how the "mush" bug was localized).
- `--seed` — vary only the noise (isolate latent vs prompt influence).
- `--steps` — vary the sampler depth (distinguish undercooking from structural bugs).

This is not incidental; the ability to inject a reference intermediate at each boundary is what made the multi-stage port debuggable (Chapter 25).

23.10 Summary and what to carry forward

- `generate.cu` is pure **orchestration**: parse → build (mmap, repack) → condition (one of 3 paths) → init N(0, I) → **denoise loop** (Euler steps calling the transformer) → **bn de-norm → unpatchify → VAE** → write image.
- The **denoise loop** (~6.5 s) dominates the per-image

cost; builds (~16 s) are one-time and amortized by a UI.

• The **three conditioning paths** and `--decode_latent`/`--seed`/`--steps` are built-in **isolation modes** for debugging (Chapter 25).

• The two **latent-boundary transforms** (bn de-norm, post_quant_conv) and the **Gaussian init** are easy-to-miss correctness requirements (Appendix B).

Chapter 24 steps back to the *method*: how every bottleneck in that ledger was actually found and fixed.

Exercises

1. **Trace a run.** For `--prompt "x" --res 512 --precision nvfp4 --steps 4`, list every stage, its shape, and which model dir it loads. What is `seq_img`?

2. **Why Gaussian.** Explain (Chapter 2) why the latent must be N(0, I) and what a uniform init does to the ODE trajectory.

3. **Amortization.** Given the build vs per-image times, compute the break-even image count where native build cost is negligible, and argue why an interactive UI changes the optimization priorities.

4. **Isolation.** You get a wrong image. Design a sequence of `--embeds`, `--decode_latent`, `--seed` runs that localizes the fault to tokenizer, encoder, transformer, or VAE.

Next: Chapter 24 — Performance methodology.

Chapter 24 — Performance Methodology

Goal of this chapter: generalize the optimization lessons scattered through the course into a reusable method. We collect the project's actual wins, extract the recurring discipline (measure the ceiling before pulling a lever), name the instruments, and state the principles that let a 9B image model go from ~133 s to ~8 s per image on the GB10.

Prerequisites: Chapters 11 (roofline), 17 (the attention journey), 19 (the cuDNN story).

24.1 The wins, collected

Four large optimizations, each from a different cause:

optimization	result	the actual lever	chapter
attention kernel	56.5 → 19.7 ms (2.87×)	occupancy diagnosis → cut smem → mma+ldmatrix	17
VAE convolutions	18.7 → 1.35 s (13.8×)	right cuDNN version + graph API + NHWC	19
model build	76 → 5.3 s (14×)	quantize once, store on disk (on-disk MXFP8)	14
text encode	98 → 0.28 s (~350×)	native encoder+tokenizer vs PyTorch load	21, 22

Plus the architectural/quantization wins baked in from the start: NVFP4/MXFP8 weights (Chapter 12), fused single-stream GEMMs (Chapter 9), unified-memory mmap (Chapter 14). Together: **~133 s → ~30 s → ~8 s/image** (with builds amortized).

Notice the levers are all *different*: an occupancy limit, a dependency misconfiguration, a redundant recomputation, and a wrong runtime entirely. There is no single trick — each required *diagnosing the specific cause*.

Part VIII costs, for reference. Image conditioning (Chapter

27) adds a VAE encode: ~0.1 s at 512px, dominated by the same convs as decode. The live-preview stream (Chapter 28) decodes ~4 low-res JPEGs per run at ~40 KB each — a small, bounded tax for watching the image form. The one *expensive* mode is **2048px (true 2K)**: ~80–90 s, because the denoise is $O(S^2)$ at sequence length ~16 896 (≈14.5 s per step) plus ~15 s each for VAE encode and decode. That is the honest state — 2K is *correct but unoptimized*; it is the clearest remaining target, and the same roofline discipline (the attention is again the bandwidth-bound bottleneck) applies.

24.2 The core discipline: measure the ceiling first

The throughline, stated once: **before optimizing, determine which ceiling a kernel is under** — compute, memory, occupancy, or "wrong kernel/path entirely" — because the lever that helps one is useless (or harmful) for another.

The canonical example (Chapter 17): the WMMA attention at 97.7 GB/s looked like it needed "more K/V reuse / bigger tiles." But 97.7 GB/s is only 36% of peak *and* the TFLOP/s was low — so it was neither memory- nor compute-bound; it was **occupancy-bound** (1 CTA/SM from 93 KiB smem). The correct lever was the *opposite* of the intuitive one: **less** shared memory, to fit more CTAs. Chasing "bigger tiles" would have been wasted effort against a bottleneck that wasn't there.

The dual example (Chapter 19): the VAE convs at ~510 ms weren't an *algorithm* problem at all — the math was correct — they were dispatching to a slow kernel because of a stale library link and the legacy API. No amount of kernel tuning would have helped; the fix was the *dispatch path*. Only a comparison to a reference (PyTorch's 3 ms) revealed that the

ceiling was "you're not on the fast path."

> **Principle.** Optimization without a confirmed bottleneck is guessing. The most expensive mistakes are *plausible* levers aimed at the wrong ceiling.

24.3 The instruments

The project used a small, blunt toolkit — no exotic profilers required:

- **The roofline arithmetic** (Chapter 11). Compute I=FLOP/byte for a kernel and compare to I_ridge=P_max/B_max. This *predicts* memory- vs compute-bound before you run anything, and tells you which lever class applies.
- **Microbenchmarks with a bandwidth model.** `tools/benchmark_attention.cu` prints, per shape: latency, achieved **GB/s** (against the 273 peak), **TFLOP/s**, the K/V re-read HBM model, and — via probes — the achieved **CTAs/SM**. Those four numbers diagnosed the occupancy limit directly.
- `cudaOccupancyMaxActiveBlocksPerMultiprocessor` + smem arithmetic. Confirms CTAs/SM analytically (93 KiB → 1, 43 KiB → 2) so you know the occupancy lever's payoff *before* writing the kernel.
- **A reference implementation for the path.** PyTorch's conv time (3 ms) was the tell that 510 ms meant "wrong kernel," not "hard problem." Always have a known-good number for what *should* be achievable.
- `nvtop`**/the GPU utilization.** The 98 s text-encode showed the **GPU idle** — immediately reframing it as an I/O/load problem (PyTorch streaming 15 GB), not a compute one. A glance at utilization is a free first diagnostic.

24.4 The optimization sequence (a

template)

From the attention journey (Chapter 17), generalized to any hot kernel on a bandwidth-bound machine:

1. **Eliminate redundant HBM traffic first.** Algorithm beats micro-optimization: online softmax + tiling (don't re-read K/V) dwarfed any tuning of the wasteful row kernel. *Are you moving bytes you don't need to?*
2. **Use the right compute units.** Tensor cores for matmuls (Chapter 13), the fast cuDNN engines for convs (Chapter 19). *Are you even on the intended hardware path?*
3. **Measure which ceiling you're under** (§24.2). GB/s vs peak, TFLOP/s vs peak, CTAs/SM. *Don't proceed on an unconfirmed bottleneck.*
4. **Pull the matching lever.** Occupancy → cut smem; memory-bound → cut bytes (quantize, fuse); compute-bound → faster instruction (lower precision); wrong path → fix the dispatch.
5. **Validate every step** (Chapter 25). A faster wrong kernel is worthless; check against an independent oracle, bit-exact where possible, and distinguish reordered-FP drift (e.g. 1.8% image diff from summation order) from bugs.
6. **Stop at the ceiling.** The attention kernel hit 279 GB/s ≈ peak — there is nothing left to win because the bytes can't arrive faster. Declare victory at the hardware limit, not when the code stops being abstractly improvable.

24.5 Know when not to optimize

Equally important: the project explicitly *declined* several optimizations because the diagnosis said they wouldn't pay (Chapter 17):

- **Bigger query tiles / persistent CTAs** — target a bandwidth/launch bottleneck the occupancy-bound kernel

didn't have.

• `ldmatrix.x4`**, K/V double-buffering** — micro-opts on a kernel already at peak HBM bandwidth; can't meaningfully help a memory-bound kernel at the ceiling.

• **A bespoke D=512 VAE-attention kernel** — the VAE decode is dominated by convs, not attention, so the flash kernel was good enough; effort went where the time was.

The discipline is to spend optimization effort *proportional to a stage's share of the wall clock, and only on a confirmed bottleneck.* The denoise loop got the deepest work because it is the per-image cost; the builds got the on-disk-quant win because they were 76 s; the text encode got a rewrite because it was 98 s of idle GPU.

24.6 Bandwidth as the design compass

Underneath every win is Chapter 11's thesis. On a machine with ~920 FLOP/byte of headroom, the recurring questions are: *how many bytes does this move, and can it move fewer?* That single compass explains quantization (fewer weight bytes), fused GEMMs (read the activation once), flash attention (don't re-read K/V), unified-memory mmap (no transfer), and the on-disk-quant build win (don't recompute). Even the compute-bound big GEMMs are helped by lower precision, which raises the *FLOP ceiling* on the same roofline. When unsure what to do, the answer is usually "move fewer bytes, or move them on a faster instruction."

24.7 Summary and what to carry forward

• Four big wins, **four different levers**: occupancy (attention), dispatch path (cuDNN convs), redundant recompute (on-disk quant build), wrong runtime (native enco der/tokenizer). **No single trick.**

• **Measure the ceiling before optimizing** —

compute/memory/occupancy/wrong-path — because the matching lever differs and the intuitive one is often wrong (attention's "bigger tiles," the conv's "tune the kernel").

• The toolkit is blunt and sufficient: **roofline arithmetic, a microbenchmark with a bandwidth model,** `cudaOccupancyMaxActiveBlocks`**, a reference number, and a glance at GPU utilization.**

• Sequence: eliminate redundant traffic → right compute units → **measure** → pull the matching lever → validate → **stop at the ceiling**. And spend effort proportional to wall-clock share, declining opts the diagnosis says won't pay.

• The compass is **bandwidth**: move fewer bytes, or on a faster instruction.

Chapter 25 covers the other half of trustworthy engineering — how correctness was established and how the hardest bugs were found.

Exercises

1. **Diagnose three.** For (a) a kernel at 36% of peak BW with low TFLOP/s, (b) a kernel at 99% of peak BW, (c) a correct kernel 170× slower than a reference — name the ceiling and the lever for each.

2. **Predict before measuring.** Using the roofline, predict whether a 256px (M=256) vs 1024px (M=4608) `Linear` is memory- or compute-bound (Chapter 11), and say which optimization helps each.

3. **The idle-GPU tell.** Explain why `nvtop` showing an idle GPU during the 98 s text-encode immediately rules out "the encoder kernel is slow" and points at I/O.

4. **When to stop.** Argue why hitting 279 GB/s on the attention kernel means further kernel work is wasted, and what would have to change about the *problem* to make more

speed possible.

Next: Chapter 25 — Validation methodology.

Chapter 25 — Validation Methodology

Goal of this chapter: how correctness was established for a from-scratch port of a 9B model, and how the hardest bugs were found. We cover the test hierarchy (FP32 component references, bit-exact wiring tests, reference-pipeline comparison), the diffusers/HF reference tooling, the master debugging technique (inject a known-good intermediate), the metrics (cosine similarity and its thresholds), and the two defi ning debugging sagas — the text-attenuation hunt and the VAE bugs.

Prerequisites: the whole course; the bugs are cross-referenced to Appendix B.

25.1 Why validation is the hard part

Porting a model is not mostly about writing kernels — it is about *proving they compute the same function as the reference*, when the reference is a 9B network whose correct output is a specific image you cannot eyeball for bit-errors. A single swapped slice, off-by-one layer, or wrong RoPE pairing produces output that is **plausible but wrong** — coherent images that ignore the prompt, or subtly degraded ones. The defenses are a layered test discipline and a debugging method built into the tool (Chapter 23's isolation flags).

25.2 The test hierarchy

Three levels, weakest-to-strongest guarantee:

1. Component tests vs an FP32 host reference. Every kernel and module has a `tests/test_*.cu` that runs it on random data and compares to a CPU FP32 computation by **co**

sine similarity and relative-L2. Thresholds reflect the precision: BF16 elementwise kernels hit `max_err ≈ 0.004` (rounding); the NVFP4 `Linear` hits cos 0.989; attention hits **cos 1.00000** (BF16 noise only). These catch *arithmetic* errors.

2. Bit-exact wiring tests (the zero-gate trick). AdaLN-Zero (Chapter 7) makes a block the *exact identity* when its gates are zero. `test_double_stream_block` and `test_single_stream_block` run a block with zeroed gates and assert the output equals the input **bit-for-bit** (`max_err = 0`). This proves the *entire wiring* — every norm, projection, RoPE, attention, split, residual connects correctly — because any miswire would perturb the supposedly-untouched residual. These catch *connection* errors that tolerance-based tests miss. The same idea validates the stack (`test_mmdit_stack`: bit-exact identity through 8 blocks).

3. Comparison to the reference pipeline (diffusers/HF). The ultimate oracle is the original model. The project compares at multiple boundaries (§25.3) — this is what catches *convention* errors (slice order, layer indices, packing order) that are arithmetically fine but semantically wrong.

> A cautionary note (Appendix B, the CuTe layout bug): **structured-input component tests can pass while the model is wrong.** The NVFP4 weight-layout bug passed all-ones and K-constant tests but corrupted random weights. Component tests must include *random* inputs, and the reference-pipeline comparison is the backstop.

25.3 The reference tooling

A suite of Python tools dumps diffusers/HF intermediates, and C++ comparators check ours against them — at *every* pipeline boundary, which is what makes isolation possible:

tool	dumps / does	checks our

```
tools/qwen3_golden.py + test_qwen_golden.cu | HF Qwen3 per-layer hidden states | encoder (cos ≥ 0.974/layer)
tools/diffusers_prompt_embeds.py (→ --embeds) | diffusers [512×12288] conditioning | transformer with known-good text
tools/cmp_transformer.cu + diffusers_block_dump.py | one transformer forward vs diffusers | transformer (cos 0.963 NVFP4 / 0.999 MXFP8)
tools/diffusers_latent_dump.py (→ --decode_latent) | diffusers ground-truth latent | VAE decoder in isolation
tools/diffusers_vae_encode_dump.py + cmp_vae_encode.cu | diffusers posterior-mean latent | VAE encoder (cos 0.9997, 256–2048px; Ch 26)
tools/diffusers_e2e.py | full diffusers pipeline image | end-to-end ground truth
tools/probe_ctx_emb.cu | per-row cos of ContextEmbedder in/out | text-side Linear ablations
tools/quant_sim.py | PyTorch fake-quant quality study | the FP4-vs-FP8 quality question (Ch 12)
```

25.4 The master technique: inject a known-good intermediate

The single most valuable debugging move, used repeatedly: **at a stage boundary, replace our intermediate with the reference's, and see if the bug persists.** This splits "is stage A wrong?" from "is stage B wrong?" in one experiment. `generate`'s flags (Chapter 23) are built for exactly this:

- `--embeds <diffusers conditioning>` → if the image is *still* wrong, the fault is in the **transformer**, not the encoder/tokenizer.
- `--decode_latent <diffusers latent>` → if the image is *still* mush, the fault is in the **VAE**, not the denoise trajectory.
- `--seed` → vary only the noise to measure latent-vs-prompt influence.

Both defining bugs below were localized by this technique in a single decisive run.

25.5 Saga 1 — the text-attenuation hunt

Symptom. Generated images were coherent but *barely depended on the prompt*: a cat prompt vs a truck prompt

differed by **0.12%** in pixels, while two random seeds (same prompt) differed by ~10%. The latent dominated the prompt ~40:1 — the text path was alive but throttled to a whisper.

Isolation. Feeding diffusers' *own* known-good conditioning via `--embeds` produced the *same* broken image. That single run proved the encoder/tokenizer were **not** the cause — even perfect conditioning failed — localizing the bug to the C++ **transformer's text handling**. (It also ruled out a long list of red herrings: tokenizer, layer indices, chat template — all already correct.)

The hunt. Suspects were investigated and *eliminated with evidence*, not hand-waving: NVFP4 outlier-collapse (disproved — scaling the 22528 outliers down 100× didn't change ContextEmbedder cos or the image, via `probe_ctx_emb`); joint-attention concat order; modulation gates. Several real fixes landed along the way (the {9,18,27} → {8,17,26} off-by-one, the chat template, seq 128→512, the patchify channel order, uniform→Gaussian noise init, 4-axis RoPE incl. the text stream) — each improved things slightly but none restored prompt control.

Root cause. The `ModulationMLP` sliced every `Flux2Modulation` output as `[scale, shift, gate]`, but diffusers emits `[shift, scale, gate]` (Chapter 7). All 32 blocks computed `(1+shift)·x + scale` instead of `(1+scale)·x + shift`, corrupting AdaLN — the *only* channel through which the prompt modulates the network. The fix (swap the slice order; keep `norm_out` scale-first) took prompt influence from **0.12% to 12.5%** (~100×), making it dominant over the latent. *That one ordering change* turned "ignores the prompt" into "renders the prompt."

The lesson. A swapped slice doesn't crash or NaN — it produces plausible output. It was findable only by (a) isolating

the stage with `--embeds`, and (b) being willing to diff *conventio ns* against the reference, not just trust the math.

25.6 Saga 2 — the VAE bugs

The "mush." Even with the transformer fixed, images were soft/mushy. Decisive test: feed diffusers' **ground-truth latent** through our VAE (`--decode_latent`) → still mush. That removed the transformer trajectory from suspicion and localized the fault to the **VAE**. Cause: the missing `post_quant_conv` (a 1×1 conv diffusers runs before the decoder, a *sibling* of the `decoder.` prefix, easily skipped). Folding it in → instant sharp cat (Chapter 20).

The patch-grid. A hard 2×2 checkerboard across the image. It was *structural*, not undercooking — proven by running 28 steps and seeing it unchanged. Cause: the missing **batch-no rm de-normalization** of the patch latent; each patch's four sub-pixels live in different channels with different stats, so skipping the de-norm mis-scaled them into a grid (Chapter 3). Loading `bn.running_{mean,var}` and applying the affine fixed it.

The lesson (both). Reference pipelines hide affine/normalization steps in unobvious places. **Diff at the** *lat ent* boundary, not just the final image — both were localized by comparing/injecting latents, and the "more steps?" test dis tinguished structural bugs from undercooking.

25.7 The metric: cosine similarity (and why)

The recurring metric is **cosine similarity** between our output and the reference, flattened. Why cosine and not, say, mean-abs-error: low-precision (NVFP4/BF16) introduces a roughly *uniform* magnitude perturbation, so the error is mostly **directional**; cosine isolates whether the *structure*

matches independent of a global scale. Thresholds that recur:

• **cos ≈ 1.00000** — bit-exact-up-to-BF16 (attention, conv, kernels). Anything less is a real difference.

• **cos ≥ 0.974** — Qwen3 encoder per layer (NVFP4 quant noise).

• **cos 0.963 (NVFP4) / 0.999 (MXFP8)** — full transformer forward; the NVFP4 gap is *quantization*, not a bug (proved by the MXFP8 number and `quant_sim.py`).

Crucially, the method **distinguishes quantization noise from bugs**: when the NVFP4 transformer hit 0.963, the question was "bug or FP4?" — answered by switching to MXFP8 (0.999) and the fake-quant study (Chapter 12). A bug would not vanish with precision; quantization noise does. Likewise, the 1.8% pixel diff between attention kernels (Chapter 17) was confirmed as reordered-summation BF16 drift — both kernels matched FP32 at cos 1.0 — not a regression.

25.8 Summary and what to carry forward

• Validate in **layers**: FP32 component tests (random inputs!), **bit-exact zero-gate wiring tests**, and **comparison to the reference pipeline** at every boundary.

• Build **isolation** into the tool: inject a known-good intermediate (`--embeds`, `--decode_latent`, `--seed`) to split which stage is wrong in one run.

• **Convention bugs** (slice order, layer indices, packing, RoPE pairing) are plausible-but-wrong and don't crash — find them by **diffing intermediates against the reference**, not by eyeballing images.

• Use **cosine similarity** (directional, scale-invariant) with precision-aware thresholds, and **distinguish quantization noise from bugs** by varying precision.

• The two defining sagas — modulation scale/shift swap

(text attenuation) and the VAE post_quant_conv/bn-de-norm (mush/grid) — were each localized by injecting a reference intermediate and diffing at the latent boundary.

This closes Part VII — the text-to-image pipeline, proven correct. **Part VIII** then goes beyond it: the VAE encoder (Chapter 26) that closes the autoencoder loop, the image-con ditioning techniques it unlocks (Chapter 27 — img2img, inpainting, upscaling), and the serving layer that streams it live (Chapter 28). The appendices follow: a glossary (A), the bug museum (B), and the build & reproduce guide (C).

Exercises

1. **Design the isolation.** Given a wrong image, write the exact sequence of `--embeds`/`--decode_latent`/`--seed` runs (and what each outcome implies) to localize the fault to one of: tokenizer, encoder, transformer, VAE.
2. **Why zero-gate.** Explain why a bit-exact zero-gate identity test catches wiring bugs that a cos>0.99 tolerance test would pass, and name a bug class it *cannot* catch.
3. **Bug or quant?** You see transformer cos 0.95. Describe the two experiments that decide whether it is a bug or quantization noise.
4. **The structured-test trap.** Explain how the CuTe layout bug passed all-ones and K-constant tests, and write the random-input test that catches it (Chapter 13).

Next: Chapter 26 — Assembling the VAE encoder, opening Part VIII.

Chapter 26 — Assembling the VAE Encoder

Goal of this chapter: build the **encoder** half of the VAE — the map from an [3, H, W] image back to a [32, H/8, W/8] latent. The course used the decoder (Chapter 20) for text-to-image; the encoder is what makes *image*-conditioned generation possible (Chapter 27: img2img, inpainting, upscaling). It is a near-exact **mirror** of the decoder, reusing the same `ResnetBlock`, `VAEAttention`, `Conv2d`, GroupNorm and SiLU primitives — the one genuinely new piece is the **stride-2 downsample with diffusers' asymmetric pad**. Anchored to `vae_encoder.{h,cu}`, `tools/cmp_vae_encode.cu`, `tools/diffusers_vae_encode_dump.py`.

Prerequisites: Chapters 03 (latent/VAE role, the bn boundary), 19 (cuDNN conv), 20 (the decoder — this chapter is its reflection), 15 (GroupNorm/SiLU).

26.1 The encoder's job and shape

The decoder of Chapter 20 expands a latent 8× to an image. The encoder runs the same gauntlet backwards: it **contracts** an [3, H, W] image 8× spatially to a latent grid, taking three 2× **downsamples** (2×2×2 = 8×), a self-attention at the lowest resolution, and a final projection. Where the decoder *tapered* channels up as resolution rose (512→…→3), the encoder *grows* them as resolution falls (3→128→256→512), the usual autoencoder hourglass.

One subtlety up front: the encoder's last conv emits **64** channels, not 32. The VAE is variational — the encoder outputs the parameters of a Gaussian *posterior*, 2× the latent channels: 32 for the mean and 32 for the log-variance, concatenated on the channel axis. These 64 channels are the

moments; §26.5 turns them into the 32-channel latent we actually use.

The encoder weights were in `vae.f2k1` all along — under the `encoder.` prefix, plus a sibling `quant_conv.` — but only the `decoder.` half had been wired into code. This chapter wires the other half; no reconversion was needed.

26.2 A mirror built from Chapter 20's primitives

The encoder needs nothing new in the primitive library. It is assembled from exactly the pieces Chapter 20 introduced:

- `ResnetBlock` (`vae_resnet.h`) — GroupNorm→SiLU→conv ×2 + optional `conv_shortcut`, identical to the decoder's. The encoder's first resnet in blocks 1 and 2 changes channel count (128→256, 256→512) and so carries a 1×1 shortcut, exactly as the decoder's channel-changing resnets did.
- `VAEAttention` (`vae_attn.h`) — the 1-head, D=512 spatial self-attention, the same object the decoder's mid-block uses, reusing the Part-IV flash kernel.
- `Conv2d` (`conv2d.h`) — the cuDNN-9 graph-API conv of Chapter 19, here also driven at **stride 2** for the downsamples.
- **GroupNorm / SiLU** kernels (Chapter 15) for `conv_norm_out`.

Because the building blocks are shared and config-driven, `VAEEncoder` is mostly *plumbing*: instantiate the right blocks at the right channel counts and thread the data through a ping-pong workspace (§26.6). The only code that did not already exist is the downsample pad.

26.3 The asymmetric downsample (the one real subtlety)

A downsample is "a 3×3 conv with stride 2" — but *which padding* is a silent contract, and getting it wrong corrupts the latent at every block boundary. diffusers' `Downsample2D` has two modes, selected by the `downsample_padding` the encoder passes:

```
# diffusers Downsample2D.forward, when use_conv and padding == 0:
pad = (0, 1, 0, 1)                          # right + bottom only
hidden = F.pad(hidden, pad)                 # asymmetric, NOT symmetric pad-1
hidden = self.conv(hidden)                  # 3×3, stride 2, padding 0
```

The VAE `Encoder` is built with `downsample_padding=0`, so it takes the asymmetric branch: pad the input by **one column on the right and one row on the bottom only**, then a stride-2 conv with *no* padding. This is *not* the same as a symmetric `padding=1` stride-2 conv — the two differ at the borders, and a symmetric pad would shift the entire feature map by a fraction of a pixel, which the golden test (§26.7) catches immediately.

cuDNN's 2D convolution descriptor only expresses *symmetric* per-dimension padding, so we can't ask it for `(0,1,0,1)` directly. We pre-pad instead, with a tiny kernel:

```
// pad_br_kernel: dst[N,C,H+1,W+1] = src[N,C,H,W], bottom row + right col zero-filled
__global__ void pad_br_kernel(const __nv_bfloat16* src, __nv_bfloat16* dst,
                              int N, int C, int H, int W) {
    const int Hd = H + 1, Wd = W + 1;
    /* … index (n,c,h,w); copy src where h<H && w<W, else 0 … */
}
```

then run the `Conv2d` with `stride=2, padding=0` on the [N,C,H+1,W+1] buffer. The output spatial size is lfloor (H+1-3)/2rfloor + 1 = H/2 for even H — exactly the halving we want. This `pad → stride-2 conv` is the whole downsample.

26.4 The assembled architecture

From `vae_encoder.h`, the full encoder (image `[3, H, W]` → moments `[64, H/8, W/8]`), with `block_out_channels = [128, 256, 512, 512]`, `layers_per_block = 2`:

```
conv_in           3 → 128  (3×3)
down_blocks[0]: 2× Resnet128                       → Downsample 128→128 (½)
down_blocks[1]: Resnet128→256(shortcut), Resnet256 → Downsample 256→256 (¼)
down_blocks[2]: Resnet256→512(shortcut), Resnet512 → Downsample 512→512 (⅛)
down_blocks[3]: 2× Resnet512                                    (no downsample)
mid_block:    Resnet512 → SelfAttn(1 head, 512) → Resnet512
conv_norm_out (GroupNorm32) → SiLU → conv_out 512 → 64  (3×3)
quant_conv         64 → 64  (1×1)
moments [64, H/8, W/8]
```

Read it against the decoder of §20.4 and it is the same diagram reflected: three downsamples (on blocks 0–2) where the decoder had three upsamples; the final block (3) has *no* downsample, just as the decoder's last up-block had no upsample; the mid-block is identical (Resnet → attention → Resnet at 512). `conv_out` projects to **64** (the moments), and a 1×1 `quant_conv` (64→64) follows — the encoder-side sibling of the decoder's `post_quant_conv`, the channel remix diffusers applies before forming the posterior.

26.5 Moments, the mean, and the bn boundary

`VAEEncoder::forward` returns the 64-channel moments. diffusers wraps them in a `DiagonalGaussianDistribution` that splits the channel axis in half:

$$\textit{mean} = \textit{moments}[:32],\ \log\sigma^2 = \textit{moments}[32:].$$

For deterministic, reproducible image conditioning we use the distribution's **mode** — the **mean** — not a sample. And because the channel axis is the outermost (NCHW), the first 32 channels of the [64, H/8, W/8] moments tensor are already a contiguous [32, H/8, W/8] block: the consumer (Chapter 27) just treats the moments pointer as a 32-channel latent.

That mean lives in **VAE latent space**. To feed the transformer it must cross the same boundary the decoder crossed in reverse (§20.5, Chapter 03). Decode applied $z \cdot \sqrt{(Var+\varepsilon)} + mean$ to *de*-standardize; encode applies the **inverse**,

$$z_t f = (z\_vae - bn.mean)/(\sqrt{(bn.var+\varepsilon)}),$$

on the 128-channel *patch* latent after `patchify`. That normalization is part of the *conditioning* path, not the encoder object, so it lives in Chapter 27 — but it is exactly the bn de-norm of Chapter 20 run backwards, with the same `bn.running_{mean,var}` and $\varepsilon=10^{-4}$.

26.6 The workspace strategy

Same idea as the decoder (§20.6): a **ping-pong** pair of device buffers sized to the largest stage, swapped on each block, plus a dedicated **pad buffer** for the `(0,1,0,1)` pre-pad and the shared sub-component scratch. The largest stage is `conv_in`'s output at full resolution, [128, H, W] — the early, high-resolution feature maps dominate, as in any encoder. At 1024px that is a few hundred MiB per buffer; comfortably within unified memory. The arena is allocated once when the `(res, precision)` pipeline is built (Chapter 28) and reused for every encode.

26.7 Correctness: the golden test

The encoder is validated with the same **inject-and-compare** discipline as the decoder (Chapter 25), against diffusers' own `AutoencoderKLFlux2`:

1. `tools/diffusers_vae_encode_dump.py` preprocesses an image to model space [-1,1] and dumps **two** files in the course's `int32 C,H,W + float32` layout: the *exact* preprocessed input pixels, and diffusers' posterior **mean** latent.
2. `tools/cmp_vae_encode.cu` uploads those exact pixels, runs `VAEEncoder`, takes the mean (first 32 of 64 channels), and reports cosine similarity against the golden.

Dumping the preprocessed *pixels* (not just the source image) removes any resize/decode mismatch from the comparison —

both sides see identical input. The result:

Resolution | latent | mid-attn tokens S | cos similarity
256px | [32,32,32] | 1 024 | 0.99975
512px | [32,64,64] | 4 096 | 0.99970
1024px | [32,128,128] | 16 384 | 0.99967
2048px | [32,256,256] | 65 536 | 0.99960

The residual (cos ≈ 10^{-3} off unity) is the expected BF16-vs-FP32 accumulation gap, not a bug — the same magnitude as every other component golden in Chapter 25.

The 2048px row carries a war story. The encoder's **mid-block attention** runs at the latent resolution, so its sequence length is S = (H/8)² — and at 2048px that is exactly **65 536**. The first time this ran it *failed at launch* inside the attention's `to_q` projection, and the failure looked for all the world like "2048 needs too much memory." It was not memory at all: it was a `gridDim.y` limit in `Linear`'s quantize kernel, tripped at pr ecisely M=65536. That is Appendix B's newest entry (B.15), and fixing it is what put the 2048 row in the table — and unblocked native 2K generation everywhere.

26.8 Where this lives in the code

Concept | Code
the encoder | vae_encoder.{h,cu} (VAEEncoder)
asymmetric downsample pad | pad_br_kernel / pad_bottom_right_bf16 (in vae_encoder.cu)
resnet / attention / conv primitives | vae_resnet.*, vae_attn.*, conv2d.* (Chapters 19–20)
moments → mean | first 32 of 64 channels (consumed in serve.cu, Chapter 27)
bn normalization (encode side) | serve.cu img2img path (Chapter 27)
quant_conv | VAEEncoder::Config::quant_conv_name
golden tools | tools/diffusers_vae_encode_dump.py, tools/cmp_vae_encode.cu

26.9 Summary and what to carry forward

• The encoder is the **mirror** of the decoder: `conv_in` → three downsampling blocks → a 512-channel mid-block (Resnet → 1-head D=512 attention → Resnet) → `conv_norm_out` → `conv_ou`

`t` (to **64** = moments) → `quant_conv`. It reuses every Chapter 20 primitive.

• The one new piece is the **downsample**: diffusers' `downsample_padding=0` means an **asymmetric** `(0,1,0,1)` **pad** then a stride-2, padding-0 conv — not a symmetric pad. We pre-pad with `pad_br_kernel` because cuDNN only does symmetric padding.

• The output is **moments**; the deterministic latent is the posterior **mean** (first 32 channels). Crossing into transformer space applies the **inverse** of Chapter 20's bn de-norm.

• Verified against diffusers' encoder at **cos 0.9997** across 256–2048px — the BF16 gap, not a bug. The 2048px case surfaced the `gridDim.y` bug (Appendix B.15).

With both halves of the VAE in hand, the autoencoder loop is closed: we can now go *image* → *latent* → *image*, which is the machinery the next chapter builds on.

Exercises

1. **Symmetric vs asymmetric pad.** For an H× W input, compute the output size and the receptive-field offset of (a) `F.pad((0,1,0,1))` + stride-2 padding-0 conv versus (b) a symmetric padding-1 stride-2 conv. Which border does each favor, and why would (b) shift the whole latent?

2. **Why 64 channels.** Explain the encoder's 2× output width in terms of the variational posterior, and why using the **mean** (mode) rather than a sample is the right choice for reproducible img2img.

3. **The bn round trip.** Write the encode-side normalization as the exact inverse of Chapter 20's decode-side de-normalization, and argue that a sign or ε error here would survive a *visual* check but fail the golden.

4. **Mid-attention cost.** The mid-block attention is

S=(H/8)². Tabulate S for 256/512/1024/2048px and explain why 2048 (and not 1024) was the first to hit the `gridDim.y` ceiling.

Next: Chapter 27 — Image conditioning: img2img, inpainting & upscaling.

Chapter 27 — Image Conditioning: img2img, Inpainting & Upscaling

Goal of this chapter: use the encoder of Chapter 26 to condition generation on an existing **image** instead of pure noise. Three capabilities fall out of one idea — *start the denoise from a noised real latent* — controlled by a single **strength** dial: **img2img** (restyle a whole image), **inpainting** (regenerate only a painted region), and **upscaling** (a hi-res pass). All of it is latent arithmetic around the Chapter 2 flow-matching schedule; no model retraining, no new kernels. Anchored to `tools/serve.cu`.

Prerequisites: Chapters 02 (flow matching, the schedule), 03 (latent/bn boundary), 23 (the txt2img loop), 26 (the encoder).

27.1 The one idea

Text-to-image (Chapter 23) starts the denoise from $x_{t=1}$=boldsymbolvarepsilon~ N(0, I) — pure noise — and integrates the flow-matching ODE down to t=0, a clean latent. The velocity field the transformer predicts transports *noise* to *data* along

$$x_t = (1-t)\, x_0 + t\, boldsymbolvarepsilon,\ t \in [1,0],$$

where t=1 is noise and t=0 is data (Chapter 2). The entire trick of image conditioning is: **don't start at t=1, and don't start from random x_0.** Encode a real image to get its clean latent x_0, jump to some intermediate t_0<1 on the *same* interpolation, x_{t0}=(1-t_0)x_0+$t_0$boldsymbolvarepsilon, and run only the **tail** of the schedule from t_0 to 0. The result is a new image that shares the original's structure (because the low-frequency content survived the partial noising) but is re-rendered toward the prompt (because the model freely

denoises from t_0). How far back you jump — t_0 — *is* the strength dial.

27.2 From image to transformer-space latent

Conditioning on an image means turning its pixels into the exact tensor the transformer's denoise loop expects: a 128-channel patch latent in *transformer* space. That is the en coder of Chapter 26 plus the boundary crossings of Chapter 03, run in order (`serve.cu`, img2img branch):

```
image [3,res,res], model space [-1,1]
  │  VAEEncoder.forward                        (Ch 26)
  ▼
moments [64, res/8, res/8] ── take mean = first 32 channels ──► z_vae [32, res/8, res/8]
  │  patchify (p=2)                            (Ch 03)
  ▼
z_vae tokens [S, 128]   ── bn-normalize  (z − bn.mean)/√(bn.var+ε) ──►  x0 [S, 128]
```

The last step is the **inverse** of Chapter 20's decode-side bn de-normalization (§26.5): decode multiplies by √(var+ε) and adds the mean; encode subtracts the mean and divides. After it, x_0 is the clean latent *in the space the transformer was trained on* — the same space a finished txt2img denoise lands in. We keep x_0 (and a fixed noise tensor boldsymbolvarepsilon) on the host, because the inpainting loop (§27.4) will need them every step.

27.3 Strength as a partial schedule

The scheduler (`FlowMatchScheduler`, Chapter 2) precomputes the t values for `steps` Euler steps, dynamically shifted, from t_0=1 down to 0. For img2img we keep the *same* schedule but enter it partway:

```
n_run  = round(steps · strength)              # how many tail steps to run
i_start = steps − n_run                        # skip the high-noise head
t0      = sched.t(i_start)                     # the entry timestep
x       = (1 − t0)·x0 + t0·ε                   # noise the latent to t0
for i in [i_start, steps):  x += dt(i) · model(x, t_i)     # run the tail
```

A few consequences worth internalizing:

• `strength = 1` gives `i_start = 0`, and the dynamic-shift schedule has t(0)=1 exactly, so x=(1-1)x₀+1·boldsymbolvarepsilon=boldsymbolvarepsilon — pure noise. Image conditioning at full strength **degenerates to txt2img**; the init image is forgotten. That is the correct, continuous limit.

• **Lower strength → fewer steps actually run** (`n_run = round(steps·strength)`). At `strength 0.3` with 8 steps only ~2 steps execute, which can look undercooked — so the UI advises raising `steps` when using low strength.

• The dial is **monotone and intuitive**: t_0 small ⇒ little noise added ⇒ stays near the original; $t_0 \to 1$ ⇒ near-total regeneration. Measured: a red coastal car → *blue at sunset* was strength 0.7 (same car, new colour/light); 0.3 was near-identical, 0.9 fully re-rendered.

This whole mechanism is ~20 lines around the existing denoise loop. The encoder did the hard part.

27.4 Inpainting: RePaint masked denoise

Inpainting changes *only* a painted region and leaves the rest untouched. The standard diffusion technique — **RePaint** — drops straight onto the flow-matching loop: at every step, **overwrite the kept (unmasked) region with its own known trajectory**, and let the model regenerate only the masked region, conditioned each step on correct surrounding context.

Let m_r∈[0,1] be a per-token mask (1 = regenerate, 0 = keep). The known trajectory of a kept token is just its noised clean latent at the current t, the same interpolation as everywhere else: $\text{known}_t = (1-t)x_0 + t\,\text{boldsymbolvarepsilon}$. The loop becomes:

```
x = (1 − t0)·x0 + t0·ε                          # init (as img2img)
for i in [i_start, steps):
    t = sched.t(i)
    x ← m · x  +  (1 − m) · ((1−t)·x0 + t·ε)     # lock kept region to known-at-t
    x += dt(i) · model(x, t)                      # model regenerates masked region
x ← m · x  +  (1 − m) · x0                        # final lock: kept region = exact x0 (t=0)
```

The kept region is pinned to the original's correct noised value *before the model sees it*, so the network always denoises the masked area against a faithful context; the masked area evolves freely. At t=0 the kept tokens equal x_0 exactly, so decoding reproduces the original there. With a **soft** mask the blend m· x+(1-m)·known feathers the boundary instead of hard-cutting it.

Mechanically this is a host round-trip per step — copy the latent down, blend, copy up — which is negligible beside the transformer forward that dominates each step. We force `strength ≥ 0.6` for inpainting so the masked region gets enough steps to actually fill.

Verified end-to-end: masking the top of a sailboat scene and prompting "stormy sky with lightning" repainted the sky — lightning and all — while the boat and sea below stayed pixel-faithful.

27.5 The mask, in token space

The mask is painted in the browser at image resolution (Chapter 28); the loop needs it at **latent-token** resolution. The token grid is H_P× W_P=(res/16)² — each token is a 2×2 latent patch, i.e. a 16×16 image-pixel block (2× patch × 8× VAE). So the pixel mask is **average-pooled** over 16×16 blocks:

$$m\_r = (1)/(16^2)Σ\_(y,x)∈block(r)\ (mask(y,x))/(255)∈[0,1],$$

and crucially the token index must match `patchify`'s order, r = h_p· W_P + w_p (the channel-slowest packing of Chapter 03 / Appendix B.4). Averaging — rather than thresholding — gives

the **soft** edge weights that feather the blend in §27.4; a token half-covered by the brush gets m_r≈0.5. `load_mask_tokens` in `serve.cu` does exactly this pooling.

27.6 Upscaling: low-strength img2img at higher resolution

Upscaling needs no new mechanism — it is **img2img at low strength and a larger resolution**, the classic "hi-res fix." Take a finished image, run it as the init at **strength ≈ 0.35** but at **2× the resolution**, reusing the source's prompt and seed. The low strength means the structure is preserved (only the high-frequency detail is re-synthesised at the higher native resolution); the larger grid means real new detail, not interpolation.

The resolution ceiling here taught a lesson. Direct upscaling worked to **1536px** but *failed at 2048* — and the failure was inside the VAE encoder's mid-attention at S=65536 tokens, the `gridDim.y` overflow of Appendix B.15, which also blocked native 2048 generation. Once that one-line grid fix landed, 2048 (true 2K) worked everywhere; the encoder golden table (§26.7) gained its 2048 row, and the upscale cap rose to 2048. 2K is *functional but slow* (~80–90 s: the denoise is $O(S^2)$ at sequence length ~16 896, plus ~15 s each for VAE encode and decode) — a correctness win, with speed left as future work.

27.7 Where this lives in the code

Concept | Code
encode → transformer-space x0 | serve.cu img2img branch (encoder + patchify + bn)
strength → partial schedule | serve.cu (n_run, i_start, t0)
inpainting RePaint loop | serve.cu denoise loop (per-step lock) + final lock
pixel mask → token weights | load_mask_tokens (serve.cu)
upscaling | webui/app.py /upscale (strength 0.35, 2× res) — Chapter 28
the encoder | vae_encoder.{h,cu} (Chapter 26)

27.8 Summary and what to carry forward

• All three features are **one idea**: start the flow-matching denoise from a *noised real latent* rather than pure noise, and run only the tail of the schedule.

• **Encode → mean → patchify → bn-normalize** turns an image into a transformer-space x_0 (Chapter 26 + the inverse bn of Chapter 20).

• **Strength** is a partial schedule: `i_start = steps − round(steps·strength)`, $x_{t_0}=(1-t_0)x_0+t_0$boldsymbolvarepsilon. Strength 1 = txt2img; low strength runs fewer steps (raise `steps` to compensate).

• **Inpainting** is RePaint: each step, lock the kept region to its known noised trajectory $(1-t)x_0+t$boldsymbolvarepsilon and let the model fill the masked region in context; a soft, token-pooled mask feathers the seam.

• **Upscaling** is low-strength 2× img2img. True 2K waited on the `gridDim.y` fix (Appendix B.15) and is correct-but-slow.

These are pure inference-time techniques — the model never knew about img2img or inpainting; the flow-matching geometry gave them for free. The final chapter productionizes all of it.

Exercises

1. **The strength limit.** Show algebraically that strength =1 reduces img2img to txt2img, using t(0)=1 of the dynamic-shift schedule. What happens at strength =0, and why does the implementation clamp `n_run ≥ 1`?

2. **RePaint vs naive masking.** A tempting shortcut is to denoise the whole latent freely and paste the original back only at the end. Explain what that loses versus per-step lockin

g, in terms of the context the model attends to.

3. **Soft vs hard masks.** Derive the visible difference at a region boundary between a thresholded ($m \in 0,1$) and an average-pooled ($m \in [0,1]$) mask, and relate it to the 16×16 pixel→token block.

4. **Upscale strength.** Why does upscaling want *low* strength (~0.35) rather than high? What would strength 0.9 do to the original composition, and why is that wrong for an upscaler?

Next: Chapter 28 — Serving: the persistent worker & live preview.

Chapter 28 — Serving: The Persistent Worker & Live Preview

Goal of this chapter: turn the one-shot `generate.cu` of Chapter 23 into something you can actually *use* — a **persistent worker** that loads the models once and serves jobs over a socket, and a small **web UI** (reachable from a phone over Tailscale) that streams the image **forming live** on screen. The interesting engineering is in three places: what stays resident, a line-oriented protocol, and decoding low-resolution previews mid-denoise without disturbing the run. Anchored to `tools/serve.cu`, `tools/webui/app.py`.

Prerequisites: Chapters 23 (the end-to-end pipeline), 26–27 (the encoder and image conditioning the worker also serves).

28.1 The model-load tax

`generate.cu` is correct but loads every model on every run: the Qwen3 encoder (four shards), the transformer, the VAE. On the GB10 that load is ~16 s — and the *actual work* of a 1024px image is ~8 s. Spending 16 s to do 8 s of work, every time, is the dominant cost for interactive use. The fix is the oldest one in serving: **load once, stay up, serve many.** `build/serve` does exactly that, and a same-config image drops from ~25 s (one-shot) to **~8 s** (resident).

28.2 What stays resident, and what gets rebuilt

Not everything can stay fixed: the transformer and VAE are sized for a specific **resolution and precision**. The worker splits state along that seam:

- **Resolution-independent, resident forever:** the

Qwen3 encoder and the BPE tokenizer. They depend only on the prompt, never on image size or quant mode, so they load once (~16 s) and never reload.

• **Keyed by** `(res, precision)`**:** the transformer, the VAE **de coder and encoder** (Chapter 26), and all the device scratch buffers. These are wrapped in a `Pipeline` object that `Worker::e nsure(res, precision)` builds lazily and **caches**. A request at t he same `(res, precision)` reuses it; a change rebuilds it — but only the transformer/VAE part, ~6 s, because Qwen is untouched.

So the first image at a new size costs ~14 s; every subsequent image at that size is ~8 s. This is the same arena-and-reuse discipline as the kernel workspaces (Chapter 20), lifted to whole models.

28.3 A line-oriented protocol

The worker listens on **localhost TCP** and speaks **newline-de limited JSON** — one JSON object per line, the simplest framing that works. A request is a single line:

```
{"prompt":"…","res":1024,"precision":"fp8","steps":8,"seed":777,"out":"/abs/path.png",
 "init_image":"…","mask_image":"…","strength":0.6,"stream":false}
```

`init_image` / `mask_image` / `strength` opt into the image-conditioning paths of Chapter 27; omit them for txt2img. The non-streaming reply is one line:

```
{"ok":true,"elapsed":8.1,"nans":0,"timing":["text encode 277ms","denoise 6.6s (8 steps)","vae 151ms"]}
```

Binding to `127.0.0.1` keeps the GPU server off the network entirely — the only thing exposed to other devices is the web front-end (§28.4), which is a separate, authenticated process. The worker does one job at a time (a single accept loop); concurrency is the web layer's problem, not the GPU's.

28.4 The web front-end

`webui/app.py` is a small Flask app meant to be reached over **Tailscale** from a phone or tablet — single-user session login, dark mobile UI, no public exposure. It shells nothing out on the hot path: it opens the worker socket and relays. Three details matter:

- **Batch-aware run metadata.** A "run" is one JSON sidecar holding a *list* of images, `images:[{seed,file,ok,nans,timing}]`, so a batch of 4 is one run with four entries. Older single-image runs are normalised on read, so the schema change is backward-compatible.
- **One GPU at a time.** A `threading.Lock` serialises GPU jobs across web requests; the worker's single accept loop is the second line of defence.
- **Graceful degradation.** If the worker is down, the app falls back to spawning the one-shot `generate` binary (slower, but it works), and records which mode was used.

The same endpoints serve txt2img, remix (img2img), inpaint, upscale, and photo upload — all of which are just different request payloads to the *same* worker (Chapter 27).

28.5 Streaming the image as it forms

The feature that makes the whole thing feel alive: while the denoise runs, the browser shows the image **forming**. The mechanism has three layers.

Worker side. When a request sets `"stream":true`, the denoise loop decodes a preview every ~`n_run/4` steps and emits it as a progress line:

```
{"event":"progress","step":4,"total":8,"w":384,"h":384,"img_b64":"…"}
```

then the normal final line. The preview is produced by `preview_b64`, which decodes the **in-progress** latent without disturbing the run. The trick is buffer hygiene: it reuses the scratch buffers that are *free between denoise steps* (`d_velocity`,

`d_latent`, `d_pixels`) and **never touches** `d_tokens` (the live latent). It runs the same bn-denorm → unpatchify → VAE-deco de tail as the final image, then downsamples to 384px and encodes **JPEG q82** — ~40 KB, versus ~500 KB for PNG, a 10× saving that matters over a phone link. Previews are spac ed (≈4 per run) because each is a full VAE decode; spacing keeps the overhead small.

Relay. Flask's `/generate_stream` is a streaming `application/x-n djson` response: a generator that holds the GPU lock, sends `st ream:true` to the worker, and **re-emits each worker line** to the browser, tagging an `img`/`nimg` index so a batch can report "image 2/4". On the final line it saves the run metadata and emits `{"event":"complete","rid":…}`.

Browser. `streamSubmit` POSTs the form with `fetch`, reads the response with a `ReadableStream` reader, splits on newlines, and on each `progress` event sets an `<img>` to `data:image/jpeg;base6 4,…`. On `complete` it navigates to the result page. No EventSource, no WebSocket — plain `fetch` streaming, which modern Safari supports.

There is a sharp edge here worth flagging (Appendix B.16): `st reamSubmit` is `async`, so it returns a **Promise**. Wiring it as `onsub mit="return streamSubmit(this)"` returns a *truthy* value, which does **not** cancel the form's native submit — the browser then POSTs to the endpoint itself and *downloads the raw NDJSON file*. The fix is to cancel default synchronously (`onsubmit="stre amSubmit(this); return false"`). A reminder that `async` function s and `onsubmit`'s truthiness contract do not mix.

28.6 Robustness: don't let a client kill the GPU

Streaming introduced a failure mode one-shot serving never had: the worker now **writes to the client repeatedly**

during a job, so if the browser disconnects mid-stream, the next `write` hits a closed socket and the OS raises `SIGPIPE` — whose default action is to *terminate the process*. A user closing a tab was killing the GPU worker (observed as exit 141). One line fixes it — `signal(SIGPIPE, SIG_IGN)` — after which a dead client just makes `write` return `EPIPE`, the job finishes, and the worker keeps serving (Appendix B.17). The general lesson: any long-lived server that streams to clients must ignore `SIGPIPE`.

28.7 Where this lives in the code

Concept	Code
persistent worker	tools/serve.cu (Worker, Pipeline)
resident vs rebuilt state	Worker::ensure(res, precision)
line protocol	serve.cu accept loop (read_line, JSON in/out)
mid-denoise preview	Worker::preview_b64 (reuses free buffers, JPEG q82)
web UI	tools/webui/app.py (Flask, login, batch metadata)
streaming relay	/generate_stream (NDJSON generator)
browser streaming	streamSubmit (fetch + ReadableStream)
launcher	tools/webui/start.sh, *.service units

28.8 Summary and what to carry forward

• A **persistent worker** amortises the ~16 s model load: Qwen + tokenizer resident forever, transformer + VAE cached by `(res, precision)` and rebuilt only on change. Same-config i mage ~8 s vs ~25 s one-shot.

• The interface is **newline-delimited JSON over localhost TCP**; the GPU server never touches the network. The web layer authenticates, serialises the GPU, and degrades to the one-shot binary if the worker is down.

• **Live preview** decodes the in-progress latent every `~n/4` steps into a 384px JPEG, reusing the *free* scratch buffers and leaving `d_tokens` untouched, and streams it as NDJSON consumed by browser `fetch`/`ReadableStream`.

• Two serving-specific bugs: an `async-onsubmit` truthiness

trap that **downloaded** the stream (B.16), and an unhandled `SIGPIPE` that let a disconnecting client kill the worker (B.17). Both are one-liners; both only appear once you stream.

That closes **Part VIII** — and the implementation arc of the course. From a text prompt we now reach a 2K PNG, from an image we reach a restyle or an inpaint, and from a phone we watch it happen live — all on one Blackwell GPU, in C++/CUDA, validated against diffusers at every seam. The appendices that follow are the reference layer: the glossary, the **bug museum** (which the last three chapters fed three fresh entries), and the build-and-reproduce guide.

Exercises

1. **The resident seam.** Justify keeping Qwen resident but rebuilding the transformer on a resolution change. What in the transformer's configuration actually depends on `res`, and what would it cost to cache *several* `(res, precision)` pipelines at once?
2. **Preview buffer hygiene.** `preview_b64` reuses `d_velocity`/`d_latent`/`d_pixels` but not `d_tokens`. Explain why each of those three is safe to clobber *between* denoise steps, and what would break if the preview used `d_tokens` as scratch.
3. **Why JPEG, why spaced.** Estimate the bytes streamed for a 16-step run at PNG-every-step versus JPEG-every-4-steps, and explain the two independent reasons (encode cost, link bandwidth) the implementation chose the latter.
4. **The two streaming bugs.** Both B.16 (download) and B.17 (`SIGPIPE`) are invisible until you stream and only appear at a client boundary. Generalise: what class of bug does adding a *streaming* interface to a request/response server tend to expose?

Next: the appendices — A — Glossary, B — The bug museum, C — Build & reproduce.

Appendix A — Glossary

Terms and acronyms used in the course, with the chapter that introduces each.

Models & math

- **Diffusion model** — generative model that learns to reverse a noising process; samples by denoising from noise. (Ch 1)
- **Score** — $\nabla_x \log p(x)$, the gradient of log-density; the quantity a diffusion model implicitly learns. (Ch 1)
- **Tweedie's formula** — relates the optimal denoiser (posterior mean of clean given noisy) to the score. (Ch 1)
- **DSM** — denoising score matching; the training objective equivalence that makes diffusion trainable. (Ch 1)
- **Flow matching** — learn a velocity field whose flow transports noise to data; the framework FLUX trains with. (Ch 2)
- **Rectified flow** — flow matching with straight-line conditional paths; constant velocity target x_1-x_0. (Ch 2)
- **Probability-flow ODE** — deterministic ODE with the same marginals as the reverse diffusion SDE; sampling = integrating it. (Ch 1–2)
- **Euler step** — first-order ODE integration step; `latent += dt·v`. (Ch 2)
- **Distillation** — training a student to match many teacher steps in few; why klein needs 4 steps. (Ch 2)
- **Dynamic shift** — resolution-aware remapping of the timestep schedule. (Ch 2)
- **VAE** — variational autoencoder; compresses image↔latent. (Ch 3)
- **ELBO** — evidence lower bound; the VAE training objective. (Ch 3)

• **Latent** — the compressed representation diffusion operates in ([32,H/8,W/8]). (Ch 3)

Architecture

• **MMDiT** — Multi-Modal Diffusion Transformer; FLUX's two-stream architecture. (Ch 4)

• **Token** — a vector the transformer processes (image patch or text vector). (Ch 4)

• **Patchify** — pack a 2×2 latent block into one 128-dim token. (Ch 3)

• **Double-stream block** — separate text/image weights, joint attention; ×8. (Ch 8)

• **Single-stream block** — unified weights, fused parallel attn+MLP; ×24. (Ch 9)

• **Joint attention** — attention over concatenated `[txt‖img]` tokens. (Ch 8)

• **RoPE** — rotary position embedding; rotate Q/K by position-dependent angles. (Ch 6)

• **4-axis RoPE** — RoPE partitioned into (T,H,W,L) axes for 2D images + text. (Ch 6)

• **AdaLN / modulation** — adaptive layer norm; per-block scale/shift/gate from the timestep. (Ch 7)

• **QK-norm** — per-head RMS-normalization of Q and K before attention. (Ch 8)

• **SwiGLU** — gated MLP activation, SiLU(g)· u. (Ch 8, 15)

• **RMSNorm / LayerNorm / GroupNorm** — normalizations (Ch 15).

• **GQA** — grouped-query attention; fewer KV heads than query heads (Qwen3: 32/8). (Ch 21)

• **Context embedder** — Linear projecting `[512×12288]` conditioning → hidden 4096. (Ch 5)

GPU / numerics

• **GB10 / Blackwell / sm_121** — the target GPU and its compute capability. (Ch 11)

• **Unified memory** — coherent CPU+GPU memory; no PCIe copy of weights. (Ch 11, 14)

• **Roofline** — performance bound min(P_max, B_max· I). (Ch 11)

• **Arithmetic intensity (I)** — FLOPs per byte of HBM traffic. (Ch 11)

• **Ridge point** — I_ridge=P_max/B_max; memory- vs compute-bound boundary. (Ch 11)

• **Memory-bound / compute-bound / occupancy-bound** — which ceiling limits a kernel. (Ch 11, 17)

• **BF16** — 16-bit float, E8M7; activation baseline. (Ch 12)

• **FP8 E4M3 / MXFP8** — 8-bit float; MXFP8 = E4M3 values + per-32 UE8M0 scales. (Ch 12)

• **NVFP4 / E2M1** — 4-bit float (values 0,.5,1,1.5,2,3,4,6) + per-16 E4M3 scales. (Ch 12)

• **Block / microblock scaling** — per-small-group scales so low precision survives outliers. (Ch 12)

• **Quantization** — mapping high-precision values to a low-precision grid + scale. (Ch 12)

• **GEMM** — general matrix multiply; what every `Linear`/conv reduces to. (Ch 13, 19)

• **Tensor core / MMA** — hardware matrix-multiply-accumulate unit/instruction. (Ch 13, 18)

• **CUTLASS / CuTe** — NVIDIA's GEMM template library / its tensor-layout layer. (Ch 13)

• `mma.sync` — raw tensor-core MMA PTX instruction (m16n8k16 here). (Ch 18)

• `ldmatrix` — cooperative shared-memory→fragment load PTX. (Ch 18)

• **Fragment** — a thread-distributed tile of a matrix in

registers. (Ch 18)

- **Occupancy / CTA / SM** — resident blocks per streaming multiprocessor. (Ch 11, 17)
- **Online softmax** — single-pass softmax with running max/sum/output. (Ch 16)
- **Flash attention** — tiled attention that streams K/V once, never materializing P. (Ch 16–17)
- **cuDNN graph API** — the modern cuDNN interface reaching Blackwell conv engines. (Ch 19)
- **NHWC / NCHW / KRSC** — tensor/filter memory layouts. (Ch 19)

Project-specific

- **F2K (`.f2k1`)** — the custom mmap-and-go weight container. (Ch 14)
- **TensorRouter** — maps tensor names → structural roles. (Ch 14)
- `Linear` — the project's quantized linear layer over `FP4Gemm`/`FP8Gemm`. (Ch 13)
- `FlowMatchScheduler` — the sampler schedule (`flux2_dynamic`). (Ch 2)
- `generate` — the end-to-end CLI: prompt → PNG. (Ch 23)
- `f2k_convert` — offline safetensors → quantized F2K. (Ch 12, 14)
- `bench_attention` / `mma_unit` — the attention benchmark / PTX-layout validator. (Ch 17, 18)
- `cmp_transformer` / `*_golden` / `diffusers_*` — reference-comparison tools. (Ch 25)
- **Capture layers** `{8,17,26}` — Qwen3 layer outputs concatenated into conditioning. (Ch 5, 21)
- **Zero-gate identity test** — bit-exact wiring test using AdaLN-Zero. (Ch 8, 25)

Appendix B — The Bug Museum

> Every nontrivial bug from the port, collected as **symptom → cause → fix → lesson**. These are referenced throughout the course; gathered here, they form a field guide to the failure modes of porting a large model to a new GPU. The recurring theme: the worst bugs **don't crash** — they produce plausible-but-wrong output, and are found only by diffing against a reference.

B.1 The modulation scale/shift swap (the big one)

• **Symptom.** Images were coherent but barely prompt-dependent: cat vs truck differed 0.12% in pixels; two seeds differed ~10%. The prompt was throttled ~40:1.

• **Cause.** `ModulationMLP` sliced every `Flux2Modulation` output as `[scale, shift, gate]`; diffusers emits `[shift, scale, gate]`. All 32 blocks computed `(1+shift)·x+shift'` instead of `(1+scale)·x+shift`, corrupting AdaLN — the only channel through which the prompt modulates the network.

• **Fix.** Swap the slice order to `[shift, scale, gate]` for img/txt/single; keep `norm_out` **scale-first** (it's `AdaLayerNormContinuous`, a different convention).

• **Result.** Prompt influence 0.12% → **12.5%** (~100×), now dominant over the latent.

• **Lesson.** The order of packed sub-tensors is a silent contract. A swap doesn't crash or NaN — diff *conventions* against the reference. (Ch 7, 25)

B.2 VAE missing post_quant_conv (the "mush")

• **Symptom.** Soft, mushy images even with a correct

transformer.

• **Cause.** diffusers runs `z = post_quant_conv(z)` (a 1×1 conv, 32→32) *before* the decoder. It is a **sibling** of the `decoder.` prefix, so a decoder starting at `conv_in` skips it.

• **Fix.** Fold it into `VAEDecoder` (`Config::post_quant_conv_name`), applied at the top of `forward`; skip gracefully if absent.

• **Localized by.** Feeding diffusers' ground-truth latent via `--decode_latent` → still mush, proving the fault was in the VAE, not the transformer trajectory.

• **Lesson.** Reference pipelines hide steps in unobvious places (a sibling conv). Inject a known-good intermediate to localize. (Ch 3, 20, 25)

B.3 VAE missing batch-norm de-normalization (the patch-grid)

• **Symptom.** A hard 2×2 checkerboard across the whole image.

• **Cause.** diffusers standardizes the patch latent with a per-channel BatchNorm and inverts it (`z·√(var+ε)+mean`) before decode; this was skipped. Each patch's four sub-pixels live in different channels with different stats → mis-scaled → grid.

• **Fix.** Load `bn.running_{mean,var}` and apply the affine to the 128-channel patch latent before `unpatchify`.

• **Proved structural by.** Running 28 steps left the grid unchanged (not undercooking).

• **Lesson.** Distinguish *structural* bugs from undercooking with a step-count sweep; diff at the latent boundary. (Ch 3, 20, 25)

B.4 Patchify channel order

• **Symptom.** Coherent but subtly wrong images.

• **Cause.** Packed patch channels as `idx = ph·p·C + pw·C + c` (channel-fastest); diffusers uses `idx = c·p·p + ph·p + pw`

(channel-slowest). Each token's 128 numbers were permuted relative to the weights.

- **Fix.** Use the channel-slowest packing in `patchify.cu`.
- **Lesson.** Layout conventions in data-movement kernels are contracts; a permutation degrades rather than destroys, which makes it sneaky. (Ch 3)

B.5 The NVFP4 CuTe scale-layout trap

- **Symptom.** `Linear` gave correct output on structured inputs (all-ones, K-constant) but **uncorrelated** output on random weights.
- **Cause.** Hand-rolled FP4 byte index `byte_idx = k·(N/2)+(n/2)` for column-major B assumed flat packing; CUTLASS uses a tiled/swizzled layout. The formula matched only column 0, so CUTLASS re-read column-0 bytes for every k — fine for K-constant weights, catastrophic for real ones.
- **Fix.** Write through CuTe: `cute::recast_ptr` + `cute::make_tensor(ptr, layout)`, assign through the tensor.
- **Lesson.** Never hand-roll sub-byte indexing under a CUTLASS layout. And **component tests must use random inputs** — structured tests hid this for a while. (Ch 13, 25)

B.6 Capture-layer off-by-one ({9,18,27} → {8,17,26})

- **Symptom.** Conditioning subtly off; plausible-but-wrong images.
- **Cause.** diffusers requests `hidden_states[9,18,27]`, but HF's `hidden_states[0]` is the embedding and `hidden_states[k]` is the output of layer k-1. The real layer outputs are `{8,17,26}`.
- **Fix.** `capture_layers = {8,17,26}` in the encoder config.
- **Lesson.** Know your reference's indexing convention exactly. The same off-by-one also traps golden comparisons (B.7). (Ch 5, 21)

B.7 HF hidden_states post-norm trap

• **Symptom.** Encoder golden comparison cos dropped to ~0.69 at the last layer.

• **Cause.** Comparing our layer-N-1 output to HF `hidden_states[N]`, which is the *post-final-norm* output, not the layer output — a gamma rotation apart.

• **Fix.** Compare against `hidden_states[N-1]`.

• **Lesson.** Validate against the *right* reference tensor; a low cos can be a comparison bug, not a model bug. (Ch 21)

B.8 Text-stream RoPE missing

• **Symptom.** Image stream worked; prompt conditioning weaker than it should be.

• **Cause.** RoPE was applied only to image Q/K; the text stream's Q/K were left unrotated in the double-stream blocks.

• **Fix.** Apply 4-axis RoPE to **both** streams (text positions (0,0,0,ell)).

• **Lesson.** Symmetric components need symmetric treatment; and the header comment ("text gets NO RoPE") went **stale** after the fix — *verify code over comments*. (Ch 6, 8)

B.9 Noise init: uniform vs Gaussian

• **Symptom.** Off-manifold, degraded images.

• **Cause.** Initial latent drawn `uniform(-1,1)`; rectified-flow models start from the ODE's noise endpoint N(0, I).

• **Fix.** Draw the initial latent standard-normal.

• **Lesson.** The sampler's start distribution must match what the field was trained to transport. (Ch 2, 23)

B.10 cuDNN 8.9 link + legacy API (the 170× conv)

• **Symptom.** A 128-ch 1024² 3×3 conv took ~510 ms (vs

PyTorch's ~3 ms); VAE decode 18.7 s.

• **Cause (two).** (1) CMake linked the unversioned `libcudnn.so` → cuDNN **8.9** (pre-Blackwell, no sm_121 conv engines). (2) Even on cuDNN 9 the **legacy API** never reaches the fast engines — they require the **graph API + NHWC**.

• **Fix.** Link `libcudnn.so.9`; rewrite `conv2d.cu` to the graph API with NHWC/KRSC, taking heuristic `config[0]`.

• **Result.** VAE decode 18.7 s → **1.35 s**.

• **Lesson.** Performance is a property of the whole dispatch path. Benchmark new dependency paths against a reference — correctness tests won't reveal a wrong-kernel dispatch. (Ch 19)

B.11 CMake CUDA arch auto-probe (sm_75 instead of sm_121)

• **Symptom.** Arch-conditional MMA instructions failed at runtime ("MMA instruction used without targeting appropriate compute capability").

• **Cause.** `enable_language(CUDA)` auto-probes and writes `CMAKE_CUDA_ARCHITECTURES` to the cache (defaulting to Turing 75 here), so a later `if(NOT DEFINED)` guard never takes effect — the binary was built for sm_75.

• **Fix.** `set(CMAKE_CUDA_ARCHITECTURES "121a" CACHE STRING ... FORCE)` **before** `enable_language(CUDA)`; wipe `build/` after changing.

• **Lesson.** Pin the GPU arch with FORCE before enabling the CUDA language; a silent arch mismatch compiles fine and fails at the MMA. (Ch 13, App C)

B.12 regO out-of-bounds write (partial tile)

• **Symptom.** Potential corruption / OOB at non-tile-aligned sequence lengths.

• **Cause.** The register-O attention kernel's final normalize/write loop wrote all 16 rows of a query tile unconditionally; for a partial last tile ($q_0+r \geq S$) that writes past the end of O.

• **Fix.** Guard with `if (q0 + r < S)` before the write.

• **Lesson.** Tiled kernels need partial-tile guards on *every* global write, not just the loads; test non-aligned shapes (S=130). (Ch 17)

B.13 Manual-load mma slower than regO

• **Symptom.** A correct, higher-occupancy `mma.sync` attention kernel ran *slower* (42.3 ms) than the regO kernel it replaced (34.3 ms).

• **Cause.** Fragments were filled with manual scalar `pack2` reads from shared memory — 96 bank-conflict-prone reads per tile, costing more than the rescale tax removed.

• **Fix.** Use `ldmatrix.x2` (conflict-free cooperative load) → 19.7 ms at peak BW.

• **Lesson.** Occupancy is necessary, not sufficient; the smem *access pattern* matters. Correctness-first then optimize, but know the optimization (ldmatrix) is required to win. (Ch 17, 18)

B.14 The minor ones

• `seq_txt` **128 → 512.** Text sequence was 128; diffusers uses 512 (`max_sequence_length`). (Ch 5)

• **Stale doc comments.** Beyond B.8, the `conv2d.h` and `rope.cu` headers had comments that lagged the code. *Verify against the* `.cu`*.* (Ch 6, 8, 19)

• **Hex-literal typos.** Repeated attempts to spell words in hex test seeds (`0xS1NG...`) don't compile — half the alphabet isn't a hex digit. Use real hex (`0xCAFEBABE`) or a number.

B.15 The gridDim.y overflow at 65 536 (the "2K needs too much memory" red herring)

• **Symptom.** Anything at **2048px** — native generation, img2img encode, upscale — failed, inside `Linear` at the VAE mid-attention's `to_q` ("activation quant launch failed"). 1536p x was fine. It read like an out-of-memory wall at 2K.

• **Cause.** Not memory — a **launch-config limit**. `Linear`'s activation-quant and bias-add kernels launched with `grid.y = M` (the row count). CUDA caps `gridDim.y` **at 65 535**. The VAE spatial attention runs at S=(res/8)² tokens, so M=65536 at *ex actly* 2048px (and 36 864 at 1536 — just under). An off-by-one against a hardware ceiling, masked by a memory-shaped guess.

• **Fix.** Carry rows past 65 535 onto `grid.z`: `row = blockIdx.y + blockIdx.z*gridDim.y`, launched as `dim3(n, min(M,65535), ceil(M/65535))`. A no-op for M<65536 (`gz=1`), so zero risk to the common path.

• **Result.** Native **2048 (true 2K)** works for encoder, decoder, and generation; the encoder golden gained its 2048 row (cos **0.99960**).

• **Lesson.** Hardware launch-config limits (`gridDim.{y,z} ≤ 65535`) are invisible until a dimension crosses them. Unlike most museum bugs this one *crashed* — but the crash's neigh bourhood ("memory at 2K") misdirected; the real tell was that it failed at one exact resolution. Suspect grid shape when a kernel dies at a round power-of-two size. (Ch 13, 26, 27)

B.16 The streaming form that downloaded itself

• **Symptom.** Tapping Generate / Remix / Inpaint **downloa ded a file** `generate_stream.ndjson` instead of showing the live preview.

• **Cause.** `streamSubmit` is an `async` function, so it returns a **Promise**. Wired as `onsubmit="return streamSubmit(this)"`, the handler returned a *truthy* Promise, which does **not** cancel a form's native submit — the browser POSTed to the streaming endpoint and saved its NDJSON body as a download. The error fallback then called `form.submit()`, causing the same download a second way.

• **Fix.** Cancel the default **synchronously**: `onsubmit="streamSubmit(this); return false"` (and `inpaintSubmit` returns `false` after firing the stream). Drop the native-submit fallback; alert on error instead.

• **Lesson.** `async` functions and `onsubmit`'s return-value contract don't mix — a Promise is truthy, and only a *synchronous* `return false` prevents default. (Ch 28)

B.17 SIGPIPE — a closed tab killing the GPU

• **Symptom.** The persistent worker exited (code **141**) when a browser disconnected during a streaming job.

• **Cause.** Streaming makes the worker **write to the client repeatedly** mid-job. A disconnected client means the next `write` hits a closed socket, and the OS raises `SIGPIPE`, whose default action *terminates the process*. Closing a tab killed the server.

• **Fix.** `signal(SIGPIPE, SIG_IGN)` at startup; `write` then returns `EPIPE`, the job finishes, the worker keeps serving.

• **Lesson.** Any long-lived server that *streams* to clients must ignore `SIGPIPE`. The bug cannot occur in plain request/response (one write, at the end) — adding a streaming interface created a new failure class at the client boundary. (Ch 28)

The patterns across all of them

1. **Plausible-but-wrong is the default failure mode.** Most of these produced coherent images, not crashes. Eyeballing the output is not validation.
2. **Convention mismatches dominate.** Slice order (B.1), layer indices (B.6, B.7), packing order (B.4), RoPE pairing/streams (B.8), layout (B.5) — porting is mostly matchi ng conventions, and conventions are invisible until you diff.
3. **Inject a known-good intermediate.** B.1 (`--embeds`) and B.2 (`--decode_latent`) were each localized in one run by replacing our intermediate with the reference's.
4. **Diff at internal boundaries, not just the image.** Latents (B.2, B.3), per-layer hidden states (B.6, B.7), per-tensor outputs (B.5) — the closer to the bug you diff, the faster you find it.
5. **Distinguish noise from bugs.** Quantization (cos 0.963), reordered-summation (1.8% pixel), and BF16 rounding are *expected*; vary precision / compare to FP32 to tell them from regressions. (Ch 25)
6. **Performance bugs are silent too.** B.10, B.11 produced *correct* output at terrible speed; only a reference *nu mber* revealed them.
7. **Random inputs in tests.** B.5 hid behind structured tests; random data caught it.
8. **New interfaces create new failure classes.** The bugs that *do* crash cluster at boundaries the core never had: a hardware launch limit at a new resolution (B.15), and a brows er/socket boundary that only exists once you stream (B.16, B.17). When you add a dimension — bigger images, a streaming client — re-test the *edges* of it, not just the happy path.

Appendix C — Build & Reproduce

> Everything needed to build the project, run the tests, and reproduce the numbers and images cited in the course. Commands assume the repo root and the GB10 environment described in Chapter 11.

C.1 Toolchain

component | version | notes
GPU | NVIDIA GB10 (Blackwell), sm_121 | the only supported target here
CUDA | 13.0 (sbsa-linux), nvcc V13.0 | aarch64 SBSA
cuDNN | 9 (libcudnn.so.9) | must link 9, not the .so→8.x symlink (App B.10)
CUTLASS | vendored submodule | third_party/cutlass
compiler | g++ 13 | aarch64
CMake | ≥ 3.28 |
Python (tools only) | .venv with torch 2.12+cu130, transformers 5.10, diffusers 0.38 | reference dumpers + (legacy) tokenization

The runtime is fully native C++/CUDA; Python is only for the offline reference tools of Chapter 25 (`.venv/bin/python tools/...`).

C.2 Submodules

The vendored dependencies are git submodules (`third_party/`): `cutlass`, `glfw`, `imgui`, `stb`, `json`. After cloning:

```
git submodule update --init --recursive
```

(The submodule *contents* are not in the repo — only references — so this step is required before building. See the push note in the project history.)

C.3 The CMake gotchas (read before building)

Two settings are load-bearing (both are war stories — App B):

1. **Pin the CUDA arch with FORCE before** `enable_languag`

e(CUDA) (App B.11):Without this, the auto-probe defaults to Turing (sm_75) and the block-scaled MMA fails at runtime. After changing it, **wipe** `build/` (the cache value sticks).

2. **Link cuDNN 9 explicitly** (App B.10): link `/usr/lib/aarch64-linux-gnu/libcudnn.so.9`, not the unversioned `libcudnn.so` dev symlink (which may point at a pre-Blackwell 8.x).

C.4 Build

```
cmake -S . -B build
cmake --build build -j
```

Targets of interest: `generate` (the CLI), `bench_attention`, `f2k_convert`, and the `test_*` suite. `tools/mma_unit.cu` builds standalone for fast PTX iteration:

```
nvcc -arch=sm_121a -o /tmp/mma_unit tools/mma_unit.cu && /tmp/mma_unit
```

C.5 Models on disk

The runtime expects converted F2K weights under `~/models/flux2-klein-9B/`:

```
 transformer_f2k/   shard-0000{1,2}.f2k1     # NVFP4 transformer (--precision nvfp4)
 transformer_mxfp8/ shard-0000{1,2}.f2k1     # MXFP8 transformer (--precision fp8)
 qwen3_f2k/         shard-{00001..00004}.f2k1# NVFP4 Qwen3 encoder
 vae_f2k/           vae.f2k1                  # VAE decoder weights
 tokenizer/         vocab.json, merges.txt, added_tokens.json, chat_template.jinja, ...
```

Produce them from a HF download with `f2k_convert` (Chapter 12/14):

```
./build/f2k_convert <safetensors...> --quant nvfp4 -o .../transformer_f2k/...
./build/f2k_convert <safetensors...> --quant mxfp8 -o .../transformer_mxfp8/...
# Qwen3: --keep-bf16 embed_tokens ; VAE: passthrough/quant as configured
```

C.6 Generate an image

```
# native, single call (no Python):
./build/generate --prompt "a cat on a skateboard" \
                 --res 1024 --precision fp8 --steps 4 --seed 0xCAFEBABE \
                 --out cat.png

# precomputed token ids (skip the native tokenizer):
.venv/bin/python tools/encode_prompt.py --seq 512 "a cat on a skateboard" /tmp/tok.bin
./build/generate --tokens /tmp/tok.bin --res 1024 --precision fp8 --out cat.png
```

```
# precomputed diffusers conditioning (skip encoder too — for transformer isolation):
.venv/bin/python tools/diffusers_prompt_embeds.py --prompt "a cat..." --out /tmp/emb.bin
./build/generate --embeds /tmp/emb.bin --res 1024 --precision fp8 --out cat.png
```

PPM output is the default; an `.png` extension writes PNG (Chapter 22). PPM→PNG if needed: `.venv/bin/python -c "from PIL import Image; Image.open('out.ppm').save('out.png')"`.

`--res` now accepts up to **2048** (true 2K, Chapter 27); 2K is correct but slow (~80–90 s, Chapter 24).

Serve it (persistent worker + web UI, Chapter 28)

```
# one-time: build the worker target
cmake --build build --target serve

# launch worker (resident models) + Flask UI; reach it over Tailscale
F2K_WEB_PASSWORD=yourpw tools/webui/start.sh        # → http://<host>:5000
```

The worker holds the models resident (~8 s/image vs ~25 s one-shot) and serves txt2img, img2img, inpaint, upscale, and photo upload — with **live preview** streaming as the image forms. `init_image`/`mask_image`/`strength` are worker request fields (the image-conditioning paths of Chapter 27), not `generate` flags.

C.7 Run the tests

```
cd build && ctest                    # the whole suite
ctest -R 'attention|transformer|tokenizer|denoise'   # a subset
```

Each component has a golden test vs an FP32 (or HF/diffusers) reference (Chapter 25). Expected: all green; `test_tokenizer` prints `ALL OK`; `test_attention` shows `cos=1.00000` across shapes including the non-aligned `S=130`.

C.8 Reproduce the course's numbers

number (chapter) | command
GEMM peaks 252/161 TFLOP/s (11, 13) | ./build/test_fp4_gemm / ./build/test_fp8_gemm
attention 56.5→34.3→19.7 ms, 279 GB/s, CTAs/SM (11, 17) | ./build/bench_attention

```
mma/ldmatrix layout validation (18) | /tmp/mma_unit (see C.4)
transformer cos 0.963/0.999 vs diffusers (10, 12, 25) |
./build/cmp_transformer <embeds> <sigma> {nvfp4,fp8}
Qwen3 encoder cos ≥0.974 vs HF (21, 25) | tools/qwen3_golden.py +
./build/test_qwen_golden
FP4-vs-FP8 quality 12.95%/2.12% (12) | .venv/bin/python tools/quant_sim.py
VAE decode timing, decode diffusers latent (19, 20, 25) |
./build/test_vae_decoder; generate --decode_latent
VAE encoder cos 0.9997 vs diffusers (26) | tools/diffusers_vae_encode_dump.py
--image X --size N; ./build/cmp_vae_encode /tmp/vae_enc_img.bin
/tmp/vae_enc_lat.bin
full timing ledger (0, 23) | ./build/generate --prompt ... --res 1024
--precision fp8 (reads the stderr timings)
```

`bench_attention` prints the device properties (SMs, opt-in smem, peak) at startup, so it doubles as the environment check.

C.9 The optimization knobs, recapped

flag	effect	chapter
`--precision {nvfp4,fp8}`	speed (NVFP4) ↔ quality (MXFP8)	12
`--res N`	output resolution (N%16==0, seq_img%128==0)	3, 13
`--steps N`	sampler depth (4 default; more = sharper detail)	2
`--seed N`	reproducible noise; reroll composition	23
`--out *.png`	PNG via stb; else PPM	22
`--prompt/--tokens/--embeds/--decode_latent`	conditioning path / isolation	23, 25

C.10 Where to go next

- To **understand** the pipeline: read Parts I–II (math + architecture).
- To **modify a kernel**: Parts III–IV, with `bench_attention`/`mma_unit` as your instruments and `test_*` as your safety net.
- To **add a feature** (new precision, resolution, sampler): the `Linear::Precision` enum (Ch 13), the `--res` derivation (Ch 23), and `FlowMatchScheduler` (Ch 2) are the seams.
- To **debug a wrong image**: Chapter 25's isolation method + Appendix B's field guide.

That is the whole system — from rectified-flow theory to a `mma.sync`+`ldmatrix` kernel at peak HBM bandwidth, building and

reproducible end to end.